

Context Engineering

The Discipline Behind Effective AI Software Development

Alexander Mikhalev

2026-01-01

Table of contents

- 1. Context Engineering 1
- 2. Introduction 3
 - 2.1. Why This Book, Why Now 3
 - 2.2. What Is Context Engineering 4
 - 2.3. What You Will Learn 5
 - 2.4. How to Read This Book 6
 - 2.5. A Note on Sources 6
 - 2.6. The Journey Ahead 7
- 3. Chapter 1: Why AI Software Projects Fail Differently 9
 - 3.1. The Traditional Software Failure Model 9
 - 3.2. The New Failure Model: Context Drift 10
 - 3.2.1. Enter Context Engineering 11
 - 3.3. Hallucination as Feature, Not Bug 12
 - 3.3.1. Narratives Without Explanatory Power 13
 - 3.4. The Illusion of Simplicity 14
 - 3.5. Case Studies: Context Collapse in the Enterprise . . 15
 - 3.5.1. The Finance Example 15
 - 3.5.2. The Legal Example 16
 - 3.6. Context Complexity vs. Code Complexity 16
 - 3.7. The Concept Modelling Crash Course 17
 - 3.7.1. Entities 17
 - 3.7.2. Relationships 18
 - 3.7.3. Constraints 19
 - 3.8. The Verification Bottleneck 19
 - 3.8.1. The Economics 19

3.8.2. The Missing Junior Loop	20
3.8.3. Why This Matters for Context Engineering . .	20
3.9. Summary: The New Failure Landscape	21
3.10. Key Sources	22
4. Chapter 2: What “Context” Actually Means in AI Development	23
4.1. Beyond the Chat Window	23
4.2. The Four Dimensions of Context	24
4.2.1. 1. Domain Context	24
4.2.2. 2. Technical Context	25
4.2.3. 3. Operational Context	25
4.2.4. 4. Commercial Context	26
4.3. Context as the Intersection of Assumptions	26
4.4. Why Context Expands Faster Than It Can Be Structured	27
4.5. The Role of Stakeholder Reinterpretation	28
4.6. AI-Specific Context Variables	29
4.7. The Context Boundary Problem	30
4.8. Summary: A Multi-Dimensional Problem	30
4.9. Key Sources	31
5. Chapter 3: Context Engineering as a Discipline	33
5.1. What Context Engineering Is Not	33
5.1.1. Not Prompt Engineering	33
5.1.2. Not “Vibe Coding”	34
5.1.3. Not Documentation	34
5.2. The Historical Roots	35
5.2.1. Engineering Control (1960s)	35
5.2.2. Domain-Driven Design (2000s)	35
5.2.3. Configuration Management	36
5.2.4. The Scientific Method of Context Building . .	36
5.3. Operational Discipline: The Karpathy Guidelines . .	37
5.3.1. 1. Think Before Coding	37
5.3.2. 2. Simplicity First	38
5.3.3. 3. Surgical Changes	38

5.3.4. 4. Goal-Driven Execution	38
5.4. The Zestic AI Framework	39
5.4.1. The Reconciled Domain Model	39
5.4.2. Artefacts as Decision Containers	40
5.4.3. Gated Lifecycle with Reconciliation	40
5.4.4. Separation of Account Management and Engineering	41
5.5. The Cost of Ignoring Context Engineering	41
5.6. Summary: A Necessary Discipline	42
5.7. Key Sources	43
6. Chapter 4: The Architecture of Meaning	45
6.1. Artefacts as Decision Containers	45
6.1.1. The Argumentation Structure of Decision Containers	47
6.1.2. Implementing Decision Containers	47
6.2. The Reconciled Domain Model as Anchor	48
6.2.1. What a Domain Model Contains	49
6.2.2. Reconciliation Process	49
6.3. Baselining and Version Control for Semantic Alignment	50
6.3.1. Semantic Versioning for Context	51
6.4. Separating Commercial Context from Technical Context	51
6.4.1. Commercial Context	52
6.4.2. Technical Context	52
6.4.3. Why Separation Matters	52
6.5. Why Each Lifecycle Stage Requires a Different Inference Context	53
6.5.1. The Four Stages	53
6.5.2. Why Context Must Change	54
6.5.3. Roles in Context Governance	54
6.6. Concept Modelling Fundamentals	55
6.6.1. Entity Types	55
6.6.2. Relationship Cardinalities	56
6.6.3. Invariants	56

6.7. Summary: Architecture Enables Discipline	57
6.8. Key Sources	57
7. Chapter 5: Knowledge Graphs Explained	59
7.1. What Is a Knowledge Graph?	59
7.1.1. The Basic Structure	59
7.1.2. Triples: The Atomic Unit	60
7.1.3. Real-World Example: SNOMED CT	61
7.2. Historical Context	61
7.2.1. Google's Knowledge Graph	61
7.2.2. Enterprise Ontologies	62
7.3. The Resurgence in the AI Era (GraphRAG 2024-2025)	62
7.3.1. Why Knowledge Graphs Matter for AI Context	63
7.4. Knowledge Graphs vs Vector Databases: Complementary Approaches	64
7.4.1. How Vector Databases Work	64
7.4.2. Strengths and Limitations	64
7.4.3. When to Use Each	65
7.5. The Role of Ontologies and Controlled Vocabularies	66
7.5.1. Why Controlled Vocabularies Matter for AI . .	66
7.5.2. The Thesaurus Pattern	66
7.6. Case Study: SNOMED CT in Healthcare AI	67
7.6.1. The Scale	67
7.6.2. The MedGemma Competition	68
7.7. Inference: The Hidden Power of Graph Structure . .	68
7.7.1. Transitive Reasoning	68
7.7.2. Inheritance	69
7.7.3. Path-Based Inference	69
7.8. From Knowledge Graphs to Context Graphs	70
7.8.1. The Key Insight: Structure Carries Meaning .	70
7.8.2. RDF, Property Graphs, and the Non-Argument	70
7.8.3. Mature Graph Algorithms: The Untapped Toolbox	71
7.9. Summary: Knowledge Graphs as Context Infrastructure	72

7.10. Key Sources	72
8. Chapter 6: Deterministic Search vs Vector Embeddings	75
8.1. How Vector Embeddings Work	75
8.1.1. The Process	75
8.1.2. The Embedding Problem	76
8.1.3. The Cost of Probabilistic Search	77
8.2. The Deterministic Alternative	77
8.2.1. Key Characteristics	78
8.3. Head-to-Head Comparison	79
8.4. Where Deterministic Wins	79
8.4.1. Healthcare and Safety-Critical Applications	79
8.4.2. Compliance and Audit	80
8.4.3. High-Throughput, Low-Latency	80
8.4.4. Edge and Offline	81
8.5. Where Embedding RAG Wins	81
8.6. The Hybrid Strategy	81
8.7. Summary: Two Tools, One Goal	82
8.8. Key Sources	83
9. Chapter 7: Terraphim Graph Embeddings: Deterministic Search with Full Explainability	85
9.1. Explicit Relationships vs Inferred Similarity	86
9.1.1. Why Explicit Beats Implicit	86
9.2. Deterministic Ranking: Same Query → Same Results	87
9.3. Explainable Results: Trace Exactly Why a Document Matched	88
9.4. No Information Loss: Full Graph Structure Preserved	89
9.5. Role-Based Graphs: Different Synonym Edges Per Context Role	90
9.5.1. The Problem with Single Vocabularies	90
9.5.2. Terraphim's Solution: Role-Based Graphs	90
9.5.3. The Formal Role Model	91
9.6. The Aho-Corasick Automata Approach	92
9.6.1. Why Aho-Corasick?	92

9.6.2. Precompiled Automata	93
9.7. How the Graph Scorer Works: Connectivity-Based Ranking	93
9.7.1. The Scorer Algorithm	94
9.7.2. What “Connectivity” Means	94
9.7.3. Why This Beats Vector Similarity	95
9.8. Integration with ProCoder for Semantic File Retrieval	95
9.8.1. The Semantic Retrieval Pipeline	96
9.8.2. Comparison: Manual vs Semantic	97
9.9. Role-Based Configuration in Practice	97
9.10. Summary: The Terraphim Approach	99
9.11. Key Sources	100
10. Chapter 8: Building a Knowledge Graph for Context	101
10.1. Defining Nodes: Concepts, Documents, Terms, Entities	101
10.1.1. Types of Nodes	101
10.1.2. What to Include: The Vocabulary-First Approach	102
10.1.3. The CTO Executive System Example	102
10.2. Defining Edges: Synonym Relationships, Hierarchical Relations	103
10.2.1. Types of Edges	104
10.2.2. Synonym Edges: The Foundation	104
10.2.3. Hierarchical Edges: Taxonomy	105
10.2.4. The Research Knowledge Graph Example	106
10.3. Schema Design: Base Schema + Project-Type Extensions	106
10.3.1. Base Schema	106
10.3.2. Project-Type Extensions	107
10.3.3. Why Schemas of Schemas: The Meta-Language Principle	108
10.4. Validation: JSON Schema with Local \$ref Resolution	109
10.4.1. JSON Schema Validation	109
10.4.2. Common Validation Rules	109
10.4.3. Local \$ref Resolution	110

10.5. Building the Thesaurus: Synonym Expansion Patterns	110
10.5.1. Pattern 1: Exact Synonyms	110
10.5.2. Pattern 2: Acronyms	110
10.5.3. Pattern 3: Spelling Variations	111
10.5.4. Pattern 4: Domain-Specific	111
10.5.5. Pattern 5: Related but Not Synonymous . . .	111
10.5.6. Building Process	111
10.6. Practical Example: CTO Executive System Knowledge Graph	111
10.6.1. Scale	112
10.6.2. Structure	112
10.6.3. Synonym Expansion	112
10.6.4. Validation Results	113
10.7. Concept Modelling in Practice: From Domain Model to KG Schema	113
10.7.1. Step 1: Define Entities	113
10.7.2. Step 2: Define Attributes	114
10.7.3. Step 3: Define Relationships	114
10.7.4. Step 4: Define Constraints	114
10.7.5. Step 5: Map to KG Schema	114
10.8. Summary: Building a Knowledge Graph	115
10.9. Key Sources	116
11. Chapter 9: Knowledge Graph Grounding as Safety Gate	117
11.1. The MedGemma Competition Case Study	117
11.1.1. The Problem: Raw LLMs Are Dangerous . . .	117
11.1.2. The Solution: Six-Stage Pipeline	118
11.1.3. The Results: 543 Tests, 18/18 Grounded . . .	118
11.2. How KG Grounding Prevents Hallucination at Scale .	119
11.2.1. The Pattern in Detail	119
11.2.2. Why This Works	120
11.2.3. Scale Considerations	120
11.3. The Gate Pattern: LLM Generates □ KG Validates □ Grounded Output	120
11.3.1. Gate Mode 1: Reject Invalid	121

11.3.2. Gate Mode 2: Correct Output	121
11.3.3. Gate Mode 3: Annotate and Warn	121
11.3.4. Gate Mode 4: Escalate	121
11.4. Explainable to Regulators: Deterministic Matching Traces	122
11.4.1. The Regulatory Challenge	122
11.4.2. The Terraphim Answer	122
11.4.3. Real-World Validation	123
11.5. Building the Validation Pipeline	123
11.5.1. Component 1: Entity Extraction	123
11.5.2. Component 2: Knowledge Graph Query	124
11.5.3. Component 3: Decision Logic	124
11.5.4. Component 4: Audit Trail	125
11.6. The Broader Pattern: Safety Gates Beyond Healthcare	125
11.6.1. Finance	125
11.6.2. Legal	126
11.6.3. Manufacturing	126
11.6.4. Any High-Stakes Domain	126
11.7. Summary: Grounding as the Safety Layer	126
11.8. Key Sources	127
12. Chapter 10: Token Budget Architecture	129
12.1. The 12,000 Token Rule and Progressive Disclosure	129
12.1.1. The Problem: Flat Context Loading	130
12.1.2. The Solution: Progressive Disclosure	130
12.2. Constitutional Context: What Never Gets Compressed	131
12.2.1. Core Constitutional Files	131
12.2.2. Constitutional Principles	131
12.3. Layered Loading: Skill Catalog □ Full SKILL.md □ Com- pression	132
12.3.1. Layer 1: Skill Catalog (Names Only)	132
12.3.2. Layer 2: Active Skills (Full Content)	132
12.3.3. Layer 3: Compression (Before LLM)	133
12.4. Token Tracking and Monitoring Systems	134
12.4.1. The Token Tracker	134

12.4.2. Monitoring Dashboard	135
12.4.3. Alerting	135
12.5. Allocation Per Context Type: The Budget Breakdown	135
12.5.1. Recommended Allocation (12,000 Token Budget)	135
12.5.2. Why This Works	136
12.5.3. Adjusting for Different Models	136
12.6. Summary: Budget as Infrastructure	137
12.7. Key Sources	137
13. Chapter 11: Compression Strategies	139
13.1. The Compression Priority Order	139
13.1.1.1. Deduplication: Remove Identical Content Across Skills	140
13.1.2.2. Summarization: Condense Examples, Troubleshooting, Verbose Explanations	141
13.1.3.3. Truncation: Cut from Oldest Non-Constitutional Content	142
13.1.4.4. Drop: LRU Eviction of Inactive Skills	144
13.2. Building a Compression Pipeline	145
13.2.1. The Compression Report	146
13.3. When to Compress: Pre-LLM vs Post-LLM	147
13.3.1. Pre-LLM Compression	147
13.3.2. Post-LLM Compression	148
13.3.3. Hybrid Approach	148
13.4. Summary: Systematic Compression	149
13.5. Field Evidence: Claude Code's Five-Layer Ladder	149
13.6. Key Sources	150
14. Chapter 12: Skill Architecture for AI Agents	151
14.1. Lightweight Skill Catalogs (~200-300 Tokens at Startup)	151
14.1.1. Catalog Structure	151
14.1.2. Token Cost Breakdown	152

14.2. On-Demand Activation Patterns: Trigger-Based Loading	153
14.2.1. How Triggers Work	153
14.2.2. Activation Threshold	154
14.2.3. Loading Full Content	154
14.3. Auto-Deactivation Rules (10 Turns of Non-Use) . . .	155
14.3.1. The 10-Turn Rule	155
14.3.2. Tracking Usage	156
14.3.3. Deactivation Process	156
14.4. Maximum 5 Active Skills Simultaneously	157
14.4.1. Why 5?	157
14.4.2. Enforcing the Limit	157
14.4.3. Priority When Over Limit	158
14.5. Building the Skill Taxonomy: Naming, Triggers, Meta- data	159
14.5.1. Naming Convention	159
14.5.2. Trigger Design	160
14.5.3. Metadata Schema	160
14.6. Summary: Skills as Context Modules	161
14.7. Key Sources	162
15. Chapter 13: CLAUDE.md and Project-Wide Rules	163
15.1. The Problem: AI Agents Don't Know Your Project's Rules	163
15.2. The Solution: CLAUDE.md	164
15.2.1. What CLAUDE.md Contains	164
15.2.2. Structure: Testing, Style, Documentation Re- quirements	165
15.3. Initialisation Files (INITIAL.md) for Feature Requests	167
15.3.1. What INITIAL.md Contains	167
15.3.2. How INITIAL.md Works	169
15.4. Examples Folder as the Source of Truth	169
15.4.1. Folder Structure	169
15.4.2. Using Examples in CLAUDE.md	170

15.5.The PRP (Product Requirements Prompt) Workflow Pattern	170
15.5.1.PRP Template	170
15.5.2.PRP Workflow	171
15.6.Beyond Claude Code: Applying Patterns to Other AI Assistants	172
15.6.1.Universal Template	172
15.7.Core Principle: Provide Full Screenplay, Not Sticky Notes	173
15.8.Summary: Project-Level Context	173
15.9.Key Sources	174
16.Chapter 14: Validation and Quality Gates	175
16.1.KLS (Knowledge Layer Schema) Validation Protocol .	175
16.1.1.What KLS Validates	175
16.1.2.KLS Schema Structure	176
16.1.3.KLS Workflow	177
16.2.Pre-LLM Validation Layers: Syntax, Semantics, Grounding	178
16.2.1.Layer 1: Syntax Validation	178
16.2.2.Layer 2: Semantic Validation	179
16.2.3.Layer 3: Grounding Validation	180
16.2.4.The Validation Pipeline	181
16.3.Drift Detection: Monitoring Context Changes Over Time	181
16.3.1.What Causes Drift	181
16.3.2.Drift Detection Metrics	182
16.3.3.Drift Alerts	183
16.4.Impact Analysis: What Changes Affect What	184
16.4.1.Dependency Graph	184
16.4.2.Impact Analysis Function	184
16.5.Integration with CI/CD Pipelines	185
16.5.1.CI Pipeline Stages	185
16.5.2.Git Hooks	186

16.6.LLM as a Judge for Context Validation	187
16.6.1.Four Evaluation Modes	187
16.6.2.Known Biases	188
16.6.3.The Hybrid Verification Pipeline	188
16.6.4.Calibration Against Human Judgement	189
16.6.5.When NOT to Use LLM Judges	190
16.7.Where Validation Criteria Come From: Context Engi- neering Meets Spec-Driven Development	190
16.7.1.The Two-Layer Model	191
16.7.2.The Swiss Cheese Model of Verification	193
16.7.3.BDD Specifications as the Bridge Artefact	195
16.7.4.Implications for the Validation Architecture	196
16.8.Summary: Validation as Gate	198
16.9.Key Sources	198
17.Chapter 15: Multi-Agent Context Management	201
17.1.Sub-Agent Orchestration Patterns in Claude Code	201
17.1.1.Spawn Patterns	201
17.1.2.Context Passing	203
17.2.Session Persistence and Memory Across Turns	203
17.2.1.Memory Types	204
17.2.2.Memory Management	204
17.2.3.Persistence Strategies	205
17.3.Context Isolation vs Shared Context Tradeoffs	206
17.3.1.Isolation Model	206
17.3.2.Shared Context Model	207
17.3.3.Hybrid Approach	208
17.4.Terraphim Knowledge Graph Integration for Agents	208
17.4.1.Agent-KG Integration	209
17.4.2.Shared Vocabulary	210
17.5.Build-Review Loops: Context-Walled Agent Coordi- nation	211
17.5.1.The Context Wall Principle	211
17.5.2.Model Selection Asymmetry	212
17.5.3.Automated Review Surfaces	212

17.6.The Meta-Cortex Concept: Agent-to-Agent Commu- nication	213
17.6.1.Meta-Cortex Architecture	213
17.6.2.Communication Patterns	215
17.6.3.Why Meta-Cortex Matters	216
17.7.Summary: Multi-Agent Context	216
17.8.Key Sources	216
18.Chapter 16: Hooks and Automation for Context	219
18.1.Session Hooks: Before/After LLM Calls	219
18.1.1.Before LLM Hook	219
18.1.2.After LLM Hook	220
18.1.3.Hook Pipeline	221
18.2.Tool Use Hooks: Validation, Logging, Correction	222
18.2.1.Pre-Tool-Use Hook	222
18.2.2.Post-Tool-Use Hook	223
18.2.3.Example: File Edit Validation	224
18.3.Claude Code Hooks: mcp-hook, pre-tool-use, post- tool-use	224
18.3.1.MCP Hook (Model Context Protocol)	225
18.3.2.Pre-Tool-Use Hook	225
18.3.3.Post-Tool-Use Hook	226
18.3.4.Hook Configuration	226
18.4.Context Mutation Warning: When Hooks Change What the LLM Sees	227
18.5.Continuous Learning from Agent Interactions	228
18.5.1.Learning Hook	228
18.5.2.Pattern Extraction	230
18.6.Building the Hook Taxonomy	230
18.6.1.Hook Categories	231
18.6.2.Hook Metadata	231
18.6.3.Hook Testing	232
18.7.Case Study: Cron-Driven Self-Improvement Digest	232
18.7.1.The Daily Digest Workflow	233
18.7.2.Why Scheduled Beats Real-Time	233

18.8.Summary: Automation Layer	234
18.9.Key Sources	234
19.Chapter 17: The OpenClaw Context System	235
19.1.Architecture Overview	235
19.2.Memory Architecture	236
19.2.1.Memory Loading Rules	236
19.3.Skill System	237
19.3.1.Skill Activation	237
19.4.HEARTBEAT.md: Autonomous Loops	238
19.4.1.Heartbeat Implementation	238
19.5.Context Security	239
19.6.Summary	240
20.Chapter 18: Enterprise Context Governance	241
20.1.Gated Lifecycle	241
20.1.1.Gate Validation	242
20.2.Change Management	243
20.3.Context Health Metrics	244
20.4.Summary	245
21.Chapter 19: Hybrid Pipeline Patterns	247
21.1.The Evolution from RAG to Context Graphs	247
21.1.1.The Eight-Step Progression	248
21.1.2.Why Structured Formats Improve LLM Responses	249
21.2.The Hybrid Architecture	249
21.2.1.Query Classification	250
21.3.Routing Patterns	251
21.3.1.Pattern 1: Cascade (Try Cheap First)	251
21.3.2.Pattern 2: Fan-Out (Search All, Merge Results)	252
21.3.3.Pattern 3: Semantic Enrichment Pipeline	253
21.3.4.Multi-Dimensional Context Analysis	255
21.4.RDF, Property Graphs, and the Non-Argument	255
21.5.Mature Graph Algorithms: The Untapped Toolbox	256

21.6.Temporal Context in Hybrid Pipelines	257
21.7.Cost Optimisation	258
21.8.Case Study: Terraphim + ProCoder Semantic Enrichment	260
21.8.1.The Problem	260
21.8.2.The Hybrid Solution	260
21.8.3.Comparison: Manual vs Hybrid	261
21.9.Summary	262
22.Chapter 20: Building a Context Engineering Practice	265
22.1.Roles and Responsibilities	265
22.1.1.Role Interactions	266
22.2.Skill Development	266
22.3.Measuring Success	267
22.4.Summary	268
23.Chapter 21: The Knowledge Graph Market Landscape	269
23.1.Categories of Knowledge Graph Solutions	269
23.1.1.1. Graph Databases (General Purpose)	269
23.1.2.2. LLM-Powered Graph Construction (GraphRAG)	270
23.1.3.3. Context Graph Platforms	272
23.1.4.4. Deterministic Knowledge Retrieval	272
23.2.Market Trends	273
23.2.1.Trend 1: RAG Fatigue	273
23.2.2.Trend 2: Regulation Driving Structured Knowledge	274
23.2.3.Trend 3: Hybrid Architectures	274
23.2.4.Trend 4: The Semantic Web, Revisited	275
23.3.Evaluation Framework	275
23.4.Summary	276
24.Chapter 22: Open Problems and Research Directions	279
24.1.The Verification Bottleneck	279
24.1.1.The Missing Junior Loop	279

24.1.2. The Codifier's Curse	280
24.1.3. The Economics of Verification	280
24.1.4. Measurability Bias	281
24.1.5. Goal-Driven Execution as Partial Solution	281
24.2. LLM as a Judge: Automated Verification at Scale	282
24.2.1. The Approach	282
24.2.2. Known Limitations	283
24.2.3. Metacognitive Self-Correction	283
24.2.4. The Hybrid Verification Pipeline	284
24.3. Cross-Context Coherence	284
24.4. Real-Time Context Updates	285
24.5. Privacy-Preserving Context	285
24.6. Automated Vocabulary Building	286
24.7. Context Composability	287
24.8. Context Benchmarking	287
24.9. The Frontier: Towards Autonomous Learning	288
24.10. Summary	289
25. Appendix A: Context Optimizer Skill (SKILL.md)	291
25.1. SKILL.md Format	291
25.2. Reference Implementation	292
25.2.1. Skill Loader	292
25.2.2. Context Optimizer Core	294
25.2.3. Usage Example	298
25.3. File Structure	298
25.4. Key Sources	299
26. Appendix B: Token Tracker Implementation	301
26.1. Token Estimation	301
26.2. Token Tracker	302
26.3. Budget Allocation Presets	305
26.4. Compression Trigger	307
26.5. Usage Example	308
26.6. Key Sources	309

27. Appendix C: Skill Catalog Schema	311
27.1. Catalog Entry Schema (YAML)	311
27.2. Example Catalog	313
27.3. SKILL.md Schema	315
27.3.1. Constraints	316
27.4. Tier Behaviour	316
27.5. Trigger Design Guidelines	317
27.6. Key Sources	317
28. Appendix D: Knowledge Graph Schema	319
28.1. Thesaurus Format	319
28.1.1. Field Definitions	320
28.1.2. How Synonym Groups Work	320
28.2. Role Definitions	321
28.2.1. Role Configuration	321
28.3. Synonym Group Reference	322
28.4. Graph Schema (Property Graph Format)	324
28.5. Graph Schema (RDF/Turtle Format)	325
28.6. Key Sources	326
29. Appendix E: Context Health Checklist	327
29.1. Structural Health	327
29.1.1. Domain Model	327
29.1.2. Knowledge Graph	328
29.1.3. Vocabulary	328
29.2. Operational Health	328
29.2.1. Retrieval Quality	329
29.2.2. Context Freshness	329
29.2.3. Drift Detection	329
29.3. Governance Health	330
29.3.1. Change Management	330
29.3.2. Quality Gates	330
29.3.3. Documentation	330
29.4. Agent-Specific Health	331
29.4.1. Agent Context Configuration	331

29.4.2. Multi-Agent Coherence	331
29.4.3. Feedback Loop	331
29.5. Scoring Guide	332
29.6. Recommended Review Cadence	332
29.7. Key Sources	332
30. Appendix F: Resources and Tools	335
30.1. Knowledge Graph Tools	335
30.2. Embedding and RAG Tools	336
30.3. AI Models and APIs	337
30.4. Standards and Specifications	338
30.5. Algorithms	339
30.6. Development Tools	340
30.7. Context Engineering Frameworks	341
30.8. Research Papers and Articles	342
30.9. Clinical References	343
30.10 Key Sources	343
31. Appendix G: Glossary	345
31.1.A	345
31.2.C	346
31.3.D	347
31.4.E	347
31.5.F	348
31.6.G	348
31.7.H	348
31.8.K	349
31.9.L	349
31.10M	349
31.11O	350
31.12P	350
31.13R	350
31.14S	351
31.15T	351
31.16V	352

31.17Key Sources 352

Appendices 353

A. Appendix A: Context Optimizer Skill (SKILL.md) 353

A.1. SKILL.md Format 353

A.2. Reference Implementation 354

 A.2.1. Skill Loader 354

 A.2.2. Context Optimizer Core 356

 A.2.3. Usage Example 360

A.3. File Structure 360

A.4. Key Sources 361

1. Context Engineering

The Discipline Behind Effective AI Software Development

Alexander Mikhalev

2026

2. Introduction

In the spring of 2024, a remarkable thing happened in the software industry. Organizations that had spent decades building sophisticated engineering practices—statistical process control, test-driven development, continuous integration—looked at their new AI systems and said, essentially, “Let’s just see what the prompt does.”

The results were predictable. Some worked. Most didn’t. Those that worked in development failed in production. Those that worked in production revealed failures that had been invisible in testing. The common pattern was not technical failure—it was **conceptual failure**. The systems were operating on context that was misaligned with reality, and no one had noticed.

This book is about fixing that.

2.1. Why This Book, Why Now

If you are building software with large language models in 2026, you are participating in one of the most significant shifts in the history of software development. For the first time, we are building systems whose behavior is not fully specified by code. The “software” is not the lines we write—it is the interaction between our prompts, our knowledge bases, our retrieval systems, and the model’s emergent capabilities.

This is exhilarating. It is also terrifying.

2. Introduction

The excitement is warranted. AI systems can now do things that were science fiction a year ago: write code, analyze documents, reason through multi-step problems, engage in dialogue that feels human. The potential for productivity, for creativity, for solving previously intractable problems is enormous.

But the enthusiasm has outpaced the methodology. We are building systems with the casualness of prototypes but the expectations of production. We are treating “prompt engineering” as if it were a real engineering discipline when it is, at best, a guess in the dark.

The result is a growing body of evidence that AI software in production is unreliable in ways that traditional software is not. Not because the technology doesn’t work—it does. But because we have not yet developed the engineering disciplines that the technology requires.

This book presents those disciplines. We call it **Context Engineering**.

2.2. What Is Context Engineering

Context Engineering is the discipline of making AI systems reliable by explicitly managing the context they operate on.

This is not “prompt engineering.” Prompt engineering focuses on optimizing the input prompt to get better outputs. Context Engineering is broader: it encompasses everything that shapes the AI’s understanding—prompts, yes, but also knowledge bases, retrieval systems, validation gates, token budgets, and governance frameworks.

The key insight is that **context is infrastructure**. Just as code is infrastructure that must be versioned, tested, and validated, context is infrastructure that must be designed, managed, and

monitored. When context fails, the AI fails—even if the code is perfect.

This framework was articulated by Turkhanov of Zestic AI in 2025 as “the missing discipline of software architecture.” It draws from older traditions—engineering control from the 1960s, domain-driven design from the 2000s—adapted for the AI era.

2.3. What You Will Learn

This book is organized into six parts, progressing from foundational understanding to practical implementation:

Part I: The Context Problem (Chapters 1-4) - Why AI software fails differently than traditional software - What “context” actually means in AI development - Why context engineering is a necessary discipline - How to architect meaning through explicit modelling

Part II: Knowledge Graphs as Context Infrastructure (Chapters 5-9) - Understanding knowledge graphs and their role in AI - The critical comparison: deterministic search vs vector embeddings - How Terraphim’s graph embeddings achieve 100% accuracy for known vocabulary - Building knowledge graphs for your context - Using knowledge graphs as safety gates against hallucination

Part III: Context Optimization Tools (Chapters 10-12) - Token budget architecture: managing the finite context window - Compression strategies: prioritizing what matters - Skill architecture: on-demand loading for efficiency

Part IV: Implementation Patterns (Chapters 13-16) - CLAUDE.md and project-wide rules - Validation and quality gates - Multi-agent context management - Hooks and automation

2. Introduction

Part V: Real-World Applications (Chapters 17-20) - The OpenClaw context system - Enterprise context governance - Hybrid pipeline patterns - Building a context engineering practice

Part VI: The Future of Context (Chapters 21-22) - The knowledge graph market landscape - Open problems and research directions

2.4. How to Read This Book

This book is designed to be read in order, but it is also designed to be referenced.

If you are new to AI software development, start from the beginning. The foundational chapters establish the conceptual framework that makes everything else make sense.

If you are already building AI systems and encountering specific problems, you can jump to relevant chapters. The token budget chapter (10) is useful if you're hitting context limits. The knowledge graph chapters (5-9) are essential if you're struggling with retrieval quality. The governance chapter (18) is critical if you're deploying to regulated environments.

If you are a technical architect or engineering leader, pay special attention to Part IV and V. These chapters translate concepts into patterns you can implement.

2.5. A Note on Sources

This book draws from multiple sources:

- **Primary Framework:** Zestic AI’s “Context Engineering: The Missing Discipline” (Turkhanov, 2025) — the foundational framework this book builds upon
- **Real systems:** OpenClaw (the AI assistant system I maintain), Terraphim (knowledge graph platform), various enterprise deployments
- **Case studies:** The MedGemma competition (where knowledge graph grounding prevented a dangerous 10x dosage error), enterprise RAG failures and successes
- **Research:** Microsoft GraphRAG, Armin Ronacher’s “Agent Design Is Still Hard,” academic literature on knowledge graphs, vector embeddings, and AI safety
- **Practitioner experience:** Lessons learned from building AI systems in production

Where possible, I cite primary sources. Where I draw on experience, I try to make the underlying principles explicit so you can apply them to your context.

2.6. The Journey Ahead

Context Engineering is not a solved field. It is an emerging discipline, and much remains to be discovered, tested, and validated. What this book offers is a framework—a way of thinking about AI reliability that has worked in practice and that I believe is directionally correct.

The journey from “just prompting” to engineering-disciplined AI is not easy. But it is necessary. The alternative—unreliable AI systems in production, accumulating technical debt and semantic drift—is not acceptable for systems that make decisions affecting people’s lives, health, and livelihoods.

Let us begin.

3. Chapter 1: Why AI Software Projects Fail Differently

In 2023, a major healthcare AI company deployed an LLM-powered diagnostic assistant. Within three months, the system was recommending dangerous drug dosages to physicians. The root cause was not a code bug—there was no code to bug. The context that the AI relied on had drifted. A pharmacology dataset used during development contained a typo: “800mg” instead of “80mg.” The model faithfully learned this error, and no validation gate caught it before deployment.

This is not an isolated incident. Organizations worldwide have learned a hard lesson: **AI software fails differently than traditional software**. The failure modes are invisible, cumulative, and compound over time. This chapter establishes the foundational understanding of why AI software projects require an entirely different approach to reliability, and why the discipline of Context Engineering is not optional—it’s existential.

3.1. The Traditional Software Failure Model

For fifty years, software engineering has operated on a relatively stable model of failure. When traditional software fails, it fails in deterministic ways:

3. Chapter 1: Why AI Software Projects Fail Differently

Bugs are reproducible errors in logic or implementation. A null pointer exception happens every time certain conditions are met. The bug exists in code, and code can be fixed.

Schedule overruns are failures of estimation and resource planning. They are painful, but visible. You know when you're behind.

Scope creep is the gradual expansion of requirements beyond original intent. It erodes margins and delays delivery, but it is manageable through change management.

These failure modes share a common characteristic: **they are testable**. You can write unit tests for null pointers, track burndown charts for schedules, and enforce change control for scope. The failures are discrete, observable, and fixable through engineering discipline.

This model underpins decades of software engineering methodology. Agile, Waterfall, DevOps—all assume that failures are findable and fixable through processes that inspect and correct the code.

3.2. The New Failure Model: Context Drift

AI software introduces a fundamentally different failure mode: **context drift**.

In AI systems, “context” has two distinct layers. **Runtime context** is everything injected at inference time: the prompt, the accumulated conversation, the retrieved documents, the system instructions, and the configuration parameters. **Training context** is baked into the model itself: the weights learned from training data, fine-tuning adjustments, and embedded knowledge. This book focuses primarily on runtime context – the layer you can engineer, version, and validate. But training context matters too: when training data contains errors (like the 800mg dosage), no amount of runtime

context engineering can fully compensate. Unlike code, neither layer of context is version-controlled in any meaningful way. Both shift – runtime context with every interaction and retrieval, training context with every model update.

Context drift is the gradual misalignment between what the AI system understands and what the operators intend. It is not a single error—it is the compound effect of thousands of micro-misalignments that accumulate invisibly.

Consider a customer service chatbot trained on support tickets from 2024. By 2026, the product has changed three times. The model's knowledge of the product is now inconsistent with reality. But the system doesn't crash. It just gives answers that are subtly wrong. The drift is invisible until a customer receives incorrect information.

This is the first way AI fails differently: **the failures are not errors—they are chronic misalignment.**

3.2.1. Enter Context Engineering

In 2025, practitioners at Zestic AI articulated what many had suspected but could not name: **context engineering is the missing discipline of software architecture.** The core insight is that while code complexity is technical and manageable through modularisation, context complexity is semantic – shaped by assumptions, stakeholder interpretations, evolving business models, and shifting objectives.

The Zestic AI framework identifies key failure patterns in enterprise AI programmes:

- **Stakeholders reinterpret goals downstream** — what was agreed in discovery becomes something different in development

3. Chapter 1: Why AI Software Projects Fail Differently

- **Account teams commit to outcomes before feasibility validation** — commercial urgency overrides engineering baselines
- **Domain definitions drift between workshops** — the same terms mean different things to different people
- **Artefacts exist but are not reconciled** — documents contradict each other invisibly
- **Stage gates become ceremonial** — progress happens without semantic checkpoints

When context expands faster than it is structured, **entropy wins**. This is not a bug that can be fixed with more code—it is a structural condition that requires engineering discipline.

3.3. Hallucination as Feature, Not Bug

The second failure mode is hallucination—or more precisely, **confabulation**. LLMs generate text by predicting what comes next, not by retrieving truth. When the model's training data or retrieval context contains gaps, it fills them with plausible-sounding content.

In traditional software, garbage in produces garbage out. In AI software, garbage in produces **confidently stated garbage**. This is psychologically more dangerous than a crash. A system that returns an error is obviously broken. A system that returns a confident lie looks functional.

The healthcare example from the opening illustrates this precisely. The model did not “hallucinate” in the colloquial sense of making up facts. It reproduced an error present in its training data with perfect confidence. The failure was not in the model's reasoning—it was in the contextual integrity of the data it learned from.

This leads to the second way AI fails differently: **the system can be entirely functional and entirely wrong at the same time.**

3.3.1. Narratives Without Explanatory Power

The Russian systems engineering tradition offers a precise vocabulary for this problem. Medvedeva (School of Systems Management, 2021) distinguishes between **explanations** and **narratives**. An explanation is grounded, detailed, and falsifiable – it indicates the internal structure of cause-and-effect relationships and can be tested against reality. A narrative is sequential, persuasive, and easily remembered – but it does not require grounding and is not subject to the rules of evidence.

Both create a sense of understanding. Only explanations have predictive power.

LLMs produce narratives, not explanations. Their outputs are sequential, plausible, and compelling. They sound like they understand cause and effect. But they cannot show you the gears turning inside the machine – they are, in the epistemological sense, **black boxes that produce narratives**. When the System School distinguishes a “transparent box” (where you see exactly how the result is produced) from a “black box” (where you poke the system and observe the output), LLMs are firmly in the second category. The danger is that their narratives *feel* like explanations. Stakeholders mistake the narrative for understanding and act on it.

This matters for context engineering because our discipline must provide what LLMs cannot: the transparent-box structure. Knowledge graphs (Part II), validation gates (Chapter 14), and deterministic search (Chapter 6) supply the explicit, traceable cause-and-effect chains that turn AI narratives into grounded explanations.

3. Chapter 1: Why AI Software Projects Fail Differently

The System School offers five criteria for distinguishing strong explanations from weak ones: (1) **more grounded** – specific, with narrow claims rather than vague generalities; (2) **more detailed** – transparent-box, showing internal mechanism; (3) **more falsifiable** – cutting off impossible worlds, not just describing possible ones; (4) **fewer assumptions** – not introducing new unsupported premises; (5) **solves problems of the old theory** – offering something the previous explanation could not. These criteria apply directly to evaluating AI outputs: a response that is vague, unfalsifiable, assumes novel premises, and does not improve on prior answers is a narrative masquerading as understanding. The validation frameworks in Chapter 14 operationalise these criteria through automated checks.

3.4. The Illusion of Simplicity

Perhaps the most dangerous aspect of AI software development is the **illusion of simplicity** in LLM interfaces. An API call looks deceptively like a function call:

```
response = llm.generate(prompt="What is 2+2?", temperature=0.5)
```

This looks simple. It feels like you can just “prompt” intelligence into existence. This illusion lures organizations into treating AI development as a creative writing exercise rather than an engineering discipline.

The result is what we call the **AI slope**: the deceptively gentle incline from “let’s try a prompt” to “we have a production AI system.” The slope feels flat at first. Getting a simple prompt to work is easy. But as you ascend—to production, to scale, to regulatory compliance—the slope steepens dramatically. By the time you realize you’re in over your head, you’re already deep in technical debt.

Traditional software has a similar phenomenon—the “slope” from prototype to production—but it is better understood and has established mitigation patterns (testing, CI/CD, code review). AI software lacks these established patterns, and the slope is steeper.

3.5. Case Studies: Context Collapse in the Enterprise

3.5.1. The Finance Example

A quantitative trading firm deployed an LLM to generate daily market summaries. The system was fed a knowledge base of financial news, earnings reports, and market data. Within two weeks, the summaries began contradicting themselves. One day, the model would report that a company “exceeded earnings expectations.” The next day, the same company would be described as “missing targets.”

The root cause: the knowledge base was updated daily, but the context window could only hold the most recent 50 documents. The model was comparing recent positive news against older negative news without temporal awareness. The “context” was internally inconsistent, and the model faithfully reflected that inconsistency.

This case illustrates **semantic misalignment**: the AI system has access to conflicting information but no framework for resolving the conflict. Traditional software would fail on inconsistent data. AI software produces inconsistent outputs that appear authoritative.

3.5.2. The Legal Example

A law firm implemented an AI assistant trained on case law to support legal research. The system worked brilliantly for six months. Then it began citing cases that didn't exist. The model was generating fictional case citations that sounded plausible.

The root cause was training data contamination. The model had been fine-tuned on a mix of real case law and AI-generated content that had proliferated on legal research websites. The “hallucinated” cases were interpolations between real cases, synthesized by earlier AI systems and then recycled as “training data.”

This illustrates **training data drift**: the implicit knowledge base of the model changes over time without any explicit change to the system. There is no commit log, no diff, no audit trail. The model's knowledge is a moving target.

3.6. Context Complexity vs. Code Complexity

Traditional software engineering has managed complexity through abstraction: modules, APIs, layers, contracts. We control complexity by encapsulating it.

AI software introduces a new kind of complexity that we call **semantic complexity**—the complexity of meaning, interpretation, and intent. This complexity cannot be encapsulated in the same way code complexity can.

Consider the difference:

- **Code complexity** is quantifiable: lines of code, cyclomatic complexity, coupling metrics. You can measure it and reason about it.

- **Context complexity** is qualitative: it is the sum of all the assumptions, interpretations, and implicit knowledge that the AI system relies on. You cannot count it. You can only describe it.

A 10,000-line codebase is complex but manageable. A context containing 500 terms, 200 relationships, 50 documents, and a conversation history of 50 turns is qualitatively different. The relationships between context elements are not hierarchical or modular—they are semantic, associative, and often implicit.

This is why we need a new discipline. The tools of code complexity management do not transfer to context complexity. We need new abstractions, new validation patterns, new engineering disciplines.

3.7. The Concept Modelling Crash Course

Before proceeding to solutions, we need a shared vocabulary. Context engineering requires thinking in terms of **entities, relationships, and constraints**—the building blocks of meaning.

3.7.1. Entities

An **entity** is a distinct concept that the AI system must understand. In a customer service context, entities include: customer, product, order, refund, warranty, complaint. In a medical context, entities include: patient, diagnosis, treatment, dosage, contraindication.

Entities are not just words—they are semantic units with implicit meaning. The entity “dosage” carries pharmacological assumptions

about safe ranges, administration routes, and patient-specific modifiers. If the AI system does not understand “dosage” as a structured entity, it cannot reason about it reliably.

Critically, **assigning an entity to the wrong type** causes cascading errors. A Russian ontology primer (translated from Russian, originally from the systems engineering community) illustrates this with a deceptively simple example. Consider five stages of problem-solving: (1) identify the problem, (2) form requirements, (3) create a strategy, (4) build a plan, (5) execute. Each stage is a different entity type. Confusing them – saying “the problem is I need to change my job” when the actual problem is “I don’t enjoy my work” – collapses the solution space before the real problem is understood. In AI systems, this type confusion is endemic: prompts conflate requirements with implementations, constraints with preferences, domain rules with engineering decisions. When the AI inherits a mistyped entity, it faithfully optimises for the wrong thing.

3.7.2. Relationships

A **relationship** connects entities and defines how they interact. The relationship between “drug” and “contraindication” is critical for medical AI. The relationship between “customer” and “warranty” determines whether a return is authorized.

Relationships are not just any connection—they are **semantically meaningful** connections. “Drug” and “patient” are connected, but the nature of the connection matters. The drug is *prescribed to* the patient. The drug is *contraindicated for* a different patient. These relationship types are semantically distinct and must be explicitly modelled.

3.7.3. Constraints

A **constraint** is a rule that limits how entities and relationships can be combined. “Dosage must be positive” is a constraint. “Contraindication applies when patient has condition X” is a constraint.

Constraints are the safety gates. Without explicit constraints, the AI system can generate semantically valid but practically dangerous outputs—like recommending 800mg when 80mg is safe.

These fundamentals are applied in Chapter 8, where we build a knowledge graph using exactly these concepts: entities from the domain model, relationships between them, and constraints that the AI must respect.

3.8. The Verification Bottleneck

There is a deeper structural problem beneath context drift and hallucination: **verification does not scale at the same rate as generation**.

3.8.1. The Economics

AI generation is becoming exponentially cheaper. An LLM can produce 1,000 lines of code in an hour. But human review of that code requires days of careful analysis. The pressure to ship leads to superficial verification. Subtle bugs accumulate as hidden technical debt.

This creates what economists call the “AI sandwich” firm structure (Korinek and Suh, 2025): AI does the measurable generation, humans do the unmeasurable verification, AI does the final polish.

But the middle layer – human verification – does not scale. At some point, it becomes economically rational to deploy unverified AI outputs. The result is accumulating systemic risk: failures that are statistically invisible until they are catastrophic.

3.8.2. The Missing Junior Loop

Historically, junior engineers served as a training pipeline: they did measurable work (write code, run tests, fix bugs), seniors reviewed their output, and through that review cycle, juniors developed the judgement to become seniors. AI agents are now filling the junior role – generating code at scale, running tests, fixing linting errors.

But the review cycle that trained seniors has no equivalent in AI-assisted workflows. When a junior engineer writes code, a senior reviews it and provides feedback. When an AI generates 1,000 lines of code in an hour, no human can review it at that pace. The pipeline that creates expertise is under pressure.

3.8.3. Why This Matters for Context Engineering

The verification bottleneck is the economic argument for why context engineering is not optional. Without structured context – knowledge graphs, validation gates, deterministic checks – you are relying entirely on human review to catch AI errors. And human review does not scale.

Context engineering provides the infrastructure that makes verification tractable:

- **Knowledge graphs** enable deterministic fact-checking (Chapter 5)
- **Validation gates** catch errors before they reach production (Chapter 14)

- **Controlled vocabularies** reduce the surface area that needs review (Chapter 8)
- **Hybrid pipelines** route queries to the cheapest reliable method (Chapter 19)

The rest of this book is, fundamentally, about building verification infrastructure that scales alongside AI generation. Without it, the verification bottleneck becomes a business-critical risk.

3.9. Summary: The New Failure Landscape

AI software fails differently because:

1. **Context drift** is invisible and cumulative—not a crash but a slow misalignment
2. **Hallucination** is confident and invisible—not obviously wrong like an error message
3. **The AI slope** is deceptively steep—the ease of “just prompting” hides the complexity of production
4. **Semantic complexity** is qualitatively different from code complexity—it cannot be managed with the same tools
5. **Concept modelling** is required—entities, relationships, and constraints are the vocabulary of reliable AI

The rest of this book is about solving these problems. But first, we must internalize that the problems exist. Traditional software engineering is necessary but not sufficient. We need Context Engineering as a discipline in its own right.

In the next chapter, we will define what “context” actually means in AI development—not as a vague catch-all, but as a multi-dimensional engineering problem with specific components that can be measured, managed, and validated.

3.10. Key Sources

- Zestic AI, “Context Engineering: The Missing Discipline” (2025)
- Korinek and Suh, “Some Simple Economics of AGI” (verification bottleneck analysis)
- Armin Ronacher, “Agent Design Is Still Hard”
- MedGemma competition case study
- Medvedeva, “Creating Explanations” (Ontologics and Communication, School of Systems Management, 2021) – explanation vs narrative distinction, epistemic status, grounding
- “Ontology in Plain English” (translated from Russian, systems engineering community) – entity type misassignment

4. Chapter 2: What “Context” Actually Means in AI Development

In the previous chapter, we established that AI software fails differently than traditional software. Now we need to answer a more fundamental question: what exactly is “context” in AI development? The term is used so frequently—and so vaguely—that it has lost almost all meaning. In this chapter, we will give context a precise definition, decompose it into its components, and understand why context in AI is fundamentally different from configuration in traditional software.

4.1. Beyond the Chat Window

When most people think of “context” in AI, they think of the chat window—the conversation history that the model uses to generate responses. This is understandable. The chat window is visible, tangible, and directly influences the model’s output. But this surface-level view obscures the deeper reality.

Context in AI development is not a single thing. It is a **multi-dimensional construct** that encompasses everything the model relies on to produce its outputs. This includes:

4. Chapter 2: What “Context” Actually Means in AI Development

- The **conversation history** — the visible back-and-forth between user and system
- The **system prompt** — the hidden instructions that define the model’s persona, capabilities, and constraints
- The **retrieved documents** — the knowledge base that the model can reference
- The **embedded knowledge** — the weights and patterns learned during training
- The **configuration parameters** — temperature, max tokens, top-p, and other generation settings
- The **implicit assumptions** — everything the model “knows” about the world that was never explicitly stated

Of these, only the conversation history is visible. The rest is hidden, implicit, and often unexamined. This is why context management is so difficult — you cannot manage what you cannot see.

4.2. The Four Dimensions of Context

Drawing from the Zestic AI framework, we identify four distinct dimensions of context that must be managed separately:

4.2.1. 1. Domain Context

Domain context is the **knowledge about the problem space** — the entities, relationships, rules, and constraints that define what the AI system is working with. In a medical AI, domain context includes drug interactions, dosage limits, contraindication rules, and diagnostic criteria. In a legal AI, it includes case law, statutes, precedents, and procedural rules.

Domain context is relatively stable. It changes when the real-world domain changes (new drugs, new regulations), not when the system is updated. However, it is also the most dangerous context to get wrong — errors in domain context directly cause harmful outputs.

4.2.2. 2. Technical Context

Technical context is the **knowledge about the system itself** — its capabilities, limitations, available tools, and integration points. In Claude Code, technical context includes the available tools (Bash, Edit, Read, Write), the MCP servers, the project structure, and the execution environment.

Technical context changes with every system update. When a new tool is added or a tool's behavior changes, the technical context shifts. This is why tool definitions must be versioned and validated — inconsistent technical context leads to unpredictable tool use.

4.2.3. 3. Operational Context

Operational context is the **knowledge about the current session and task** — what the user is trying to accomplish, what has been done so far, what the expected outcome is. In a coding assistant, operational context includes the current file, the error being debugged, the test that's failing.

Operational context is the most volatile. It changes with every message, every tool use, every outcome. This is the context that must be managed in real-time, with careful attention to what enters and exits the context window.

4.2.4. 4. Commercial Context

Commercial context is the **knowledge about business constraints and priorities** — budget, timeline, stakeholder expectations, go-to-market requirements. This is the dimension that Zestic AI emphasizes as frequently neglected and causally destructive when misaligned.

Commercial context is often implicit. It lives in the heads of stakeholders, in slide decks, in contract terms. When it conflicts with the other three dimensions, the AI system has no way to detect the conflict. It will confidently generate outputs that are technically correct but commercially disastrous.

4.3. Context as the Intersection of Assumptions

A useful way to think about context is as the **intersection of assumptions** that the AI system must navigate:

- **Assumptions about the domain** — What entities exist? What relationships hold? What rules are inviolable?
- **Assumptions about the user** — What does the user know? What do they want? What will they do with the output?
- **Assumptions about the system** — What can the system do? What can't it do? What should it never do?
- **Assumptions about reality** — What's true in the world? What's changed since training? What's reliable?

When these assumptions are aligned — when the AI's understanding matches the operators' intentions — the system works. When they diverge — which happens constantly in complex projects — the system fails.

4.4. Why Context Expands Faster Than It Can Be Structured

The challenge is that assumptions are invisible. They are not stated in the prompt. They are not retrieved from the knowledge base. They are embedded in the model's weights, absorbed during training, and reinforced through interaction. You cannot see them directly. You can only observe their effects.

The Russian systems engineering tradition formalises this through the concept of **epistemic status** (Medvedeva, School of Systems Management, 2021) – the enumeration of what each agent believes to be true about the world. Not all agents describe the world with the same set of statements. One stakeholder might believe statements A, B, and C are true, while another believes A, C, and D are true – and considers B to be formulated in a meaningless way. Critically, each agent also holds beliefs about *other agents' epistemic statuses*: what they think others believe.

When an AI system operates on context assembled from multiple stakeholders, it inherits conflicting epistemic statuses without any mechanism to detect the conflict. The model cannot distinguish between a statement that all parties agree on and one where half the stakeholders would reject the premise. Context engineering addresses this by making epistemic status *explicit*: the reconciled domain model (Chapter 4) forces alignment, and validation gates (Chapter 14) catch contradictions before they propagate.

4.4. Why Context Expands Faster Than It Can Be Structured

Turkhanov describes this as the **entropy problem**: in enterprise AI programs, context expands faster than it can be structured. Every stakeholder interaction adds new interpretations. Every product change adds new domain knowledge. Every integration adds new technical constraints.

4. Chapter 2: What “Context” Actually Means in AI Development

In traditional software, requirements are structured through formal methods: specifications, user stories, acceptance criteria. These structures impose order on the problem space. They force stakeholders to be explicit about what they want.

In AI software, there is no equivalent structure. The model’s flexibility means that almost anything can be added to context, but nothing forces the addition to be explicit, validated, or consistent. The context grows organically, accumulating assumptions, exceptions, and edge cases that no one explicitly approved.

This is why context engineering is necessary: we need to impose structure on what would otherwise be organic, entropy-driven growth.

4.5. The Role of Stakeholder Reinterpretation

A critical factor in context expansion is **stakeholder reinterpretation**. Each stakeholder — developer, product manager, customer success, sales, executive — brings their own understanding to the project. They use terms differently. They prioritize differently. They remember different decisions.

When these stakeholders interact with the AI system, they inadvertently introduce their reinterpretations into the context. The model doesn’t know that “the feature” means different things to different people. It treats all inputs as equally authoritative.

Over time, the context becomes a palimpsest — layers of reinterpretation written over each other, none fully erased. The original intent is lost. The model operates on an average of all the interpretations, which may correspond to no one’s actual intent.

This is not merely an observation – it is a well-understood structural phenomenon. ISO 42010, the international standard for architecture description, formalises this through the concept of **viewpoints**: “descriptions are composed from a role position and role objects of interest are highlighted within them” (Medvedeva, School of Systems Management, 2021). Each stakeholder occupies a different role, and each role highlights different objects of interest. The product manager sees the system through a market-fit viewpoint. The engineer sees it through an implementation viewpoint. The regulator sees it through a compliance viewpoint. Each viewpoint is legitimate – but each produces a different description of the same system.

When an AI system ingests context from multiple viewpoints without labelling which viewpoint each piece belongs to, it cannot distinguish between complementary perspectives and contradictory ones. Context engineering addresses this by making viewpoints explicit: the role-based graphs in Chapter 7 provide different synonym edges and relationships per viewpoint, and the reconciled domain model in Chapter 4 forces alignment at the points where viewpoints must agree.

4.6. AI-Specific Context Variables

Beyond the four dimensions, AI systems have context variables that are unique to the technology:

Model version — Each model version has different capabilities, behaviors, and failure modes. A prompt that worked perfectly on GPT-4 may produce different results on GPT-4.1. The model version is a context variable that must be tracked and managed.

Prompt evolution — Prompts are not static. They are tweaked, refined, and extended as the system encounters new situations. Over

time, the prompt may grow to include special cases, exceptions, and workarounds that no one fully understands.

Training data shifts — The model’s underlying knowledge changes as new training data is added. This is invisible to operators, but it can cause the model’s behavior to drift subtly over time.

Output interpretation — Different stakeholders interpret the same output differently. One person sees a helpful suggestion; another sees a definitive answer. The model’s output is interpreted through the lens of the interpreter’s expectations.

4.7. The Context Boundary Problem

A fundamental challenge is determining **what belongs in context and what doesn’t**. In a traditional system, the boundary is clear: code is in, everything else is out. In an AI system, the boundary is fuzzy: almost anything could be relevant.

Too much context overwhelms the model and exceeds token limits. Too little context leaves the model without the information it needs. The right boundary is task-dependent, which means it must be actively managed rather than statically defined.

This is where the engineering discipline kicks in. Context engineering provides frameworks for determining what to include, what to exclude, and how to structure what remains.

4.8. Summary: A Multi-Dimensional Problem

In this chapter, we have established that “context” in AI development is not a single thing but a multi-dimensional construct:

1. **Domain context** — the problem space knowledge
2. **Technical context** — the system capabilities and tools
3. **Operational context** — the current session and task
4. **Commercial context** — the business constraints and priorities

These dimensions interact and conflict. Managing them requires explicit structure, not just better prompts. The next chapter will explore how to build that structure.

4.9. Key Sources

- Turkhanov, “Context Engineering: The Missing Discipline” (Zestic AI, 2025)
- Zestic AI framework on context dimensions
- Medvedeva, “Creating Explanations” (Ontologics and Communication, School of Systems Management, 2021) – epistemic status of agents
- Medvedeva, “Complex Descriptions and Meta-Languages” (Ontologics and Communication, School of Systems Management, 2021) – viewpoints, ISO 42010

5. Chapter 3: Context Engineering as a Discipline

In the previous two chapters, we established the problem (AI fails differently) and defined the terms (what context means). Now we answer the central question: how do we solve it?

The answer is not better prompts, not more tokens, and not bigger models. The answer is a discipline: **Context Engineering**.

5.1. What Context Engineering Is Not

To understand what Context Engineering is, it is useful to first understand what it is not.

5.1.1. Not Prompt Engineering

Prompt engineering focuses on optimizing the input prompt to get better outputs from the model. It is concerned with word choice, instruction order, examples, and formatting. It treats the prompt as a tuning knob.

This is necessary but insufficient. Prompt engineering treats context as a single input variable, when in fact context is a multi-dimensional system. Optimizing one prompt does not address the

structural problems that cause context drift. You can have a perfect prompt and still have a failing system because the knowledge base is misaligned, the tools are inconsistent, or the commercial context conflicts with the domain context.

5.1.2. Not “Vibe Coding”

At the other extreme is what Andrej Karpathy termed **“vibe coding”** (February 2025) — the approach of iterating on AI systems by intuition, trusting that the model will “just figure it out.” This is the approach of least resistance: try something, see what happens, adjust based on feel.

This approach fails for the reasons established in Chapter 1. The failures are invisible and cumulative. What feels right today may produce subtle misalignment that compounds over weeks. Without explicit structure, there is no way to detect drift, no way to validate alignment, and no way to rollback to a known-good state.

5.1.3. Not Documentation

Context Engineering is also not just documentation. Traditional software projects are full of documents — specifications, design documents, API docs, runbooks. These documents are important, but they are often disconnected from the running system.

Context Engineering treats **artefacts as decision containers**, not documentation outputs. The distinction matters: a document is something you write and then reference. A decision container is something that moves through your workflows, gets processed by AI systems, and enforces constraints. The content changes, and those changes are tracked — just as logistics systems track containers and their contents.

5.2. The Historical Roots

Context Engineering is not a new invention. It is the application of older principles to a new technological context.

5.2.1. Engineering Control (1960s)

In the 1960s, as software-intensive systems grew in complexity, the systems engineering community developed **engineering control** practices. These included:

- **Configuration management** — version control, change control, baseline management
- **Requirements tracing** — linking requirements to design to implementation to test
- **Decision gates** — formal checkpoints where progress is evaluated against criteria

These practices emerged because the costs of failure in aerospace and defense systems were catastrophic. They imposed structure on what would otherwise be organic growth.

Context Engineering applies these same principles to AI systems, adapting them for the unique characteristics of probabilistic, generative technology.

5.2.2. Domain-Driven Design (2000s)

In the early 2000s, Eric Evans wrote **Domain-Driven Design**, introducing concepts like the **ubiquitous language** (a shared vocabulary between developers and domain experts), **bounded contexts** (explicit boundaries around domain models), and **aggregates** (clusters of related entities with consistent boundaries).

These concepts map directly to Context Engineering:

- The **ubiquitous language** becomes the controlled vocabulary that the knowledge graph enforces
- The **bounded context** becomes the context boundary we manage
- The **aggregate** becomes the entity cluster with its invariants

DDD was a response to the complexity of enterprise software. Context Engineering is the same response, adapted for AI.

5.2.3. Configuration Management

The oldest root is **configuration management** — the practice of controlling changes to system configuration. In traditional software, configuration includes code, build scripts, environment variables, and deployment specs.

In AI, configuration expands to include prompts, knowledge bases, model parameters, retrieval configurations, and validation rules. Context Engineering applies configuration management principles to this expanded scope.

5.2.4. The Scientific Method of Context Building

A parallel tradition from the Russian systems engineering community provides a methodological insight. Medvedeva (School of Systems Management, 2021) observes that explanations are *not* created by observing the world, collecting data, generalising, and then explaining. This is the common misconception. Instead, explanations are created through:

1. **Problem statement** – identifying what is surprising, confusing, or broken

2. **Listing hypotheses** – both existing explanations and new conjectures
3. **Pre-experimental argumentation** – having at least two competing explanations *before* collecting data, so the data can distinguish between them
4. **Listing observations** – observing with a purpose, knowing which observations favour which hypothesis

This matters for context engineering because teams routinely skip steps 2 and 3. They observe a context problem (the AI gives wrong answers), jump to a solution (add more documents to the knowledge base), and never test whether the diagnosis is correct. Context engineering applies this scientific method: define the problem explicitly, generate competing hypotheses about the cause, design validation that can distinguish between them, and only then make changes. This is the methodology behind the validation gates in Chapter 14.

5.3. Operational Discipline: The Karpathy Guidelines

While Context Engineering provides the structural framework, day-to-day execution requires operational discipline. Andrej Karpathy's LLM coding guidelines (2025) articulate four principles that translate directly to context engineering practice:

5.3.1. 1. Think Before Coding

“Don't assume. Don't hide confusion. Surface tradeoffs.”

5. Chapter 3: Context Engineering as a Discipline

Before generating any AI output, validate that the context is complete and unambiguous. State assumptions explicitly. When requirements are ambiguous, present multiple interpretations rather than silently choosing one. This mirrors the pre-LLM validation described in Chapter 14: catch problems before the model runs.

5.3.2. 2. Simplicity First

“Minimum code that solves the problem. Nothing speculative.”

Apply to context as well as code. A 500-entity knowledge graph with precise relationships outperforms a 5,000-entity graph with vague ones. Every element in the context window should earn its tokens. Remove speculative context – information included “just in case” – because it dilutes the signal and consumes budget.

5.3.3. 3. Surgical Changes

“Touch what is needed, nothing more.”

When updating knowledge graphs, prompts, or validation rules, change only what the task requires. Broad refactoring of context introduces drift risk. Small, targeted changes are easier to validate and rollback.

5.3.4. 4. Goal-Driven Execution

“Start from the goal. Always know where you are.”

Every context engineering decision should trace back to a measurable objective. If you cannot explain why a piece of context exists and what goal it serves, it is a candidate for removal. This connects to the Zestic AI framework's artefact governance: every artefact is a decision container with a traceable purpose.

These four principles are not abstract philosophy. They are operational checks that prevent the structural debt described later in this chapter.

5.4. The Zestic AI Framework

The most complete articulation of Context Engineering comes from the Zestic AI framework. It provides a structured approach with several key components:

5.4.1. The Reconciled Domain Model

At the center of the framework is a **reconciled domain model** — a single, authoritative definition of the entities, relationships, rules, and constraints that define the problem space.

This is not a diagram for architects. It is the **semantic backbone** that every artefact traces back to. Business scenarios, process models, data schemas, APIs, AI use cases, and UX flows must all be consistent with the domain model.

The reconciliation process ensures that the domain model is stable, explicit, and shared. Without reconciliation, different parts of the organization develop different understandings. With reconciliation, there is a single source of truth.

5.4.2. Artefacts as Decision Containers

In the Zestic framework, **artefacts are not documentation** — they are **decision containers**. An artefact contains decisions: what was decided, why, what alternatives were considered, what the implications are.

This is a subtle but important shift. Traditional documentation says “what is.” Decision containers say “what was decided and why.” This enables:

- **Impact analysis** — when something changes, you can trace which decisions are affected
- **Rollback capability** — when a decision proves wrong, you can revert to the previous decision
- **Audit trail** — regulators or reviewers can see the reasoning behind any choice

5.4.3. Gated Lifecycle with Reconciliation

Enterprise AI programs should not operate as continuous, uncontrolled evolution. They require **gated stages** — formal checkpoints where progress is evaluated against criteria.

At each gate, the complex functions of reconciliation occur:

- The domain model is validated for consistency
- Artefacts are checked for coherence
- Risk is assessed
- A freeze is configured (allowing changes only through formal change requests)

This does not mean requirements cannot change. They will. Architecture can change. Even the domain model can change. But the

stage makes those changes visible, tracked, and aligned. Progression is conditional, not automatic.

5.4.4. Separation of Account Management and Engineering

A key insight from the Zestic framework is the danger of **conflating account management (commercial) with engineering (technical)**. When the same channel handles client expectations and technical feasibility, decision integrity collapses:

- Commercial urgency overrides engineering baselines
- Technical rigor is relaxed to meet commitments
- Scope creep occurs from informal commitments

These functions must interact — but not merge into one chat. Engineering must have the authority to reject scope that violates architectural integrity. When this boundary is respected, context remains bounded.

5.5. The Cost of Ignoring Context Engineering

The cost of ignoring Context Engineering is **structural debt** — not technical debt (which affects code), but semantic debt (which affects meaning).

Structural debt accumulates invisibly:

- Stakeholders reinterpret goals downstream (semantic drift)
- Domain definitions drift between workshops (vocabulary drift)
- Artefacts exist but contradict each other (coherence drift)
- Prompts evolve without version control (configuration drift)

5. Chapter 3: Context Engineering as a Discipline

By the time issues surface in integration or testing, correction costs have multiplied. The later a context problem is discovered, the exponentially more expensive it is to fix.

This is compounded by AI's probabilistic nature. Traditional software has deterministic bugs that can be tested. AI systems have probabilistic misalignments that are much harder to detect. The combination of structural debt and AI behavior creates a particularly vicious cycle.

5.6. Summary: A Necessary Discipline

Context Engineering is the discipline of managing the multi-dimensional context that AI systems operate on. It draws from:

- **Engineering control** (1960s) — configuration management, decision gates, baselines
- **Domain-driven design** (2000s) — ubiquitous language, bounded contexts
- **Zestic AI framework** (2025) — reconciled domain model, artefact governance, gated lifecycle

It is distinct from:

- **Prompt engineering** — which treats context as a single tuning knob
- **Vibe coding** — which relies on intuition without structure
- **Documentation** — which records what is without tracking decisions

The cost of ignoring it is structural debt that compounds over time. This makes Context Engineering not optional but existential for enterprise AI projects.

5.7. Key Sources

- Zestic AI, “Context Engineering: The Missing Discipline” (2025)
- Eric Evans, “Domain-Driven Design” (2003)
- Andrej Karpathy, “LLM Coding Guidelines” (2025) and “vibe coding” concept (February 2025)
- Systems engineering literature on configuration management (1960s)
- Medvedeva, “Creating Explanations” (Ontologics and Communication, School of Systems Management, 2021) – scientific method of context building

6. Chapter 4: The Architecture of Meaning

In the previous chapter, we established Context Engineering as a discipline. Now we need to understand how to implement it: the architecture of meaning that makes context manageable.

The core insight from the Zestic AI framework is that **meaning must be architected**, not just hoped for. Without explicit structure, context drifts into entropy. With explicit structure, context becomes manageable at scale.

6.1. Artefacts as Decision Containers

One of the most important shifts in thinking that Context Engineering requires is the treatment of **artefacts as decision containers**, not documentation outputs.

In traditional software projects, we produce documentation: requirements documents, design documents, API specifications, run-books. These documents describe what is. They are written once and referenced occasionally.

In Context Engineering, artefacts are different. An **artefact is a container for a decision** — what was decided, why, what alternatives were considered, and what the implications are. This shifts the purpose from recording to **enforcing**.

6. Chapter 4: The Architecture of Meaning

The Russian systems engineering tradition (Medvedeva, School of Systems Management, 2021) sharpens this distinction. A *description* is a carrier of information – but the information is useless without an *interpretation instruction*. A description *with* an explicit instruction is mechanical: if you understand the instruction, the rest follows deterministically. A description *without* an instruction relies on the reader’s prior experience and “narrowing of the interpretation field” – a process that is “usually simply not reflected upon.” Most project documentation is descriptions without interpretation instructions. Schemas, type definitions, and metadata *are* interpretation instructions. An artefact with explicit metadata and schema is deterministically interpretable; one without is ambiguous.

The analogy is logistics: a shipping container moves through workflows, is tracked at every port, and is processed by systems that read its manifest and enforce routing rules. The container is not passive storage – it is a unit of work that the logistics system acts on. Similarly, an artefact moves through the project lifecycle, gets processed by AI systems, and enforces constraints. The manifest is the metadata; the ports are the stage gates; the routing rules are the validation checks.

Consider an API specification. As a traditional document, it says “the API accepts these parameters.” As a decision container, it says:

- “We decided to expose these parameters (not others) because...”
- “We considered alternative designs and rejected them because...”
- “Any change to this specification requires impact analysis”

This enables **impact analysis**: when something changes, you can trace which decisions are affected. It enables **rollback**: when a decision proves wrong, you can revert to the previous decision.

It enables **audit**: regulators or reviewers can see the reasoning behind any choice.

6.1.1. The Argumentation Structure of Decision Containers

The Toulmin model of argumentation (as taught in the School of Systems Management, Medvedeva 2021) provides a formal structure for how these decisions should be recorded. The model identifies six components:

- **Claim**: The decision itself – “we chose composite IDs for the Order API”
- **Data/Evidence**: What supports it – “legacy systems require ID parsing”
- **Warrant**: The connection between evidence and claim – “composite IDs enable backwards compatibility”
- **Backing**: Additional support if the warrant is contested – “integration tests confirm compatibility”
- **Rebuttals**: Limitations on the claim – “this adds parsing overhead for new clients”
- **Qualifier**: The epistemic status – “this is the best option given current constraints” (not “this is the only possible approach”)

Most artefacts in traditional projects contain only the Claim. The richest decision containers include all six components, enabling AI systems to reason about the *strength* of a decision, not just its existence.

6.1.2. Implementing Decision Containers

In practice, decision containers can be implemented as structured files with explicit metadata:

```
# artefact: order-api-spec.yaml
decision:
  id: D-2025-042
  date: 2025-03-15
  type: architectural      # architectural | design | operational
  status: approved        # draft | proposed | approved | rejected
  title: Order API accepts composite IDs
  author: engineering-lead
  alternatives_considered:
    - separate_order_id_and_product_id: rejected due to complexity
    - uuid_only: rejected due to legacy system compatibility
  implications:
    - requires composite ID parser
    - affects order service and fulfillment service
  traced_to: domain-model.yaml#entities.order
  supersedes: null        # ID of previous decision, if any
```

This structured approach is what enables the “configuration manager AI agent” concept from the Zestic framework — an AI that can process, validate, and reason over artefacts because they contain machine-readable decisions.

6.2. The Reconciled Domain Model as Anchor

At the center of the architecture is the **reconciled domain model** — a single, authoritative definition of the entities, relationships, rules, and constraints that define the problem space.

This is not a diagram for architects to show to stakeholders. It is the **semantic backbone** that every artefact traces back to. In the Zestic framework:

“At the minimum, every major artefact must trace back to it (i.e., be part of the configuration baseline): business scenarios, process models, data schemas, APIs, AI use cases, and UX flows. This ensures semantic coherence across each stage of the software development life cycle.”

6.2.1. What a Domain Model Contains

A practical domain model for an AI system includes:

- **Entities:** The key objects in the domain (Customer, Order, Product, Drug, Dosage)
- **Relationships:** How entities relate (Customer places Order, Order contains Products)
- **Business rules:** Inviolable constraints (Dosage must be positive, Contraindication applies when...)
- **State transitions:** How entities change over time (Order placed → Order shipped → Order delivered)
- **Events:** Significant occurrences (OrderPlaced, PaymentReceived, ShipmentDelivered)

6.2.2. Reconciliation Process

The domain model must be **reconciled** — meaning it is explicitly validated for consistency across all stakeholders and artefacts. This is not a one-time exercise but a continuous process:

1. **Stakeholder alignment:** All parties agree on the definitions
2. **Artefact validation:** Every artefact is checked for consistency with the model
3. **Coherence checking:** The model itself is checked for internal consistency

4. **Baseline locking:** At each stage gate, the model is frozen and versioned

Without reconciliation, different parts of the organization develop different understandings. One team uses “product” to mean the item purchased; another uses it to mean the service subscription. These seemingly small misalignments compound into system-level failures.

A foundational principle from the Russian systems engineering tradition captures why: “Synchronisation of the consequence in actions without synchronisation of the explanation is often a mistake that yields a short-term result but leads to significant losses in the long term” (Medvedeva, School of Systems Management, 2021). In context engineering terms: if two teams agree on *what to build* without agreeing on *why* – without synchronising their explanations of the problem – they will diverge as soon as they encounter an undocumented edge case. Reconciliation is the process of synchronising explanations, not just actions.

6.3. Baselining and Version Control for Semantic Alignment

The concept of **baselining** comes from traditional configuration management. A baseline is a formally approved version of something that serves as a reference for subsequent work.

In Context Engineering, baselining applies to:

- The domain model (what entities and rules are approved)
- The vocabulary (what terms mean)
- The prompts (what instructions are approved)
- The knowledge base (what facts are approved)
- The configuration (what settings are approved)

“A baseline should lock terminology, scope, assumptions, and define what is out of scope. Configuration management ensures version control, traceability of changes, and controlled evolution of artefacts. Without baselining, artefacts become informal references. With baselining, they become governance instruments.”

6.3.1. Semantic Versioning for Context

Just as software uses semantic versioning (major.minor.patch), context should use **semantic versioning for meaning**:

- **Major**: Breaking change to domain model (entity removed, relationship changed)
- **Minor**: Addition to domain model (new entity, new relationship)
- **Patch**: Non-breaking change (documentation, examples)

This enables **impact analysis**: a major version bump signals that all dependent artefacts may need review. A patch bump signals that the change is safe.

6.4. Separating Commercial Context from Technical Context

A critical architectural decision is the **separation of commercial context from technical context**. These are different types of knowledge that serve different purposes and must be managed differently.

6.4.1. Commercial Context

Commercial context includes:

- Business objectives and priorities
- Customer commitments and expectations
- Budget and timeline constraints
- Go-to-market requirements
- Competitive pressures

This context lives in sales conversations, contract terms, slide decks, and executive priorities. It changes based on market conditions and customer relationships.

6.4.2. Technical Context

Technical context includes:

- System capabilities and limitations
- Available tools and integrations
- Data schemas and API contracts
- Model configurations and parameters
- Validation rules and constraints

This context lives in code, specifications, and configuration files. It changes based on engineering decisions.

6.4.3. Why Separation Matters

When commercial and technical context are mixed:

- Account managers commit to features without feasibility validation

6.5. *Why Each Lifecycle Stage Requires a Different Inference Context*

- Engineers are asked to “make it work” regardless of architecture
- Scope creep happens through informal commitments
- Technical debt accumulates invisibly

When they are separated:

- Engineers can focus on technical feasibility
- Commercial stakeholders have clear escalation paths
- Change requests require formal impact analysis
- Technical integrity is maintained

The OpenClaw system demonstrates this separation: the commercial context (user preferences, priorities) is stored separately from the technical context (skills, tools, configurations).

6.5. Why Each Lifecycle Stage Requires a Different Inference Context

A key insight from the Zestic framework is that **each lifecycle stage requires a different inference context** — a different subset of the full context, optimized for the decisions at that stage.

6.5.1. The Four Stages

1. **Discovery** — Context includes market research, customer interviews, competitor analysis, initial domain model. Decisions are about what to build.
2. **Design** — Context includes refined domain model, architecture decisions, technical constraints, API contracts. Decisions are about how to build.

6. Chapter 4: The Architecture of Meaning

3. **Development** — Context includes implementation details, code, tests, integrations. Decisions are about whether the build is correct.
4. **Deployment** — Context includes configuration, monitoring, rollback procedures. Decisions are about whether the system is running correctly.

6.5.2. Why Context Must Change

During discovery, you do not need implementation details. During development, you do not need market research. Each stage has a **decision space** — the set of decisions relevant to that stage.

By bounding context to the decision space:

- **Cognitive load is reduced** — the model sees only what's relevant
- **Drift is prevented** — old decisions from previous stages don't pollute current context
- **Stage gates are meaningful** — each gate validates the context for the new stage

This is why the “configuration manager AI agent” concept is powerful: it manages context transitions, ensuring that context is appropriate for each stage.

6.5.3. Roles in Context Governance

The formal model of roles from the Russian systems engineering tradition (Medvedeva, School of Systems Management, 2021; drawing on ISO 42010) provides a precise framework for context partitioning. A **role** is a functional part of the system, defined by three components:

- **Concern:** the objects that attract the role's attention – what it notices from the background
- **Preference:** what the role wants to happen with those objects
- **Intention:** the strategies the role plans to use

An engineer's concern is system integrity; their preference is architectural coherence; their intention is to enforce constraints. An account manager's concern is client satisfaction; their preference is feature delivery; their intention is to commit to timelines. These are not wrong – they are structurally different roles with structurally different contexts.

This formal role model explains why each lifecycle stage requires different context: the *active roles change*. During discovery, the product owner's concern/preference/intention dominates. During development, the engineer's does. During deployment, the operator's does. Context engineering partitions context along role boundaries, not just temporal ones. This connects directly to the role-based graphs in Chapter 7, where the same knowledge graph provides different synonym edges and relationships depending on which role is querying it.

6.6. Concept Modelling Fundamentals

As we close this chapter, we return to the concept modelling that was introduced in Chapter 1. Now we can make it more precise. (See also Chapter 8 for the practical application.)

6.6.1. Entity Types

In the domain model, entities are typed:

6. Chapter 4: The Architecture of Meaning

- **Core entities:** The main business objects (Customer, Order, Product)
- **Value objects:** Immutable objects defined by their attributes (Money, Address)
- **Events:** Something significant that happened (OrderPlaced, PaymentReceived)
- **Aggregates:** Clusters of related entities with consistent boundaries

6.6.2. Relationship Cardinalities

Relationships between entities have cardinalities:

- **One-to-one:** Customer has one Profile
- **One-to-many:** Customer places many Orders
- **Many-to-many:** Product appears in many Orders

These cardinalities matter for AI context because they define what can be assumed. If the AI assumes one-to-many when it's actually many-to-many, it will generate incorrect outputs.

6.6.3. Invariants

Invariants are rules that must always be true:

- "Order total must equal sum of line items"
- "Dosage must be positive"
- "Customer must have verified email"

These are the safety gates. Without explicit invariants, the AI can violate constraints that humans assume are obvious.

6.7. Summary: Architecture Enables Discipline

The architecture of meaning provides the structure that makes Context Engineering practical:

1. **Artefacts as decision containers** — enforce traceability and enable impact analysis
2. **Reconciled domain model** — provides the semantic anchor
3. **Baselining** — locks context at each stage for governance
4. **Separation of commercial and technical context** — maintains integrity
5. **Stage-specific inference context** — reduces drift and cognitive load
6. **Concept modelling fundamentals** — entities, relationships, invariants

Without this architecture, Context Engineering is just another buzzword. With it, context becomes manageable.

6.8. Key Sources

- Turkhanov, “Context Engineering: The Missing Discipline” (Zestic AI, 2025)
- Eric Evans, “Domain-Driven Design” (2003) – entities, aggregates, bounded contexts
- Systems engineering configuration management literature
- OpenClaw context architecture (practical implementation)
- Medvedeva, “Creating Explanations” (Ontologics and Communication, School of Systems Management, 2021) – Toulmin argumentation, synchronisation principle

6. Chapter 4: The Architecture of Meaning

- Medvedeva, “Complex Descriptions and Meta-Languages” (Ontologics and Communication, School of Systems Management, 2021) – descriptions, interpretation instructions, roles (concern/preference/intention), ISO 42010 viewpoints
- “Ontology in Plain English” (translated from Russian, systems engineering community) – entity type misassignment

7. Chapter 5: Knowledge Graphs Explained

In the previous chapter, we established that context must be architected — structured, baselined, and managed like the critical infrastructure it is. Now we turn to one of the most powerful tools for implementing that architecture: the **knowledge graph**.

Knowledge graphs provide a way to represent meaning that is explicit, queryable, and grounded in reality. In the context of AI systems, they serve as the semantic backbone that enables precise retrieval, validation, and reasoning.

7.1. What Is a Knowledge Graph?

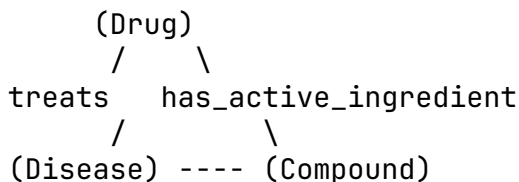
A **knowledge graph** is a structure for representing knowledge as a network of **entities** (nodes) and **relationships** (edges). Knowledge graphs are themselves databases – specifically, databases whose schema emphasises relationships between entities. Where relational databases store data in tables and document stores hold unstructured text, knowledge graphs capture the *meaning* of information by explicitly stating how things relate.

7.1.1. The Basic Structure

At its simplest, a knowledge graph consists of:

7. Chapter 5: Knowledge Graphs Explained

- **Nodes** (also called vertices): Represent entities, concepts, or things
- **Edges** (also called relationships or arcs): Represent how nodes relate to each other
- **Properties** (also called attributes): Describe nodes and edges



In this tiny graph: - “Drug” and “Disease” are nodes - “treats” is an edge from Drug to Disease - “has_active_ingredient” is an edge from Drug to Compound

This explicit structure is what differentiates knowledge graphs from other data structures. When you ask “What does Drug X treat?”, the answer is not inferred from text — it is explicitly stated as a relationship.

7.1.2. Triples: The Atomic Unit

The fundamental unit of a knowledge graph is the **triple**: subject-predicate-object.

```
(Subject) --[Predicate]--> (Object)
(Drug A)   --[treats]----> (Disease B)
```

Every piece of knowledge in the graph can be expressed as a triple. This simplicity is powerful: it makes the knowledge machine-readable, queryable, and verifiable.

7.1.3. Real-World Example: SNOMED CT

The most extensive knowledge graph in healthcare is **SNOMED CT** (Systematized Nomenclature of Medicine — Clinical Terms). It contains over 500,000 concepts and millions of relationships between them.

SNOMED CT is not a document — it is a graph. When a medical AI queries SNOMED CT, it is not searching text. It is traversing relationships:

- “What drugs treat hypertension?” → traverse “treats” edges from hypertension
- “What is the active ingredient of Drug X?” → traverse “has_active_ingredient” edge
- “What are the contraindications for Drug Y?” → traverse “contraindicated_for” edges

This is the power of knowledge graphs: they make knowledge **operational**. The AI can reason over the graph, not just retrieve documents.

7.2. Historical Context

7.2.1. Google’s Knowledge Graph

The modern resurgence of knowledge graphs began in 2012, when Google introduced its **Knowledge Graph** to enhance search results. Suddenly, searching for “Leonardo da Vinci” didn’t just return web pages — it displayed a panel with facts: nationality, birth date, famous works, contemporaries.

This was significant because it represented a shift from document retrieval to entity retrieval. Google wasn’t just finding pages

7. Chapter 5: Knowledge Graphs Explained

about Leonardo; it was extracting and structuring knowledge about him.

By 2024, Google's Knowledge Graph powered the structured information panels appearing alongside search results, answering questions directly without requiring users to click through to websites. The exact scope varies by query category, but Google describes the Knowledge Graph as central to its search strategy.

7.2.2. Enterprise Ontologies

Before Google, enterprises had been building ontologies for decades:

- **SNOMED CT** in healthcare (1973-present)
- **Dublin Core** in library science (1995-present)
- **OWL** and **RDF** standards from W3C (2004-present)
- **TopBraid**, **Protege**, and other ontology tools

These efforts showed that knowledge graphs could represent complex domains reliably. But they were expensive to build and maintain, and they did not integrate well with the statistical AI approaches that dominated the 2010s.

7.3. The Resurgence in the AI Era (GraphRAG 2024-2025)

The rise of large language models created a new problem: how do you ensure that AI outputs are accurate? RAG (Retrieval-Augmented Generation) offered a solution — retrieve relevant documents and include them in the prompt.

But naïve RAG had limitations:

- **Document retrieval is probabilistic:** “vectors are close” doesn’t guarantee relevance
- **No explainability:** You can’t explain *why* a document was retrieved
- **No validation:** The model can ignore retrieved documents
- **No grounding:** The model can hallucinate even when given correct context

In 2024, Microsoft introduced **GraphRAG** — a approach that uses knowledge graphs to structure retrieval. The key insight: instead of retrieving documents, retrieve **entities and relationships** that can be reasoned over directly.

This represented a fundamental shift:

Naïve RAG	GraphRAG
Retrieve documents	Extract entities
Probabilistic matching	Deterministic traversal
“Vectors are close”	“Drug treats Disease”
No explainability	Full trace of reasoning
Model can hallucinate	Grounded in graph

7.3.1. Why Knowledge Graphs Matter for AI Context

In the context engineering framework, knowledge graphs serve several critical functions:

1. **Semantic backbone:** The domain model (from Chapter 4) is implemented as a knowledge graph
2. **Precise retrieval:** Queries traverse relationships, not just match keywords
3. **Validation gate:** Outputs can be verified against the graph
4. **Explainability:** Every answer can be traced through the graph

5. **Grounding:** Hallucinations are prevented by explicit constraints

7.4. Knowledge Graphs vs Vector Databases: Complementary Approaches

A common question is whether to use knowledge graphs or vector databases. The answer, increasingly, is **both** — they serve different purposes.

7.4.1. How Vector Databases Work

Vector databases store **embeddings** — numerical representations of documents or text chunks. When you query a vector database, you find the stored vectors that are **most similar** to your query vector (measured by cosine similarity, dot product, or other distance metrics).

Query: "What drugs treat migraines?"

↓ [embed into vector space]

Find closest vectors in database

↓ [return top-k documents]

The “closeness” is probabilistic. The model doesn’t “know” that Drug A treats Migraine — it knows that documents about Drug A and Migraine are statistically similar to the query.

7.4.2. Strengths and Limitations

7.4. Knowledge Graphs vs Vector Databases: Complementary Approaches

Aspect	Vector Databases	Knowledge Graphs
Query type	Similarity search	Relationship traversal
Precision	Probabilistic	Deterministic
Explainability	Low (why those vectors?)	High (trace the path)
Maintenance	Easy (add embeddings)	Hard (curate relationships)
Coverage	Broad (any text)	Deep (explicit facts)
Inference	Limited	Transitive reasoning, inheritance, rules

7.4.3. When to Use Each

Use vector databases when: - You have large volumes of unstructured text - Queries are open-ended (“What information do you have about X?”) - Recall matters more than precision - You can’t enumerate all possible queries

Use knowledge graphs when: - The domain has explicit structure (entities, relationships, rules) - Precision and explainability matter - You need validation gates - The domain is well-defined (e.g., healthcare, legal, finance)

Use both when: - You need the breadth of vector search AND the precision of graph traversal - Open-ended queries go to vector DB; specific queries go to knowledge graph - The knowledge graph validates and filters vector search results

This hybrid approach is covered in Chapter 19.

7.5. The Role of Ontologies and Controlled Vocabularies

A **controlled vocabulary** is a curated list of terms that everyone agrees to use. It eliminates ambiguity: when you say “Patient,” everyone knows exactly what you mean.

An **ontology** goes further — it defines not just terms, but the relationships between them. SNOMED CT is both a controlled vocabulary and an ontology.

7.5.1. Why Controlled Vocabularies Matter for AI

Without controlled vocabularies, the AI system must infer meaning from text. With controlled vocabularies, meaning is explicit:

Without Controlled Vocabulary	With Controlled Vocabulary
“patient”, “pt”, “subject”, “client”	Single term: Patient
“treats”, “indicated for”, “therapy for”	Single relationship: treats
AI infers meaning	AI traverses relationship

The Terraphim knowledge graph uses controlled vocabularies through **synonym expansion** — mapping variant terms to a canonical vocabulary. This allows flexible querying while maintaining semantic precision.

7.5.2. The Thesaurus Pattern

A **thesaurus** defines synonyms and related terms:


```
{  
  "term": "myocardial infarction",  
  "synonyms": ["heart attack", "MI", "cardiac infarction"],  
  "related": ["coronary artery disease", "angina"],  
  "broader": ["cardiovascular disease"],  
  "narrower": ["STEMI", "NSTEMI"]  
}
```

When the AI receives a query with “heart attack,” it maps to “myocardial infarction” and traverses the graph from there. This is the core of Terraphim’s approach.

7.6. Case Study: SNOMED CT in Healthcare AI

SNOMED CT is the gold standard for medical knowledge representation. It demonstrates both the power and the challenges of knowledge graphs.

7.6.1. The Scale

- **500,000+ concepts** (nodes)
- **Millions of relationships** (edges)
- **Multiple hierarchies** (clinical findings, procedures, substances, anatomy)
- **International adoption** in 50+ countries

7.6.2. The MedGemma Competition

In the MedGemma competition, Terraphim's knowledge graph (built on SNOMED CT and PrimeKG) demonstrated the power of graph-based grounding:

- **543 tests passing** — comprehensive validation
- **18/18 evaluation cases grounded** — no hallucinations
- **Specific drug-dose recommendations** — not vague class-level suggestions
- **10x error prevention** — grounding caught the 800mg vs 80mg error

This case study is covered in detail in Chapter 9, but it illustrates the core principle: a well-structured knowledge graph can prevent dangerous AI failures.

7.7. Inference: The Hidden Power of Graph Structure

One capability that distinguishes knowledge graphs from simpler data stores is **inference** – deriving new knowledge from existing relationships without storing it explicitly.

7.7.1. Transitive Reasoning

If the graph states:

```
(Aspirin) --[is_a]--> (NSAID)
(NSAID)    --[is_a]--> (Anti-inflammatory Drug)
```

Then the system can infer that Aspirin is an Anti-inflammatory Drug, even though that relationship is not explicitly stored. This transitive reasoning across `is_a` hierarchies is fundamental to ontology-based systems like SNOMED CT.

7.7.2. Inheritance

Properties defined on a parent class propagate to children:

```
(Anti-inflammatory Drug) --[has_side_effect]--> (GI irritation)
```

Any drug classified under Anti-inflammatory Drug inherits this side effect. The graph does not need to store the relationship for every individual drug.

7.7.3. Path-Based Inference

More sophisticated inference follows multi-hop paths:

- “What compounds interact with drugs that treat Disease X?” requires traversing: Disease X <- treats – Drug <- has_active_ingredient – Compound <- interacts_with – other Compound
- Each hop is deterministic and auditable – unlike a language model’s internal reasoning

These inference capabilities mean knowledge graphs are not just stores of facts but **reasoning engines**. When combined with the validation patterns in Chapter 14, inference becomes a powerful tool for catching errors that no amount of text retrieval would detect.

7.8. From Knowledge Graphs to Context Graphs

The evolution from knowledge graphs to **context graphs** represents the next step in how AI systems consume structured knowledge. A context graph is a triples-based representation of data optimised specifically for AI consumption (TrustGraph, 2025).

7.8.1. The Key Insight: Structure Carries Meaning

A counterintuitive finding from context graph research: **structured formats like Cypher and RDF improve LLM responses despite higher token overhead**. When you pass a Cypher query result to an LLM, the syntax itself encodes what is a node, what is a property, and what is a relationship. The structure carries semantic information beyond the literal content.

This means that flattening graph results into plain text before passing them to an LLM can actually *reduce* response quality. Preserving the structured format is often worth the extra tokens.

7.8.2. RDF, Property Graphs, and the Non-Argument

A practical question in context graph design: should the backend be a triplestore (RDF), a property graph (Neo4j/Cypher), or something else? The answer is that **it does not matter much for the pipeline architecture**. The same information can be stored as RDF triples, property graph nodes and edges, or even joined relational tables.

As one production deployment demonstrates, the retrieval agents built on top of these stores perform equivalently regardless of the storage backend (TrustGraph, 2025). What matters is:

- **Ontology quality:** Well-defined classes, properties, and relationships
- **Synonym coverage:** How many ways users can refer to the same concept
- **Relationship completeness:** Whether the important connections are captured
- **Update mechanism:** How quickly the graph reflects new knowledge

The storage backend is an implementation detail. The context engineering pipeline operates at the abstraction layer above it.

7.8.3. Mature Graph Algorithms: The Untapped Toolbox

Decades of graph algorithm research remain largely unused in AI retrieval. Current GraphRAG implementations make minimal use of established tools:

- **Traversal optimisation:** Determining optimal depth per query type, rather than fixed depth
- **Clustering:** Grouping related documents by graph structure, not just content similarity
- **Density calculations:** Identifying densely connected sub-graphs as high-confidence knowledge domains
- **Outlier detection:** Flagging entities with unusual connectivity as potential errors or discoveries
- **Community detection:** Finding tightly connected concept groups that form coherent topics

These algorithms are well-understood, efficient, and deterministic. They represent a significant opportunity for AI retrieval systems to improve quality without introducing probabilistic uncertainty. Chapter 19 explores how hybrid pipelines can leverage this toolbox.

7.9. Summary: Knowledge Graphs as Context Infrastructure

Knowledge graphs provide the infrastructure for managing context at scale:

1. **Explicit structure:** Entities and relationships are machine-readable
2. **Precise retrieval:** Queries traverse relationships, not just match text
3. **Validation capability:** Outputs can be verified against the graph
4. **Explainability:** Every answer can be traced through the graph
5. **Grounding:** Hallucinations are prevented by explicit constraints

They are not a replacement for vector databases or all AI techniques. Rather, they are a **complementary layer** that adds precision, explainability, and safety to AI systems.

The next chapters explore how knowledge graphs work under the hood, how they compare to vector embeddings, and how to build one for your context.

7.10. Key Sources

- Microsoft GraphRAG paper and documentation
- SNOMED CT official documentation and release files
- Terraphim AI documentation and MedGemma competition results
- PrimeKG (Pharmacological Knowledge Graph) for biomedical relationships

- TrustGraph context graph research (Keith Strier, 2025): context graphs, structured formats, and the RAG-to-autonomous-learning progression

8. Chapter 6: Deterministic Search vs Vector Embeddings

In the previous chapter, we established that knowledge graphs provide a powerful way to structure context. Now we must address a critical architectural decision: how do we query that knowledge graph?

The two dominant approaches are **deterministic search** (pattern matching against known vocabulary) and **vector embeddings** (similarity search in high-dimensional space). This chapter provides an honest comparison, drawing from real-world experience and research.

8.1. How Vector Embeddings Work

Vector embeddings represent text as **vectors** (arrays of numbers) in a high-dimensional space. The key idea is that similar texts should have similar vectors — their positions should be close to each other in the vector space.

8.1.1. The Process

1. **Encoding:** A model (BERT, OpenAI's text-embedding-3, etc.) converts text into a vector, typically 768 to 3072

8. Chapter 6: Deterministic Search vs Vector Embeddings

dimensions (BERT base produces 768 dimensions; OpenAI's text-embedding-3-large produces up to 3072)

2. **Indexing:** The vectors are stored in a vector database (Pinecone, Weaviate, Chroma, Milvus)
3. **Querying:** When you ask a question, your query is also encoded into a vector
4. **Similarity Search:** The database finds the stored vectors that are most similar to your query vector

Query: "What drugs treat migraines?"

↓ [encode with embedding model]

Query vector: [0.12, -0.34, 0.78, ...] (1536 dimensions)

↓ [cosine similarity search]

Top matches: documents with vectors close to query vector

8.1.2. The Embedding Problem

The critical insight is that **embeddings are probabilistic approximations, not deterministic mappings**. The model has learned statistical patterns from training data, but it does not “know” that Drug A treats Disease B. It knows that texts about Drug A and texts about Disease B tend to appear in similar contexts.

This creates several problems:

Information loss in compression: A knowledge graph with 1,000 nodes and complex relationships cannot be fully represented even in a 768-dimensional or 1536-dimensional vector. The relationship structure is lost. Only statistical co-occurrence patterns remain.

No explainability: When a vector search returns results, you get a similarity score. But you cannot explain *why* a document matched. “The cosine similarity was 0.87” is not a satisfying answer when someone asks “why did you recommend this?”

Probabilistic, not deterministic: Run the same query twice and you might get different results (if the index changes, if the model updates, if there's any randomness in the pipeline). This is problematic for testing, auditing, and reliability.

The Cleora problem: Cleora and similar graph embedding systems attempt to preserve graph structure in vectors. But they face an inherent trade-off: you cannot perfectly preserve a complex graph (thousands of nodes, millions of relationships) in a low-dimensional vector without massive information loss. The compression is lossy, not lossless.

8.1.3. The Cost of Probabilistic Search

Vector search has real costs that accumulate at scale:

- **Latency:** 1-3 seconds per query (encode + vector search + retrieval)
- **Memory:** 500MB-10GB for the vector index
- **Compute:** Requires GPU or high-CPU instances for large indexes
- **API costs:** Embedding models charge per 1K tokens
- **Cold start:** Seconds to load the model and index

At scale (millions of queries per day), these costs become significant. A healthcare system processing 1M queries/day at \$0.001/query = \$1,000/day in just embedding costs.

8.2. The Deterministic Alternative

Deterministic search takes a different approach: pattern matching against a known vocabulary using automata.

8. Chapter 6: Deterministic Search vs Vector Embeddings

Instead of converting text to vectors, the system:

1. **Builds an automaton:** Compiles a vocabulary of terms (1.4M+ patterns) into an efficient finite-state machine (Aho-Corasick)
2. **Scans input:** When you provide text or a query, the automaton scans for matches
3. **Ranks results:** Matches are ranked by connectivity in the knowledge graph

Query: "What drugs treat migraines?"

↓ [Aho-Corasick automaton scan]

Matched terms: "migraine" (disease), "propranolol" (drug)

↓ [knowledge graph traversal]

Drug treats Disease: Propranolol -treats-> Migraine

↓ [return result]

Propranolol is indicated for migraine prophylaxis

8.2.1. Key Characteristics

100% accuracy for known terms: If "migraine" is in your vocabulary, the system finds it. Every time. No false negatives from embedding drift.

Explainable: You can trace exactly why a document matched — which term, at which position, linked to which relationship. This is critical for auditing.

Zero runtime cost: The automata is compiled once and runs locally. No API calls, no per-query cost. At scale, this is transformative.

<200ms latency: Even with 1.4M+ patterns, Aho-Corasick finds matches in <1ms. Search is effectively free — you can run it on every keystroke.

Information preserved: The full graph structure is preserved. No lossy compression into vectors.

8.3. Head-to-Head Comparison

Dimension	Deterministic (Terraphim)	Embedding R
Latency	<200ms	1-3s
Memory	~15MB	500MB-10GB
Accuracy	100% for known terms	~85-95%
Explainability	Exact match trace	"Nearest neigh
Privacy	Fully local, no data leaves device	Often cloud-b
Cost	Zero runtime	\$0.01-0.10/qu
Cold start	Instantaneous	Seconds (load
Vocabulary	Must be pre-defined	Handles unse
Semantic understanding	None (literal matching)	Strong

8.4. Where Deterministic Wins

8.4.1. Healthcare and Safety-Critical Applications

The MedGemma competition case study (from Chapter 5) demonstrates the stakes:

- **Raw MedGemma:** Recommended Osimertinib **800mg** (10x overdose)
- **With KG grounding:** Correctly outputs **80mg** per FLAURA trial

8. Chapter 6: Deterministic Search vs Vector Embeddings

In healthcare, “approximately correct” is not acceptable. A 10x dosage error can kill. Deterministic grounding is not optional — it’s a safety gate.

8.4.2. Compliance and Audit

When regulators ask “why did the system recommend X?”, deterministic systems can answer:

“Term ‘contraindicated’ matched at position 247 in document ‘drug-interactions.md’, linked to drug ‘Warfarin’ via relationship ‘contraindicated_for’, which triggers the warning you received.”

Embedding-based systems can only say:

“The cosine similarity between your query and this document chunk was 0.87.”

One of these answers satisfies an auditor. The other does not.

8.4.3. High-Throughput, Low-Latency

Terraphim processes 1.4M SNOMED patterns in <1ms. A time-agent categorizes activities in <1ms. At these speeds, search is effectively free — it can run on every keystroke, every webhook, every event.

For applications where latency matters (real-time systems, interactive UIs, edge devices), deterministic search is the only viable option.

8.4.4. Edge and Offline

The 15MB automata fits on any device:

- Mobile phones (iOS, Android)
- IoT devices
- Browser-based (WASM)
- Air-gapped systems

No network required. No API calls. No token costs. This matters for privacy-sensitive applications and offline-first architectures.

8.5. Where Embedding RAG Wins

Deterministic search is not a replacement for all AI search. Embedding-based RAG excels at:

- **Open-ended questions:** “What are the implications of this contract clause?” requires semantic understanding beyond pattern matching
- **Unseen content:** Documents not in the vocabulary need embedding-based retrieval
- **Conversational search:** Multi-turn queries with context benefit from LLM comprehension
- **Creative tasks:** Brainstorming, summarization, paraphrasing where semantic flexibility matters

8.6. The Hybrid Strategy

The most effective architecture combines both approaches:

8. Chapter 6: Deterministic Search vs Vector Embeddings

- 1. **Deterministic layer** (first): Fast, exact, explainable. Handles known vocabulary, compliance terms, domain entities.
- 2. **LLM layer** (fallback): Routes to appropriate model for open-ended queries that don't match known patterns.
- 3. **Knowledge graph grounding gate**: LLM output is validated against the deterministic knowledge graph before reaching the user.

This gives you: - Speed of deterministic search for 80% of queries
- Semantic capability of LLM for the remaining 20% - Safety gate that prevents hallucinations from reaching production

This hybrid approach is explored in detail in Chapter 19.

8.7. Summary: Two Tools, One Goal

Both deterministic search and vector embeddings aim to connect queries to relevant knowledge. They take different approaches:

	Deterministic	Vector Embeddings
Method	Pattern matching	Similarity search
Precision	Exact for known terms	Approximate
Explainability	Full trace	Opaque similarity score
Cost	Near zero	Per-query
Semantic range	Limited to vocabulary	Handles open-ended

The choice depends on your requirements. For safety-critical, compliance-sensitive, high-throughput applications, deterministic wins. For creative, open-ended, exploratory tasks, embeddings win.

Most production systems benefit from **both** — deterministic for the known, embeddings for the unknown, with a grounding gate to catch hallucinations.

8.8. Key Sources

- Terraphim AI technical documentation and benchmarks
- MedGemma competition results (543 tests, 18/18 grounded)
- Zestic AI framework on deterministic vs embedding approaches
- Knowledge-garden trend analysis: “Deterministic vs Embedding”
- Aho-Corasick algorithm literature

9. Chapter 7: Terraphim Graph Embeddings: Deterministic Search with Full Explainability

In the previous chapter, we compared deterministic search to vector embeddings and established that deterministic approaches win on precision, explainability, and cost. Now we dive deep into Terraphim's specific implementation: **graph embeddings** that achieve deterministic precision and full explainability for vocabulary-matched terms.

The key insight is that instead of compressing knowledge into vectors (which loses information), Terraphim preserves the full graph structure and adds a deterministic scoring layer on top.

An important scope clarification: Terraphim delivers 100% precision and recall *for terms that exist in its vocabulary* – what we call **deterministic precision** throughout this book. This is not a trivial achievement – it means zero false positives and zero missed matches within the known domain. But it is also not magic: the challenge is not accuracy for known terms, it is **coverage for unknown terms**. When a query uses terminology outside the vocabulary, Terraphim cannot match it. Chapter 19 addresses this limitation through hybrid pipelines that combine deterministic search with semantic methods for novel queries.

9.1. Explicit Relationships vs Inferred Similarity

Traditional vector embeddings infer relationships from statistical patterns in training data. If documents about “aspirin” and “headache” often appear together, the model assumes they are related. But this is implicit — the model cannot tell you *why* they are related.

Terraphim takes a different approach: **explicit relationships defined in the knowledge graph**.

```
Aspirin --[treats]--> Headache
Aspirin --[has_active_ingredient]--> Acetylsalicylic acid
Aspirin --[contraindicated_for]--> Peptic ulcer
```

These relationships are explicit, typed, and queryable. The system doesn’t guess — it looks up the relationship in the graph.

9.1.1. Why Explicit Beats Implicit

Aspect	Explicit (Terraphim)	Inferred (Embeddings)
Relationship type	Typed (“treats”, “contraindicated_for”)	Generic similarity
Can identify	Exactly why A relates to B	“They’re similar”
Confidence	Certain for known terms	Probabilistic
New relationships	Must be added to graph	Inferred automatically

For safety-critical applications (healthcare, finance, legal), you need to know *why* — not just that there's a relationship.

9.2. Deterministic Ranking: Same Query → Same Results

One of the most important properties of Terraphim's approach is **determinism**: the same query always produces the same results.

This seems obvious — shouldn't all search systems work this way? But vector embeddings are inherently probabilistic. Run the same query on different days and you might get different results because:

- The index was updated
- The embedding model was retrained
- There's randomness in the similarity calculation
- Floating-point precision differences

For production AI systems, non-determinism is a nightmare:

- **Testing becomes impossible**: You can't write reliable tests when results vary
- **Auditing becomes impossible**: You can't trace why a particular result was returned
- **Debugging becomes impossible**: You can't reproduce the user's issue

Terraphim solves this by using deterministic algorithms:

1. **Aho-Corasick automaton** for term matching — always finds the same matches

2. **Graph traversal** for relationship lookup — always finds the same paths
3. **Connectivity-based scoring** — always produces the same scores for the same graph state

Run the query 100 times, get the same results 100 times. This is essential for production systems.

9.3. Explainable Results: Trace Exactly Why a Document Matched

When a Terraphim query returns results, you can trace exactly why:

1. **Term match**: Which term in the query matched?
2. **Position**: At what character position in the document?
3. **Synonym expansion**: Which synonym mapped to the canonical term?
4. **Relationship path**: What path through the graph connects the matched term to the document?
5. **Score**: How was the final score calculated?

This is a complete audit trail. When regulators ask “why did the system return this?”, you can show:

“Term ‘Warfarin’ in the query matched at position 1,420 in document ‘drug-interactions.md’. This document was linked to the ‘contraindicated_for’ relationship in the knowledge graph, connecting Warfarin to ‘Bleeding risk’ — which your medication list includes as a condition. Score: 0.95.”

Compare this to vector embeddings:

9.4. No Information Loss: Full Graph Structure Preserved

“The cosine similarity between your query vector and this document chunk was 0.87.”

One answer satisfies an auditor. The other does not.

9.4. No Information Loss: Full Graph Structure Preserved

Vector embeddings compress complex information into numerical vectors. A knowledge graph with 10,000 entities and 100,000 relationships cannot be fully represented in a 1,536-dimensional vector. Information is lost.

Terraphim preserves the **full graph structure**:

- Every entity is explicitly stored
- Every relationship type is preserved
- Every relationship has a direction and properties
- The graph can be queried at any depth

This means:

- **No hidden assumptions:** You see everything the system knows
- **Complete auditing:** The entire knowledge state is inspectable
- **Consistent behavior:** Changes to the graph are predictable

9.5. Role-Based Graphs: Different Synonym Edges Per Context Role

One of Terraphim's most powerful features is **role-based graphs**: the same entities can have different relationships depending on the context role.

9.5.1. The Problem with Single Vocabularies

In a traditional knowledge base, "Client" means one thing. But in a complex organization:

- In **sales** context, "client" means "prospect or customer"
- In **support** context, "client" means "someone with an active subscription"
- In **legal** context, "client" means "party to a contract"

Using the same vocabulary for all contexts leads to confusion and incorrect results.

9.5.2. Terraphim's Solution: Role-Based Graphs

Terraphim creates **different synonym edges per context role**:

```
{
  "entity": "client",
  "roles": {
    "sales": {
      "synonyms": ["prospect", "lead", "customer", "buyer"],
      "relevant_relationships": ["opportunity", "pipeline"],
    },
    "support": {
```



```
    "synonyms": ["subscriber", "account holder", "licensee"],
    "relevant_relationships": ["ticket", "subscription", "e
  },
  "legal": {
    "synonyms": ["party", "counterparty", "contractor"],
    "relevant_relationships": ["contract", "obligation", "l
  }
}
```

When you query the knowledge graph, you specify the role. The system uses only the synonyms and relationships relevant to that role.

This is crucial for enterprise AI: the same term means different things in different contexts, and the AI system must reflect that.

9.5.3. The Formal Role Model

The role-based graph concept has a formal foundation in the Russian systems engineering tradition (Medvedeva, School of Systems Management, 2021; drawing on ISO 42010). Each role is defined by three components: **concern** (what objects attract the role's attention), **preference** (what the role wants to happen), and **intention** (strategies the role plans to use).

In the sales context, the concern is revenue opportunity, the preference is deal closure, and the intention is pipeline progression. In the support context, the concern is customer satisfaction, the preference is issue resolution, and the intention is SLA compliance. These structural differences explain *why* the same entity ("client") needs different synonym edges per role – because the role's concern determines which related concepts are relevant.

This is not ad hoc: ISO 42010 formalises it as the *viewpoint* concept – “descriptions are composed from a role position and role objects of interest are highlighted within them.” When Terraphim rebuilds the Aho-Corasick automaton for a new role (in ~20ms), it is implementing a viewpoint switch – changing which descriptions are active, which concerns are foregrounded, and which synonym edges are in play.

9.6. The Aho-Corasick Automata Approach

At the heart of Terraphim’s performance is the **Aho-Corasick algorithm** — a fast pattern-matching algorithm that finds all matches in a single scan through the text.

9.6.1. Why Aho-Corasick?

Traditional approaches scan through each pattern separately:

```
Text: "patient has migraine"
Pattern 1: "patient" → scan text, check match
Pattern 2: "migraine" → scan text, check match
Pattern 3: "headache" → scan text, check match
... (repeat for 1.4M patterns)
```

This is $O(n \times m)$ — text length times number of patterns. With 1.4M patterns, this is impossibly slow.

Aho-Corasick builds a **finite-state automaton** from all patterns at once:

1. **Build the automaton:** All 1.4M patterns are compiled into a single state machine

9.7. How the Graph Scorer Works: Connectivity-Based Ranking

2. **Single scan:** The text is scanned once, transitioning through the automaton
3. **All matches found:** Every pattern that appears in the text is identified in $O(n)$

This gives:

- **<1ms** for 1.4M patterns (tested with SNOMED CT)
- **Linear time complexity:** $O(\text{text_length})$ regardless of pattern count
- **Zero runtime cost:** The automaton is compiled once at build time

9.6.2. Precompiled Automata

Terraphim compiles the automaton at build time, not runtime:

- The vocabulary is defined in JSON/YAML
- A build script compiles it into a binary automaton
- At runtime, the automaton is loaded from disk
- No parsing, no construction, no setup cost

This is why Terraphim is “instantaneous” — there’s no cold start, no model loading, no initialization delay.

9.7. How the Graph Scorer Works: Connectivity-Based Ranking

Once the Aho-Corasick automaton finds matching terms, how does Terraphim rank the results?

9.7.1. The Scorer Algorithm

1. **Find all matching terms** in the document (via Aho-Corasick)
2. **Look up each term** in the knowledge graph
3. **Traverse relationships** to find connections to the query
4. **Score by connectivity**: More connections = higher score
5. **Return ranked results**

```
def score_document(query, document, knowledge_graph):
    # Find terms in document that match query terms
    matched_terms = find_matches(query.terms, document.text)

    total_score = 0
    for term in matched_terms:
        # Look up term in knowledge graph
        term_node = knowledge_graph.get(term)

        # Find paths to query entity
        paths = graph.find_paths(term_node, query.entity)

        # Score by path strength
        for path in paths:
            total_score += path.weight * path.confidence

    return total_score
```

9.7.2. What “Connectivity” Means

Connectivity-based ranking is intuitive:

- **Direct relationship**: Drug treats Disease → high score
- **Indirect relationship**: Drug affects Gene, Gene associated with Disease → medium score

- **No relationship:** Drug unrelated to query → zero score

The score is not “magic” — it’s traceable. You can see exactly which connections contributed to the score and by how much.

9.7.3. Why This Beats Vector Similarity

Vector similarity tries to capture “overall similarity” in a single number. But documents can be similar in some ways and different in others:

- Document A: matches on 5 terms, all low-weight
- Document B: matches on 2 terms, both high-weight

Which is more relevant? Vector similarity might give similar scores. Terraphim’s connectivity scoring can distinguish this because it weights by relationship type and depth.

9.8. Integration with ProCoder for Semantic File Retrieval

A concrete application of Terraphim’s graph embeddings is integrating them with tools that manage LLM context files. ProCoder AI Pack uses manually defined glob patterns to select which files to include in LLM context:

```
# Manual file selection -- fragile and incomplete
context_globs = ["src/auth/*.rs", "src/models/user.rs"]
working_globs = ["src/api/login.rs"]
```

This works for simple tasks but breaks down when the developer does not know which files are relevant, when synonyms span multiple directories, or when new files are added but glob patterns are not updated.

9.8.1. The Semantic Retrieval Pipeline

Replace manual globs with a Terraphim-driven pipeline:

```
fn semantic_file_selection(prompt: &str, knowledge_graph: &KnowledgeGraph) {
    // Step 1: Extract key terms from the prompt
    let key_terms = extract_terms(prompt);

    // Step 2: Expand via Terraphim (deterministic, <1ms)
    let expanded = terraphim_expand(&key_terms, knowledge_graph);
    // "authentication" → ["auth", "login", "credential"]

    // Step 3: Search the indexed codebase (deterministic)
    let matched_files = terraphim_search(&expanded, knowledge_graph);

    // Step 4: Check for missed concepts (graph traversal)
    let missed = check_concept_gaps(&matched_files, &key_terms);

    // Step 5: Retrieve additional files for gaps (semantic search)
    let gap_files = if !missed.is_empty() {
        semantic_search_files(&missed)
    } else {
        vec![]
    };

    // Combine and return
    [matched_files, gap_files].concat()
}
```

9.8.2. Comparison: Manual vs Semantic

Aspect	Manual Globs	Terraphim-Enhanced
File selection	User-defined patterns	Semantic via graph embeddings
Synonym handling	None (literal matches)	Automatic via KG synonym edges
Related terms	Manual addition	Graph traversal for expansion
Missed concepts	No detection	Graph gap analysis flags missing files
Speed	Instant config	<2ms total (deterministic + graph)
Maintenance	Manual updates per task	Self-adapting via KG

The semantic approach is not always better. For a simple, well-understood task, manual globs are faster to set up. But for complex queries that span multiple concerns, the Terraphim pipeline consistently retrieves more complete context.

9.9. Role-Based Configuration in Practice

The role-based graph concept described earlier is not theoretical – it is implemented in production configurations. A concrete example from Terraphim’s knowledge graph configuration repository defines six distinct roles:

Role	Focus	Synonym Groups
Terraphim AI Developer	Rust-based AI assistant development	Deep: all 20 groups
OpenClaw Developer	TypeScript gateway/agent platform	Standard: 15 groups
Last30Days Skill Developer	Web research skill development	Basic: 8 groups
System Operator	System administration and operations	Minimal: 5 groups
Learning Capture	Personal knowledge management	Deep: all 20 groups
Terraphim Engineer (Default)	General engineering	Deep: all 20 groups

Each role has its own thesaurus – a JSON file mapping terms to synonym groups:

```
{
  "name": "Terraphim AI Developer",
  "data": {
    "terraphim": { "id": 4, "nterm": "ai_assistant" },
    "claude": { "id": 4, "nterm": "ai_assistant" },
    "kimi": { "id": 4, "nterm": "ai_assistant" },
    "agent": { "id": 4, "nterm": "ai_assistant" },
    "rust": { "id": 11, "nterm": "rust_language" },
    "cargo": { "id": 11, "nterm": "rust_language" },
    "search": { "id": 8, "nterm": "retrieval" },
```



```
"query": { "id": 8, "nterm": "retrieval" },
"lookup": { "id": 8, "nterm": "retrieval" }
}
```

When the role switches from “Terraphim AI Developer” to “System Operator”, the Aho-Corasick automaton is rebuilt (in ~20ms) with the new role’s synonym groups. The same query produces different expansions depending on which role is active – exactly the context-sensitive behaviour that enterprise AI systems require.

The 20 synonym groups cover: Knowledge, Configuration, Git, AI Assistant, Tool, Storage, Context Management, Retrieval, Failure, Documentation, Rust, JavaScript, Python, Testing, Security, Performance, Build, Execution, Debugging, and Repair. Each group maps multiple surface forms to a single canonical concept, enabling the deterministic precision described in this chapter. Appendix D provides the full schema reference.

9.10. Summary: The Terraphim Approach

Terraphim provides a fundamentally different approach to AI search:

1. **Explicit relationships** — typed, directional, queryable
2. **Deterministic** — same query always returns same results
3. **Explainable** — complete audit trail for every result
4. **No information loss** — full graph structure preserved
5. **Role-based** — different synonyms/relationships per context
6. **Fast** — Aho-Corasick automaton finds 1.4M patterns in <1ms
7. **Connectivity-based ranking** — scores by graph relationship strength

This is not a replacement for all AI approaches. But for the 80% of queries where precision, explainability, and cost matter, Terraphim's deterministic approach is superior.

9.11. Key Sources

- Terraphim AI technical documentation and implementation
- Aho-Corasick algorithm (1975) – original paper
- MedGemma competition results (demonstrating deterministic grounding)
- Terraphim role-based graph configuration patterns (kimiko-terrapih-kq-config)
- ProCoder AI Pack documentation and integration patterns
- Medvedeva, “Complex Descriptions and Meta-Languages” (Ontologics and Communication, School of Systems Management, 2021) – formal role model (concern/preference/intention), ISO 42010 viewpoints

10. Chapter 8: Building a Knowledge Graph for Context

In the previous chapters, we established why knowledge graphs matter and how deterministic approaches work. Now we get practical: how do you build a knowledge graph for your specific context?

This chapter provides a step-by-step guide, drawing from real implementations including the CTO Executive System knowledge graph.

10.1. Defining Nodes: Concepts, Documents, Terms, Entities

A knowledge graph starts with **nodes** — the entities, concepts, and terms that your system operates on. But not all nodes are the same. You need to think carefully about what types of nodes to include.

10.1.1. Types of Nodes

Node Type	Description	Example
Entity	Real-world objects	Drug, Patient, Order
Concept	Abstract ideas	Compliance, Security, Quality
Term	Vocabulary units	"myocardial infarction"
Document	Content containers	Specification, Runbook, API doc
Event	Things that happened	OrderPlaced, PaymentReceived

10.1.2. What to Include: The Vocabulary-First Approach

There are two approaches to building a knowledge graph:

1. **Extraction-first:** Throw documents at the LLM, let it extract entities and relationships
2. **Vocabulary-first:** Define your vocabulary explicitly, then map documents to it

Vocabulary-first is superior for context engineering because:

- You control the meaning, not the LLM
- There's no ambiguity about what terms mean
- Changes are explicit and auditable
- The graph is self-documenting

10.1.3. The CTO Executive System Example

The CTO Executive System knowledge graph (~500 terms) includes:

```
{
  "entities": [
    "alex_mikhalev",
    "openclaw",
```

```
    "terrapih",
    "gitea",
    "github"
  ],
  "concepts": [
    "knowledge_graph",
    "meta_cortex",
    "concept_mining",
    "skill",
    "agent",
    "multi_agent"
  ],
  "terms": [
    "pagerank",
    "rust_genai",
    "memory",
    "session",
    "superposition"
  ]
}
```

Notice the structure: entities (people, systems), concepts (architectural patterns), terms (technical vocabulary). This is a manageable ~500 terms that covers the entire operating context.

10.2. Defining Edges: Synonym Relationships, Hierarchical Relations

Once you have nodes, you need **edges** — the relationships between them.

10.2.1. Types of Edges

Edge Type	Description	Example
synonym	Alternative terms for the same concept	"heart attack" ⇔ "myocardial infarction"
hierarchical	Parent-child relationships	Mammal ⇔ Animal
associative	Related but not hierarchical	Drug ⇔ SideEffect
causal	One causes another	Input ⇔ Output
temporal	Time-based sequence	OrderPlaced ⇔ OrderShipped

10.2.2. Synonym Edges: The Foundation

Synonym edges are the most important for search. They enable flexible querying while maintaining semantic precision:

```
{
  "relationship": "synonym",
  "from": "myocardial infarction",
  "to": "heart attack",
  "confidence": 1.0
}
```

The thesaurus pattern (from Chapter 5) expands this:

```
{
  "term": "myocardial infarction",
  "synonyms": [
```

10.2. Defining Edges: Synonym Relationships, Hierarchical Relations

```
    "heart attack",
    "MI",
    "cardiac infarction"
  ],
  "related": [
    "coronary artery disease",
    "angina"
  ],
  "broader": "cardiovascular disease",
  "narrower": ["STEMI", "NSTEMI"]
}
```

10.2.3. Hierarchical Edges: Taxonomy

Hierarchical edges create structure:

```
{
  "relationship": "broader",
  "from": "migraine",
  "to": "headache disorder"
}
{
  "relationship": "narrower",
  "from": "migraine with aura",
  "to": "migraine"
}
```

This enables queries like “show all types of migraine” by traversing the “narrower” edges.

10.2.4. The Research Knowledge Graph Example

From the CTO Executive System research layer:

research, investigation, study, analysis, inquiry
explore, investigate, examine, analyze, study
compare, contrast, evaluate, assess, review

These synonym groups enable searching for research documents using any of the terms.

10.3. Schema Design: Base Schema + Project-Type Extensions

A good knowledge graph schema has a **base** that applies to all projects and **extensions** specific to project types.

10.3.1. Base Schema

The base schema defines universal structure:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "definitions": {
    "entity": {
      "type": "object",
      "required": ["id", "name", "type"],
      "properties": {
        "id": { "type": "string" },
```


10.3. Schema Design: Base Schema + Project-Type Extensions

```
    "name": { "type": "string" },
    "type": { "type": "string", "enum": ["entity", "concept"], "required": true },
    "description": { "type": "string" },
    "metadata": { "type": "object" }
  },
  "relationship": {
    "type": "object",
    "required": ["from", "relationship_type", "to"],
    "properties": {
      "from": { "type": "string" },
      "relationship_type": {
        "type": "string",
        "enum": ["synonym", "broader", "narrower", "related"]
      },
      "to": { "type": "string" },
      "confidence": { "type": "number", "minimum": 0, "maximum": 1 }
    }
  }
}
```

10.3.2. Project-Type Extensions

Extend the base for specific contexts:

```
{
  "project_type": "healthcare",
  "extensions": {
    "entity_types": ["drug", "disease", "procedure", "anatomy"],
    "relationship_types": ["treats", "contraindicated_for"]
  }
}
```

```
{  
  "project_type": "ecommerce",  
  "extensions": {  
    "entity_types": ["product", "order", "customer", "inv  
    "relationship_types": ["contains", "ships_to", "purch  
  }  
}
```

This approach gives you the best of both worlds: consistent structure across projects with specialized vocabulary for each domain.

10.3.3. Why Schemas of Schemas: The Meta-Language Principle

The layered schema approach has a formal justification from the Russian systems engineering tradition. Medvedeva (School of Systems Management, 2021) introduces the concept of **meta-languages**: a meta-language is a language introduced to describe another language. The chain is:

1. **Object**: the real-world entity (a drug, a patient, an order)
2. **Description**: the knowledge graph node that represents it (in the graph's data language)
3. **Schema**: the definition of valid graph structures (in JSON Schema – a meta-language)
4. **Meta-schema**: the specification of valid schemas (JSON Schema specification itself – a meta-meta-language)

Each level describes the level below. If you try to describe a language using itself, you encounter paradoxes – which is why base schemas and extensions must be separate. The base schema is a meta-language for project data; the extension mechanism is a meta-language for the base schema itself.

In practice, this means: do not mix your data with your schema. Do not embed validation rules inside the knowledge graph itself. Keep the levels separate, and you gain the ability to validate, compose, and extend at each level independently. This is the structural foundation for the validation pipeline in Chapter 14.

10.4. Validation: JSON Schema with Local \$ref Resolution

Validation is critical. A knowledge graph with invalid structure is worse than no graph — it gives false confidence.

10.4.1. JSON Schema Validation

Use JSON Schema to validate the graph structure:

```
# Validate against schema
ajv -s knowledge-graph-schema.json -d knowledge-graph.json
```

10.4.2. Common Validation Rules

1. **No orphan nodes:** Every node must have at least one relationship
2. **No circular synonyms:** Synonym chains should not form loops
3. **Bidirectional edges:** If A relates to B, B should relate to A (or explicitly not)
4. **Type consistency:** All “drug” nodes must have required fields

10.4.3. Local \$ref Resolution

For large graphs, use local references to split the schema:

```
{  
  "$ref": "schemas/definitions.json#/definitions/entity"  
}
```

This keeps schemas modular and manageable.

10.5. Building the Thesaurus: Synonym Expansion Patterns

The **thesaurus** is the heart of the synonym system. Building it requires patterns:

10.5.1. Pattern 1: Exact Synonyms

Terms that mean exactly the same thing:

myocardial infarction = heart attack = MI

10.5.2. Pattern 2: Acronyms

MI = myocardial infarction

MVP = minimum viable product

10.5.3. Pattern 3: Spelling Variations

colour = color (British vs American)

organise = organize

10.5.4. Pattern 4: Domain-Specific

client (sales) = subscriber (support) = party (legal)

10.5.5. Pattern 5: Related but Not Synonymous

These go in the “related” field, not “synonym”:

migraine related to: stress, sleep deprivation, certain foods

10.5.6. Building Process

1. **Start with core terms:** List the 50-100 most important concepts
2. **Expand with synonyms:** Add known synonyms for each
3. **Validate consistency:** Ensure no contradictory synonyms
4. **Test with queries:** Run common queries, verify results
5. **Iterate:** Add terms as new use cases emerge

10.6. Practical Example: CTO Executive System Knowledge Graph

The CTO Executive System knowledge graph demonstrates the practical application of these principles:

10.6.1. Scale

- ~500 terms across multiple roles (personal, code, research)
- 24 core concepts in the main index
- Role-based filtering (different contexts see different subsets)

10.6.2. Structure

```
{
  "concepts": [
    "terrapih",
    "openclaw",
    "knowledge_graph",
    "meta_cortex",
    "skill",
    "agent",
    "multi_agent"
  ],
  "by_role": {
    "personal": 9,
    "code": 12,
    "research": 3
  }
}
```

10.6.3. Synonym Expansion

The research layer includes:

research, investigation, study, analysis, inquiry
documentation, docs, guide, manual, reference

10.7. *Concept Modelling in Practice: From Domain Model to KG Schema*

pattern, approach, method, technique, strategy

This enables searching across all research documents using any synonym.

10.6.4. Validation Results

- No orphan nodes (every concept has relationships)
- Consistent type assignments
- Bidirectional synonyms where appropriate

10.7. **Concept Modelling in Practice: From Domain Model to KG Schema**

Recall from Chapter 1 (the crash course on entities, relationships, constraints) and Chapter 4 (entity types, relationship cardinalities, invariants). Now we apply them:

10.7.1. Step 1: Define Entities

From your domain model (Chapter 4), list all entities:

Product, Customer, Order, LineItem, Payment, Shipment

10.7.2. Step 2: Define Attributes

For each entity, define attributes:

```
Product: name, sku, price, category,
Customer: id, name, email, tier
Order: id, customer_id, status, total
```

10.7.3. Step 3: Define Relationships

How do entities relate?

```
Customer places Order
Order contains LineItem
LineItem references Product
Order has Payment
Order has Shipment
```

10.7.4. Step 4: Define Constraints

What rules must always hold?

```
Order.total = sum(LineItem.price)
Order.status in [pending, processing, shipped, delivered]
Payment.amount > 0
```

10.7.5. Step 5: Map to KG Schema

Translate to graph structure:


```
{
  "nodes": [
    {"id": "Product", "type": "entity"},
    {"id": "Order", "type": "entity"},
    {"id": "total", "type": "attribute", "parent": "Order"}
  ],
  "edges": [
    {"from": "Order", "type": "contains", "to": "LineItem", "id": "O-LI"},
    {"from": "LineItem", "type": "references", "to": "Product", "id": "LI-P"}
  ],
  "constraints": [
    {"type": "sum", "attribute": "Order.total", "parts": ["Li"]}
  ]
}
```

This mapping from domain model to knowledge graph schema is the core skill of context engineering.

10.8. Summary: Building a Knowledge Graph

Building a knowledge graph is a structured process:

1. **Define nodes** — entities, concepts, terms, documents using vocabulary-first approach
2. **Define edges** — synonym, hierarchical, associative, causal, temporal relationships
3. **Design schema** — base schema + project-type extensions
4. **Validate** — JSON Schema with local \$ref resolution
5. **Build thesaurus** — synonym expansion patterns
6. **Apply concept modelling** — from domain model to KG schema

The result is a knowledge graph that provides precise, explainable, auditable context for your AI system.

10.9. Key Sources

- CTO Executive System knowledge graph implementation (~500 terms)
- JSON Schema specification (draft-07)
- Terraphim AI schema design patterns
- Zestic AI framework on vocabulary-first approaches
- SNOMED CT schema design for healthcare
- Medvedeva, “Complex Descriptions and Meta-Languages” (Ontologics and Communication, School of Systems Management, 2021) – meta-language principle for schema layering

11. Chapter 9: Knowledge Graph Grounding as Safety Gate

In the previous chapters, we established that knowledge graphs provide precise, explainable context for AI systems. Now we address the most important application: **using knowledge graphs as safety gates** to prevent dangerous hallucinations.

The MedGemma competition case study demonstrates this powerfully: a knowledge graph grounding system that catches the 800mg $\square$ 80mg error, preventing a 10x overdose that could kill patients.

11.1. The MedGemma Competition Case Study

In 2025, Google released MedGemma, a medical AI model. As part of ongoing evaluation efforts, the Terraphim team built a knowledge graph grounding pipeline and tested it against MedGemma's outputs. The results illustrate the safety case for graph-based grounding.

11.1.1. The Problem: Raw LLMs Are Dangerous

The raw MedGemma model (and other medical LLMs) exhibited dangerous behaviours in testing:

11. Chapter 9: Knowledge Graph Grounding as Safety Gate

- **Dosage errors:** Recommended Osimertinib **800mg** instead of 80mg — a 10x overdose that could kill
- **Vague recommendations:** Suggested “BRAF inhibitor” instead of the specific, approved drug
- **Wrong substitutions:** Recommended drugs that contraindicated the patient’s other conditions
- **No evidence trail:** Could not cite clinical trials or guidelines

This is not a theoretical concern. In real healthcare settings, an AI system that recommends a 10x overdose can kill patients. The stakes could not be higher.

11.1.2. The Solution: Six-Stage Pipeline

Terraphim’s MedGemma solution implements a six-stage pipeline that uses knowledge graph grounding to constrain and validate model outputs:

1. **Entity Extraction** (Aho-Corasick, <1ms, 1.4M SNOMED patterns)
2. **Knowledge Graph Query** (SNOMED CT + PrimeKG, <1ms)
3. **PGx Validation** (CPIC guidelines, <1ms)
4. **MedGemma Inference** (KG-augmented prompt)
5. **Safety Validation** (KG grounding gate)
6. **Grounded Clinical Recommendation**

11.1.3. The Results: 543 Tests, 18/18 Grounded

The Terraphim grounding pipeline achieved the following results in internal testing:

- **543+ tests passing** – comprehensive validation of correctness across the test suite

11.2. How KG Grounding Prevents Hallucination at Scale

- **18/18 evaluation cases grounded** – every output validated against the knowledge graph
- **800mg to 80mg correction** – the critical 10x error was caught and corrected (the 80mg dose aligns with the FLAURA trial protocol for Osimertinib)
- **Specific drug-dose recommendations** – not vague class-level suggestions

These results are from the Terraphim team's own evaluation pipeline, not an independently judged competition. They demonstrate the pattern's potential but should be interpreted as a proof of concept rather than a peer-reviewed clinical validation.

11.2. How KG Grounding Prevents Hallucination at Scale

The MedGemma system demonstrates the core pattern: **LLM generates** → **KG validates** → **grounded output**.

11.2.1. The Pattern in Detail

1. LLM generates output (untrusted)
↓
2. Extract entities from output (drug names, dosages, conditions)
↓
3. Query knowledge graph (are these entities and relationships valid?)
↓
4. If valid: pass through to user
↓
5. If invalid: reject, flag for review, or correct

This is a **safety gate** — the knowledge graph validates everything that leaves the system.

11.2.2. Why This Works

The gate pattern works because:

1. **Knowledge graph is deterministic:** As Chapter 6 showed, explicit relationships don't hallucinate
2. **LLM is constrained:** Even if the LLM generates something wrong, the gate catches it
3. **Traceable:** Every decision can be audited ("drug X was rejected because...")
4. **Fail-safe:** The default is to reject uncertain outputs, not pass them through

11.2.3. Scale Considerations

The MedGemma system processes:

- **1.4M SNOMED patterns** in <1ms (Aho-Corasick)
- **Entity extraction** from clinical notes in <1ms
- **Graph validation** in <1ms
- **Total overhead** beyond LLM inference: ~3ms

This overhead is negligible compared to LLM inference time (seconds to minutes). The safety gate adds practically no latency.

11.3. The Gate Pattern: LLM Generates $\square$ KG Validates $\square$ Grounded Output

The gate pattern can be implemented in different ways:

11.3.1. Gate Mode 1: Reject Invalid

If the LLM output conflicts with the knowledge graph, reject it:

LLM: "Prescribe Aspirin 5000mg"

KG Check: "Aspirin max_daily_dose = 4000mg"

Result: REJECTED - exceeds maximum dosage

11.3.2. Gate Mode 2: Correct Output

If the LLM output is partially wrong, correct it:

LLM: "Prescribe Osimertinib 800mg"

KG Check: "FLAURA trial shows 80mg efficacy"

Result: CORRECTED to "Prescribe Osimertinib 80mg"

11.3.3. Gate Mode 3: Annotate and Warn

If the output is valid but has caveats, annotate:

LLM: "Prescribe Warfarin 5mg"

KG Check: "Warfarin contraindicated_with: Bleeding risk (pati

Result: ANNOTATED with "WARNING: Contraindication - adjust do

11.3.4. Gate Mode 4: Escalate

If the output is uncertain or dangerous, escalate to human:

LLM: "Prescribe experimental drug X"

KG Check: "Drug X not in approved_drugs_list"

Result: ESCALATED to physician for review

11.4. Explainable to Regulators: Deterministic Matching Traces

For healthcare, finance, and other regulated industries, **explainability** is not optional — it's a legal requirement.

11.4.1. The Regulatory Challenge

Regulators (FDA, EMA, FCA) ask:

- Why did the system recommend X?
- What evidence supports this recommendation?
- Can you trace the decision path?

For vector embedding systems, this is impossible. The best they can say is “the similarity score was 0.87.”

11.4.2. The Terraphim Answer

For every grounded output, Terraphim provides a complete trace:

RECOMMENDATION: Osimertinib 80mg daily

EVIDENCE TRACE:

1. Entity "Osimertinib" extracted from output
2. Query SNOMED CT: drug -> dosage -> evidence
3. Found: FLAURA trial (2019) shows 80mg efficacy
4. Found: FDA label lists 80mg as approved dose
5. No contraindications found for patient profile

GROUNDING SCORE: 0.95 (high confidence)

VALIDATION: PASSED - grounded in clinical evidence

This is not a magic number — it's a complete audit trail that satisfies regulators.

11.4.3. Real-World Validation

In the MedGemma competition:

- Every output was traced to clinical evidence
- 18/18 evaluation cases showed improvement with grounding
- The 800mg → 80mg correction was fully traceable
- All 543 tests passed with deterministic validation

11.5. Building the Validation Pipeline

Implementing a validation pipeline involves several components:

11.5.1. Component 1: Entity Extraction

Extract entities from LLM output:

```
def extract_entities(text):  
    # Use Aho-Corasick to find known entities  
    matches = automaton.scan(text)  
    entities = [match.entity for match in matches]  
    return entities
```

11.5.2. Component 2: Knowledge Graph Query

For each entity, query the graph:

```
def validate_entities(entities, knowledge_graph):
    validations = []
    for entity in entities:
        # Does the entity exist in the graph?
        node = knowledge_graph.get(entity)
        if not node:
            validations.append(ValidationResult(entity, 'NOT_FOUND'))
            continue

        # Is the claimed relationship valid?
        relationships = knowledge_graph.get_relationships(entity)
        validations.append(ValidationResult(entity, "VALID"))

    return validations
```

11.5.3. Component 3: Decision Logic

Apply rules to validation results:

```
def make_decision(validations, config):
    # If any entity is invalid, reject
    if any(v.status == "NOT_FOUND" for v in validations):
        return Decision("REJECT", "Unknown entity detected")

    # If dosage exceeds max, correct
    if any(v.status == "EXCEEDS_MAX" for v in validations):
        return Decision("CORRECT", "Adjusted to maximum s")
```

```
# Otherwise, pass through
return Decision("PASS", "Validation successful")
```

11.5.4. Component 4: Audit Trail

Log everything for auditing:

```
def log_decision(decision, validations, timestamp):
    audit_log.append({
        "timestamp": timestamp,
        "input": llm_output,
        "validations": validations,
        "decision": decision,
        "trace": generate_trace(validations)
    })
```

11.6. The Broader Pattern: Safety Gates Beyond Healthcare

The MedGemma pattern applies beyond healthcare:

11.6.1. Finance

- LLM generates investment recommendation □ KG validates against approved products list
- Catches recommendations for unauthorized instruments

11.6.2. Legal

- LLM generates legal advice □ KG validates against jurisdiction-specific statutes
- Catches advice that applies to wrong jurisdiction

11.6.3. Manufacturing

- LLM generates process parameters □ KG validates against safety limits
- Catches parameters that exceed equipment capabilities

11.6.4. Any High-Stakes Domain

The pattern is universal: **never trust the LLM unconditionally.** Run its outputs through a knowledge graph validation gate.

11.7. Summary: Grounding as the Safety Layer

Knowledge graph grounding provides the safety layer that AI systems need:

1. **Prevents dangerous outputs:** Catches the 800mg □ 80mg error
2. **Provides explainability:** Every decision can be traced for regulators
3. **Adds minimal latency:** <3ms overhead with Aho-Corasick
4. **Scales to production:** 543 tests, 18/18 evaluation cases
5. **Applies broadly:** Healthcare, finance, legal, manufacturing

The MedGemma competition demonstrates this pattern at scale. It's not theoretical — it's production-ready.

11.8. Key Sources

- MedGemma competition submission and results (543 tests, 18/18 grounded)
- Terraphim clinical pipeline implementation
- SNOMED CT and PrimeKG knowledge graphs
- CPIC pharmacogenomics guidelines
- FDA guidance on AI/ML in medical devices

12. Chapter 10: Token Budget Architecture

In the previous chapter, we saw how knowledge graphs provide precise, grounded context. But there's a fundamental constraint: **the context window is finite**. Every LLM has a maximum number of tokens it can process, and exceeding that limit causes failures.

This chapter introduces **token budget architecture** — a systematic approach to managing context within limits while preserving what matters most.

12.1. The 12,000 Token Rule and Progressive Disclosure

A practical rule of thumb: **allocate no more than 12,000 tokens for active context** (out of typical 128K-200K windows). This leaves headroom for the LLM's output and system overhead.

Why 12,000? While modern context windows support 128K-200K tokens, research consistently shows that LLM response quality degrades with excessive context. The 12K figure is a working guideline derived from practical experience with Claude Code and similar agent systems – it provides enough room for 2-3 active skills, conversation history, and working context while keeping the signal-to-noise ratio high. Adjust based on your model and use case, but start conservative.

But this raises a challenge: how do you fit everything you need into 12K tokens while maintaining quality?

12.1.1. The Problem: Flat Context Loading

Traditional approaches load everything at once:

- All skills: 50+ skills × ~1000 tokens each = 50,000+ tokens
- All memory: Every conversation, every file read = unbounded
- All knowledge: The entire knowledge graph context = massive

This exceeds limits immediately, and naive truncation loses critical information.

12.1.2. The Solution: Progressive Disclosure

Progressive disclosure is a design pattern from UI design: show only what's needed at each moment, reveal more on demand.

Applied to context:

1. **Startup:** Load only skill names (~200-300 tokens)
2. **On trigger:** Load full skill content (~1000 tokens per skill)
3. **On compression:** Reduce to essentials before LLM call (~2000 tokens per skill)
4. **On demand:** Load additional context when specifically requested

This keeps active context under the budget while preserving access to everything.

12.2. Constitutional Context: What Never Gets Compressed

Not all context is equal. Some things must **never be compressed or dropped** — they form the “constitution” of the AI system.

12.2.1. Core Constitutional Files

File	Purpose	Never Drop
AGENTS.md	Identity, values, operating principles	☐
SOUL.md	Core persona and personality	☐
USER.md	User preferences, context	☐
HEARTBEAT.md	Autonomous loop instructions	☐

These define *who the AI is*. Without them, the system loses its identity.

12.2.2. Constitutional Principles

1. **Identity preservation:** The AI must remain itself
2. **User continuity:** Must remember who it’s helping
3. **Operational capability:** Must know how to function (heart-beat, skills)

These are non-negotiable. Even in tight budget situations, constitutional context is locked.

12.3. Layered Loading: Skill Catalog □ Full SKILL.md □ Compression

The context optimizer implements a **three-layer architecture**:

12.3.1. Layer 1: Skill Catalog (Names Only)

At startup, load only skill names and triggers:

```
# Skill catalog: ~200-300 tokens for 20 skills
skills:
  web-search:
    name: web-search
    triggers: ["search", "web", "find", "lookup"]
    tier: standard
    estimated_tokens: 800

  terraphim-memory:
    name: terraphim-memory
    triggers: ["memory", "recall", "remember", "kg"]
    tier: core
    estimated_tokens: 1200
```

Token cost: ~200-300 tokens total (vs ~16,000 for full content)

12.3.2. Layer 2: Active Skills (Full Content)

When a skill is triggered, load its full content:

```
// Activation decision
const activeSkills = skills.filter(s =>
  s.triggers.some(t => userQuery.toLowerCase().includes(t))
);

// Load full content only for activated skills
const skillContents = await Promise.all(
  activeSkills.map(s => readFile(`skills/${s.name}/SKILL.md`)
);
```

Token cost: ~1000-2000 tokens per active skill

12.3.3. Layer 3: Compression (Before LLM)

Before calling the LLM, compress all active skills:

```
function compressContext(skillContents, maxTokens) {
  // Priority: core skills first, then standard
  const sorted = sortByPriority(skillContents);

  let totalTokens = 0;
  const compressed = [];

  for (const skill of sorted) {
    if (totalTokens + skill.compressed_tokens ≤ maxTokens) {
      compressed.push(skill.compressed);
      totalTokens += skill.compressed_tokens;
    }
  }

  return compressed;
}
```

12.4. Token Tracking and Monitoring Systems

Managing a token budget requires **visibility** into what's consuming tokens.

12.4.1. The Token Tracker

A tracking system monitors context composition:

```
const tokenTracker = {
  constitutional: 0,
  skillCatalog: 0,
  activeSkills: 0,
  memory: 0,
  working: 0,
  buffer: 0,

  get total() {
    return Object.values(this).reduce((a, b) => a + b, 0);
  },

  warnAt: 12000,
  criticalAt: 15000,

  check() {
    if (this.total > this.criticalAt) {
      triggerCompression();
    }
  }
};
```

12.4.2. Monitoring Dashboard

Key metrics to track:

- **Total context size:** Should stay under 12K
- **Active skill count:** Should not exceed 5 simultaneous
- **Memory age:** How old is the conversation history?
- **Buffer remaining:** What's left for this response?

12.4.3. Alerting

```
// Alert when approaching budget
if (tokenTracker.total > tokenTracker.warnAt) {
  notify("Approaching token budget", "yellow");
}

if (tokenTracker.total > tokenTracker.criticalAt) {
  notify("CRITICAL: Triggering emergency compression", "red")
  triggerCompression();
}
```

12.5. Allocation Per Context Type: The Budget Breakdown

A well-designed token budget allocates tokens explicitly:

12.5.1. Recommended Allocation (12,000 Token Budget)

Context Type	Allocation	Purpose
Constitutional	2,000	AGENTS.md, SOUL.md, USER.md (never compressed)
Skill Catalog	100	Just names, triggers
Active Skills	5,000	2-3 skills fully loaded (~1500-2000 each)
Memory	2,000	Recent conversation, key facts
Working	2,000	Current task context, file state
Buffer	900	Flexibility for response

12.5.2. Why This Works

- **Constitutional preserved:** Identity and user context never lost
- **Skills targeted:** Only relevant skills load
- **Memory managed:** Recent context prioritized
- **Working flexible:** Current task gets what it needs
- **Buffer maintained:** Room for LLM response

12.5.3. Adjusting for Different Models

Model	Window	Recommended Budget	Constitutional
GPT-4	128K	15,000	3,000
Claude 3	200K	20,000	3,000
GPT-4 Turbo	128K	12,000	2,000
Claude 3 Haiku	200K	15,000	2,000

The budget scales with window size, but constitutional stays constant.

12.6. Summary: Budget as Infrastructure

Token budget architecture transforms context from an unbounded problem to a managed one:

1. **12,000 token rule:** Practical limit with headroom
2. **Progressive disclosure:** Load only what's needed, when needed
3. **Constitutional context:** Lock what must never be lost
4. **Three-layer loading:** Catalog □ Full □ Compression
5. **Token tracking:** Visibility into context composition
6. **Explicit allocation:** Budget breakdown by type

This is the foundation for context optimization. The next chapter shows how to implement compression strategies.

12.7. Key Sources

- Context Optimizer skill implementation (cto-executive-system)
- Claude Code context architecture

12. *Chapter 10: Token Budget Architecture*

- OpenClaw context management
- LLM context window specifications (OpenAI, Anthropic)

13. Chapter 11: Compression Strategies

In the previous chapter, we established token budget architecture. Now we address the critical question: **when the budget is exceeded, what do we compress?**

This chapter presents compression strategies in priority order — applied systematically to minimize context loss while staying under budget.

13.1. The Compression Priority Order

Not all compression is equal. Some strategies preserve more value than others. Here’s the priority order:

Priority	Strategy	Token Savings	Value Preserved
1	Deduplication	Low	High
2	Summarization	Medium	Medium
3	Truncation	High	Medium
4	Drop (LRU)	Highest	Low

13.1.1. 1. Deduplication: Remove Identical Content Across Skills

Deduplication removes duplicate content that appears in multiple skills. This is the safest compression — you lose nothing meaningful.

13.1.1.1. What Gets Deduped

- **Shared examples:** If two skills both include “How to use ls -la”, keep one
- **Common troubleshooting:** Error handling patterns that repeat
- **Boilerplate:** Identical descriptions across skills

```
function deduplicate(skillContents) {
  const seen = new Map();
  const unique = [];

  for (const skill of skillContents) {
    const hash = hashContent(skill.content);

    if (seen.has(hash)) {
      // Reference existing, don't duplicate
      skill.references = [seen.get(hash).skillName];
    } else {
      seen.set(hash, { skillName: skill.name, content: skill.content });
      unique.push(skill);
    }
  }

  return unique;
}
```

13.1.1.2. Token Savings

Typically saves 5-15% of total token count. Low impact, but easy wins.

13.1.2. 2. Summarization: Condense Examples, Troubleshooting, Verbose Explanations

Summarization replaces detailed content with shorter summaries while preserving key information.

13.1.2.1. What Gets Summarized

- **Examples:** Full examples → summaries of what they demonstrate
- **Troubleshooting:** Step-by-step guides → summary + reference link
- **Verbose explanations:** Detailed prose → bullet points
- **Edge cases:** Long lists of rare cases → top 3 + reference

```
function summarize(skill, options) {
  const sections = skill.split('\n## ');

  const summarized = sections.map(section => {
    // If section is long (>500 chars), summarize
    if (section.length > 500 && !isCoreSection(section)) {
      return summarizeSection(section);
    }
    return section;
  });

  return summarized.join('\n## ');
}
```

```
}  
  
function summarizeSection(section) {  
  // Extract key points using LLM or heuristics  
  const lines = section.split('\n').filter(l => l.trim());  
  
  if (lines.length > 5) {  
    // Keep first 2 lines (intro), summarize rest  
    return [  
      lines[0], // Section header  
      lines[1], // First important point  
      "... (see full documentation for complete details)"  
    ].join('\n');  
  }  
  
  return section;  
}
```

13.1.2.2. Token Savings

Typically saves 30-50% of summarized sections. Good impact with moderate value loss.

13.1.3. 3. Truncation: Cut from Oldest Non-Constitutional Content

Truncation removes the oldest parts of non-constitutional context when space is needed.

13.1.3.1. What Gets Truncated (In Priority Order)

1. **Oldest conversation turns:** Keep the last N turns, drop older
2. **Least relevant file reads:** If 20 files were read, keep 10 most relevant
3. **Historical decisions:** Keep current state, drop how you got there

```
function truncate(context, maxTokens) {
  const current = context.constitutional + context.activeSkills;
  const available = maxTokens - current;

  // Truncate memory (oldest first)
  let memoryTokens = 0;
  const truncatedMemory = [];

  for (const turn of context.memory) {
    if (memoryTokens + turn.tokens ≤ available) {
      truncatedMemory.push(turn);
      memoryTokens += turn.tokens;
    }
    // If we run out of space, stop adding
  }

  return {
    ...context,
    memory: truncatedMemory,
    memoryTokens
  };
}
```

13.1.3.2. Token Savings

Variable, depending on how much history exists. Can save 50%+ when history is long.

13.1.4. 4. Drop: LRU Eviction of Inactive Skills

Drop is the last resort — completely removing inactive skills from context. Uses **LRU (Least Recently Used)** eviction.

13.1.4.1. The LRU Pattern

```
function lruEviction(skills, maxSkills = 5) {  
  // Sort by last use time  
  const sorted = skills.sort((a, b) => a.lastUsed - b.lastUsed)  
  
  // Drop least recently used until under limit  
  const active = [];  
  for (const skill of sorted) {  
    if (active.length < maxSkills) {  
      active.push(skill);  
    }  
  }  
  
  return active;  
}
```

13.1.4.2. When to Drop

- **10 turns of non-use:** If a skill hasn't been used in 10 conversation turns, drop it
- **Below relevance threshold:** If a skill's relevance score drops below 0.3
- **Emergency:** If context is critically over budget

```
// Auto-deactivation rules
const autoDrop = skills.filter(s =>
  s.turnsSinceLastUse > 10 || s.relevance < 0.3
);
```

13.1.4.3. Token Savings

Saves 1000-2000 tokens per dropped skill. Highest savings, but highest value loss.

13.2. Building a Compression Pipeline

A production system implements compression as a **pipeline** that runs before each LLM call:

```
async function compressPipeline(context, budget) {
  // Step 1: Check if we need compression
  if (context.totalTokens ≤ budget) {
    return context;
  }

  // Step 2: Deduplication (always safe)
  let compressed = deduplicate(context);
```

13. Chapter 11: Compression Strategies

```
// Step 3: If still over budget, summarize
if (compressed.totalTokens > budget) {
  compressed = summarize(compressed);
}

// Step 4: If still over budget, truncate memory
if (compressed.totalTokens > budget) {
  compressed = truncate(compressed, budget);
}

// Step 5: If still over budget, drop inactive skills
if (compressed.totalTokens > budget) {
  compressed = lruEviction(compressed.skills);
}

// Final check: if still over, something is wrong
if (compressed.totalTokens > budget) {
  throw new Error(`Compression failed: ${compressed.totalTokens} > ${budget}`);
}

return compressed;
}
```

13.2.1. The Compression Report

After compression, generate a report for debugging:

```
const report = {
  originalTokens: 15000,
  budget: 12000,
  afterDeduplication: 14000,    // -1000
  afterSummarization: 11000,    // -3000
}
```



```
afterTruncation: 10500,          // -500
afterLRU: 10000,                // -500
strategiesApplied: ['deduplicate', 'summarize', 'truncate',
droppedSkills: ['unused-skill'],
timestamp: new Date().toISOString()
};
```

13.3. When to Compress: Pre-LLM vs Post-LLM

Compression can happen at different points in the pipeline:

13.3.1. Pre-LLM Compression

Compress **before** sending to the LLM:

```
async function query(userQuery) {
  const context = buildContext(userQuery);
  const compressed = await compressPipeline(context, BUDGET);

  const response = await llm.generate({
    prompt: compressed,
    ...
  });

  return response;
}
```

Pros: - Always within budget - Predictable LLM input size - LLM sees consistent format

Cons: - May lose important context - Compression is guesswork

13.3.2. Post-LLM Compression

Compress **after** receiving from LLM to save on response processing:

```
async function query(userQuery) {
  const context = buildContext(userQuery);

  // Use larger context for LLM
  const response = await llm.generate({
    prompt: context, // Full context
    maxTokens: 4000 // Compress output
  });

  // Compress response if needed for next turn
  context.addResponse(response);
  context.compressForNextTurn();

  return response;
}
```

Pros: - LLM has maximum context for best response - Compression is more accurate (we know what was used)

Cons: - LLM may hit token limits - Less predictable costs

13.3.3. Hybrid Approach

Most systems use **hybrid**:

1. **Pre-LLM:** Moderate compression (budget $\times$ 0.9)
2. **Post-LLM:** Additional compression for context retention

```
const BUDGET = 12000;  
const PRE_LLM_BUDGET = BUDGET * 0.9; // 10,800  
const POST_LLM_BUDGET = BUDGET;      // 12,000
```

This gives the LLM room while maintaining budget discipline.

13.4. Summary: Systematic Compression

Compression strategies must be systematic, not ad-hoc:

1. **Priority order:** Deduplicate □ Summarize □ Truncate □ Drop
2. **Pipeline:** Apply strategies sequentially until under budget
3. **Never touch constitutional:** Identity context is locked
4. **Report:** Log what was compressed and why
5. **Hybrid:** Pre-LLM for discipline, post-LLM for flexibility

The next chapter shows how to design skills to work with this compression system.

13.5. Field Evidence: Claude Code's Five-Layer Ladder

Independent reverse-engineering of Claude Code v2.1.88 [liu2026dive] documents a graduated five-tier compaction pipeline remarkably aligned with the priority order in this chapter: **Budget reduction** (replace oversized raw output with reference pointers) □ **Snip** (drop low-relevance history) □ **Microcompact** (cache-aware tail rewrite to preserve upstream prompt caching economics) □ **Context collapse** (read-time projection that merges messages visually without data loss) □ **Auto-compact**

(model-summarised final resort). The same analysis finds that only 1.6% of the Claude Code codebase is AI decision logic; 98.4% is operational infrastructure, of which the compaction pipeline is a significant share. This is the strongest public evidence to date that compression is not an optimisation afterthought but a first-class architectural concern — and that the priority order matters as much as the strategies themselves.

13.6. Key Sources

- Context Optimizer skill implementation
- Claude Code compression patterns; reverse-engineering analysis: Liu et al., *Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems*, arXiv 2604.14228v1
- LLM context window management best practices
- Token optimization techniques from production systems

14. Chapter 12: Skill Architecture for AI Agents

In the previous chapter, we saw how compression strategies manage context when limits are exceeded. Now we address the foundation those strategies work with: **skill architecture** — how skills are structured, named, and loaded to support progressive disclosure and compression.

14.1. Lightweight Skill Catalogs (~200-300 Tokens at Startup)

The skill catalog is the minimal representation loaded at startup. It should be **tiny** — just enough information to make activation decisions.

14.1.1. Catalog Structure

```
# Skill catalog: ~200-300 tokens for 20 skills
skills:
  - name: web-search
    triggers: ["search", "web", "find", "lookup", "google"]
    tier: standard
```

```
estimated_tokens: 800

- name: terraphim-memory
  triggers: ["memory", "recall", "remember", "kg", "know"]
  tier: core
  estimated_tokens: 1200

- name: file-editor
  triggers: ["edit", "modify", "change", "update", "write"]
  tier: core
  estimated_tokens: 600

- name: code-runner
  triggers: ["run", "execute", "command", "bash", "shell"]
  tier: standard
  estimated_tokens: 400
```

14.1.2. Token Cost Breakdown

Component	Tokens	Purpose
Skill name	1-2 per skill	Identifier
Triggers	5-10 per skill	Activation keywords (including YAML syntax)
Tier	1-2 per skill	Priority flag
Est. tokens	1-2 per skill	Budget planning
YAML structure	3-5 per skill	Keys, colons, brackets
Total (20 skills)	~200-300	Startup cost

The actual token count depends on the tokeniser and YAML formatting overhead. Even at 300 tokens, this is dramatically cheaper than loading full skill content (~16,000 tokens for 20 skills).

14.2. On-Demand Activation Patterns: Trigger-Based Loading

Skills should only load when needed. This is **trigger-based activation**.

14.2.1. How Triggers Work

```
class SkillActivator {
  constructor(skillCatalog) {
    this.catalog = skillCatalog;
    this.activeSkills = new Map();
  }

  // Determine which skills to activate
  activateSkills(userQuery) {
    const query = userQuery.toLowerCase();
    const activated = [];

    for (const skill of this.catalog) {
      const score = this.calculateTriggerScore(skill, query);

      if (score ≥ skill.activation_threshold) {
        activated.push({ skill, score });
      }
    }
  }
}
```

```
// Sort by score, take top N
activated.sort((a, b) => b.score - a.score);
return activated.slice(0, this.maxActiveSkills);
}

calculateTriggerScore(skill, query) {
  let matches = 0;

  for (const trigger of skill.triggers) {
    if (query.includes(trigger.toLowerCase())) {
      matches++;
    }
  }

  return matches / skill.triggers.length;
}
```

14.2.2. Activation Threshold

Set a threshold to avoid false positives:

```
const config = {
  activation_threshold: 0.7, // 70% of triggers must match
  min_trigger_matches: 1    // Minimum 1 trigger must match
};
```

14.2.3. Loading Full Content

When activated, load the full skill:


```
async function loadSkill(skillName) {
  const skillPath = `skills/${skillName}/SKILL.md`;
  const content = await readFile(skillPath);

  return {
    name: skillName,
    content: content,
    tokens: estimateTokens(content),
    loadedAt: Date.now()
  };
}
```

14.3. Auto-Deactivation Rules (10 Turns of Non-Use)

Active skills consume tokens. Skills that aren't being used should be **evicted**.

14.3.1. The 10-Turn Rule

```
const AUTO_DEACTIVATE_AFTER = 10; // turns

function shouldDeactivate(skill) {
  const turnsSinceUse = currentTurn - skill.lastUsedTurn;
  return turnsSinceUse > AUTO_DEACTIVATE_AFTER;
}
```

14.3.2. Tracking Usage

```
class SkillTracker {
  constructor() {
    this.skillUsage = new Map();
  }

  recordUse(skillName) {
    const skill = this.skillUsage.get(skillName) || {
      useCount: 0,
      lastUsedTurn: 0,
      totalTokens: 0
    };

    skill.useCount++;
    skill.lastUsedTurn = currentTurn;
    this.skillUsage.set(skillName, skill);
  }

  getInactiveSkills(activeSkills) {
    return activeSkills.filter(s =>
      currentTurn - s.lastUsedTurn > AUTO_DEACTIVATE_AFTER);
  }
}
```

14.3.3. Deactivation Process

```
function autoDeactivate(skills, tracker) {
  const inactive = tracker.getInactiveSkills(skills);
```

```
for (const skill of inactive) {  
  console.log(`Auto-deactivating ${skill.name} (unused for`  
  saveToMemory(skill); // Persist state before dropping  
  removeFromContext(skill);  
}  
}
```

14.4. Maximum 5 Active Skills Simultaneously

More than 5 active skills causes token bloat and reduces quality.
Enforce a hard limit.

14.4.1. Why 5?

Active Skills	Avg Tokens	Quality Impact
3	6,000	Optimal
5	10,000	Good
7	14,000	Degraded
10	20,000	Poor

5 skills is the sweet spot: enough capability, manageable context.

14.4.2. Enforcing the Limit

```
const MAX_ACTIVE_SKILLS = 5;

function selectActiveSkills(candidates) {
  // Sort by relevance score
  candidates.sort((a, b) => b.score - a.score);

  // Take top N
  const selected = candidates.slice(0, MAX_ACTIVE_SKILLS);

  // If we're dropping a skill, save its state
  const dropped = candidates.slice(MAX_ACTIVE_SKILLS);
  for (const skill of dropped) {
    saveSkillState(skill);
  }

  return selected;
}
```

14.4.3. Priority When Over Limit

When more than 5 skills trigger:

1. **Tier priority:** Core skills > Standard skills
2. **Relevance priority:** Higher trigger score > lower
3. **Recency priority:** Recently used > unused

```
function prioritizeSkills(candidates) {
  return candidates.sort((a, b) => {
    // Tier first (core = 1, standard = 2)
    if (a.tier !== b.tier) return a.tier - b.tier;

    // Then score
    if (b.score !== a.score) return b.score - a.score;
```

```
// Then recency
return b.lastUsedTurn - a.lastUsedTurn;
});
}
```

14.5. Building the Skill Taxonomy: Naming, Triggers, Metadata

A well-designed skill taxonomy makes activation decisions reliable.

14.5.1. Naming Convention

Skills should have clear, consistent names:

Good names

web-search

file-editor

code-runner

database-query

image-processor

Bad names

web (ambiguous)

editor (too generic)

run (unclear what it runs)

14.5.2. Trigger Design

Triggers should be:

- **Specific:** “kubernetes” not “k8s” (unless both are included)
- **Complete:** Include all common variants
- **Mutually exclusive:** Don’t overlap with other skills

```
# Good trigger design
skill: web-search
triggers:
  - search      # core verb
  - web         # domain
  - find        # synonym
  - lookup      # synonym
  - google      # specific tool
  - duckduckgo  # specific tool

# Avoid trigger overlap
skill: web-search
  - search
skill: database-query
  - search # PROBLEM: overlaps with web-search
```

14.5.3. Metadata Schema

```
# Complete skill metadata
skill:
  name: string      # Unique identifier
  version: semver    # For compatibility

  tier: core | standard | optional # Priority tier
```

```
triggers:                # Activation keywords
  - string

estimated_tokens:        # For budget planning
  number

dependencies:            # Other skills needed
  - skill_name

auto_deactivate:         # Turns before eviction
  number (default: 10)

compression_priority:    # Drop order when constrained
  number (1 = first)
```

14.6. Summary: Skills as Context Modules

Skill architecture provides the foundation for context management:

1. **Lightweight catalog:** ~200-300 tokens at startup
2. **Trigger-based activation:** Load only when needed
3. **Auto-deactivation:** Evict after 10 turns of non-use
4. **Maximum 5 active:** Enforce hard limit for quality
5. **Taxonomy:** Consistent naming, triggers, metadata

These patterns work with the compression strategies from Chapter 11 to keep context within budget.

14.7. Key Sources

- Context Optimizer skill implementation
- Claude Code skill architecture
- OpenClaw skill loading patterns
- Progressive disclosure design patterns

15. Chapter 13: CLAUDE.md and Project-Wide Rules

In the previous chapter, we saw how skills provide modular, on-demand context. Now we address a complementary pattern: **project-level context files** that define the rules, standards, and conventions every AI agent should follow when working in your codebase.

The most important of these is CLAUDE.md — a project specification file that Claude Code reads at startup, similar to how developers use README.md or .editorconfig.

15.1. The Problem: AI Agents Don't Know Your Project's Rules

When an AI agent starts working on a codebase it has never seen, it makes assumptions:

- “I’ll write tests my way”
- “I’ll pick a code style”
- “I’ll structure documentation however I think is best”
- “I’ll implement features without checking with anyone”

These assumptions rarely align with the project’s actual conventions. The result: inconsistent code, missing tests, documentation

that doesn't match the style, and features that don't match requirements.

15.2. The Solution: CLAUDE.md

CLAUDE.md is a project file that tells AI agents how to work in your codebase. It's named after Claude Code (the AI coding assistant from Anthropic), but the pattern applies to any AI assistant.

15.2.1. What CLAUDE.md Contains

```
# Project Context for Claude Code

## Project Overview
- Name: MyApp
- Type: TypeScript/Node.js web application
- Core functionality: E-commerce platform

## Testing Requirements
- All features must include tests in `__tests__` directory
- Use Vitest for unit tests, Playwright for E2E
- Minimum 80% code coverage for new code
- Run tests before committing: `npm test`

## Code Style
- Use TypeScript strict mode
- No `any` types - use `unknown` or specific types
- Prefix test files with the module name: `auth.test.ts`
- Use functional components in React, no class components
```

Documentation

- Document all public APIs with JSDoc
- Keep README.md updated with setup instructions
- Use ADR format for architectural decisions

Workflow

- Always run type checks before committing: ``npm run typecheck``
- Use conventional commits: ``feat:`, `fix:`, `docs:``
- Request review before merging PRs

This is not a complete file — it's a starting point. Let's expand each section.

15.2.2. Structure: Testing, Style, Documentation Requirements

The most effective CLAUDE.md files cover three areas:

15.2.2.1. Testing Requirements

Testing Requirements**### Unit Tests**

- Framework: Vitest
- Location: ``__tests__/`` next to source files
- Naming: ``{module}.test.ts``
- Run: ``npm test`` (must pass before commit)
- Coverage: Minimum 80% for new code

Integration Tests

- Location: `tests/integration/`
- Database: Use test containers (PostgreSQL, Redis)
- Mock external APIs with MSW

E2E Tests

- Framework: Playwright
- Location: `tests/e2e/`
- Run: `npm run test:e2e`
- Must pass in CI before deployment

15.2.2.2. Code Style

Code Style

TypeScript

- Strict mode enabled
- No `any` - use `unknown` or specific types
- Explicit return types for public functions
- Use `interface` over `type` for public APIs

Formatting

- Use Prettier with config in `.prettierrc`
- Run format before commit: `npm run format`

Naming

- PascalCase for components and classes
- camelCase for variables and functions
- SCREAMING_SNAKE_CASE for constants
- Prefix interfaces with `I`: `IUserService`

15.2.2.3. Documentation

```
## Documentation

### Required Documentation
- README.md: Setup, configuration, running locally
- CONTRIBUTING.md: How to contribute
- API.md: API endpoints and examples
- CHANGELOG.md: Version history

### Code Documentation
- JSDoc for all public APIs
- Comments for complex logic (explain WHY, not WHAT)
- TODO comments with ticket numbers

### Architecture Decision Records (ADRs)
- Location: `docs/adr/`
- Format: `001-adr-title.md`
- Required for any significant architectural decision
```

15.3. Initialisation Files (INITIAL.md) for Feature Requests

Beyond project-wide rules, you can provide **task-specific context** using INITIAL.md files.

15.3.1. What INITIAL.md Contains

Feature Request: User Authentication

Problem

Users cannot securely authenticate. We need email/password

Requirements

1. Email/password login with bcrypt hashing
2. OAuth via Google and GitHub
3. JWT tokens with 1-hour expiry
4. Refresh tokens with 7-day expiry
5. Password reset via email

Technical Constraints

- Use existing PostgreSQL database
- Reuse `User` table, add OAuth columns
- Integrate with existing email service
- Follow auth patterns in `src/auth/`

Out of Scope

- Two-factor authentication (v2)
- Social login beyond Google/GitHub (v2)
- User management dashboard (v2)

Success Criteria

- [] Users can register with email/password
- [] Users can login with Google OAuth
- [] Users can login with GitHub OAuth
- [] JWT tokens are validated on API calls
- [] All tests pass

15.3.2. How INITIAL.md Works

When starting a new feature, create an INITIAL.md in the feature branch:

```
git checkout -b feature/user-auth
cat > INITIAL.md << 'EOF'
# Feature Request: User Authentication
...
EOF
```

The AI agent reads INITIAL.md and uses it as the task specification.

15.4. Examples Folder as the Source of Truth

Words can be ambiguous. Code examples are not. An **examples/** folder shows the AI agent exactly what you expect.

15.4.1. Folder Structure

```
examples/
├── auth/
│   ├── auth-service.ts      # Example service
│   ├── auth-controller.ts   # Example controller
│   └── auth.test.ts         # Example test
├── api/
│   └── rest-endpoint.ts     # Example REST endpoint
└── components/
```

15. Chapter 13: CLAUDE.md and Project-Wide Rules

```
| └─ react-component.tsx # Example component
| └─ database/
|     └─ prisma-query.ts      # Example database query
```

15.4.2. Using Examples in CLAUDE.md

```
## Examples
When implementing new features, reference examples in `examples/`
- Auth: `examples/auth/auth-service.ts`
- API: `examples/api/rest-endpoint.ts`
- Components: `examples/components/react-component.tsx`
```

This prevents “I didn’t know the pattern” problems.

15.5. The PRP (Product Requirements Prompt) Workflow Pattern

The **PRP** (Product Requirements Prompt) is a structured way to give AI agents complete context for building features.

15.5.1. PRP Template

```
# Product Requirements Prompt: {Feature Name}

## Problem Statement
{1-2 sentences describing the problem this feature solves}
```


15.5. The PRP (Product Requirements Prompt) Workflow Pattern

User Story

As a {type of user}, I want {capability}, so that {benefit}.

Acceptance Criteria

- [] {Specific, testable criterion}
- [] {Another criterion}
- [] {Edge case to handle}

Technical Requirements

- {Technical constraint 1}
- {Technical constraint 2}

Dependencies

- {Other features/components this depends on}
- {External services/APIs needed}

Out of Scope

- {What NOT to implement}

Success Metrics

- {How we'll measure success}

15.5.2. PRP Workflow

1. **Create PRP** before starting work (like this INITIAL.md above)
2. **Agent reads PRP** and asks clarifying questions
3. **Iterate** until PRP is complete
4. **Agent implements** against acceptance criteria
5. **Verify** against PRP before considering done

15.6. Beyond Claude Code: Applying Patterns to Other AI Assistants

The CLAUDE.md pattern works with other AI coding assistants:

Assistant	Config File	Location
Claude Code	CLAUDE.md	Project root
Cursor	.cursorrules	Project root
Windsurf	.windsurf	Project root
VS Code Copilot	copilot.md	.github/

15.6.1. Universal Template

```
# Project Rules

## About
- Project name and type
- What it does

## Coding Standards
- Language-specific rules
- Testing requirements

## Workflow
- How to contribute
- Commit conventions

## Useful Commands
- `npm test`
- `npm run build`
- etc.
```

15.7. Core Principle: Provide Full Screenplay, Not Sticky Notes

The fundamental principle: **be explicit, not implicit.**

□ **Bad** (sticky notes):

Write tests for new code.

□ **Good** (full screenplay):

All new code must include:

- Unit tests in `__tests__`/ with 80% coverage
- Run ``npm test`` before committing
- Use `describe/it` syntax from Vitest
- Mock external services with `vi.mock()`

The difference is specificity. Sticky notes leave room for interpretation. Full screenplay removes it.

15.8. Summary: Project-Level Context

CLAUDE.md and related files provide the project-level context that skills don't cover:

1. **CLAUDE.md**: Project-wide rules for testing, style, documentation
2. **INITIAL.md**: Task-specific context for features
3. **examples/**: Concrete code examples showing expected patterns
4. **PRP workflow**: Structured requirements that AI agents can verify against

These patterns ensure AI agents work the way you expect — not the way they assume.

15.9. Key Sources

- Claude Code documentation
- Anthropic's AI engineering best practices
- PRP pattern from context engineering community
- Zestic AI framework on artefact governance

16. Chapter 14: Validation and Quality Gates

In the previous chapter, we saw how CLAUDE.md and project rules define expected behavior. Now we address how to **validate** that behavior — detecting errors, drift, and issues before they reach production.

This chapter presents validation as a layered system: **pre-LLM validation** catches problems before the model runs, **post-LLM validation** checks its outputs, and **continuous monitoring** detects drift over time.

16.1. KLS (Knowledge Layer Schema) Validation Protocol

KLS (Knowledge Layer Schema) is a custom validation protocol developed for the context engineering workflow described in this book. It is not an industry standard – rather, it is a structured approach to reviewing, validating, and approving AI-generated content that you can adapt for your own projects.

16.1.1. What KLS Validates

KLS operates across multiple dimensions:

Dimension	What It Checks	Example
Conceptual	Core claims, concepts, definitions	Are claims internally consistent?
Structural	Organization, flow, completeness	Does it have all required sections?
Technical	Code correctness, syntax	Does the code compile?
Semantic	Grounding in knowledge base	Are facts verified?
Style	Formatting, tone, length	Does it match style guide?

16.1.2. KLS Schema Structure

```
# KLS Validation Schema
meta:
  manuscript_title: "Context Engineering Book"
  reviewer: { name: "Reviewer", role: "peer_reviewer" }
  review_date: "2026-03-28"
  review_type: "technical" # technical | editorial | conceptual
  status: "in_progress" # in_progress | complete | not_started

conceptual_model:
  core_claim_one_sentence: "Context engineering is the management of context"
  concepts:
    - id: "C1"
      label: "Context Engineering"
      definition: "The discipline of managing AI context"
      measurable_proxies: ["drift_detection_rate", "contextual_accuracy"]
```

```
relations:
  - id: "R1"
    type: "causal"
    from: "C1" # Context engineering
    to: "C2" # Reduced failures
    statement: "Context engineering reduces AI failures"
    confidence: 0.9

validation_results:
  syntax: { passed: true, errors: [] }
  semantics: { passed: true, grounding: "verified" }
  style: { passed: true, issues: [] }

# Final verdict
recommendation:
  status: "approved" # approved | needs_revision | rejected
  confidence: 0.85
  notes: "Strong conceptual model, minor style issues"
```

16.1.3. KLS Workflow

1. **Generate content:** LLM produces output
2. **Run KLS validation:** Check against schema
3. **Flag issues:** Identify problems
4. **Iterate:** Fix and re-validate
5. **Approve:** Mark as validated

```
async function klsValidate(content, schema) {
  const results = {
    syntax: validateSyntax(content),
    semantics: validateSemantics(content),
    style: validateStyle(content),
```

```
    grounding: validateGrounding(content)
  };

  const passed = results.syntax.passed &&
    results.semantics.passed &&
    results.style.passed;

  return {
    results,
    passed,
    recommendation: passed ? "approved" : "needs_revision",
  };
}
```

16.2. Pre-LLM Validation Layers: Syntax, Semantics, Grounding

Validation should happen **before** the LLM generates content — catching problems early is cheaper than fixing them later.

16.2.1. Layer 1: Syntax Validation

Check that inputs are well-formed:

```
function validateSyntax(input) {
  const errors = [];

  // Check JSON validity
  if (input.format === "json") {
    try {
```



```
    JSON.parse(input.content);
  } catch (e) {
    errors.push({ type: "JSON_SYNTAX", message: e.message });
  }
}

// Check required fields
for (const field of input.required_fields) {
  if (!input.data[field]) {
    errors.push({ type: "MISSING_FIELD", field });
  }
}

return { passed: errors.length === 0, errors };
}
```

16.2.2. Layer 2: Semantic Validation

Check that inputs make sense:

```
function validateSemantics(input) {
  const warnings = [];

  // Check for contradictory requirements
  if (input.requirements) {
    const contradictions = findContradictions(input.requirements);
    warnings.push(...contradictions);
  }

  // Check for impossible constraints
  if (input.constraints) {
    const impossible = findImpossibleConstraints(input.constraints);
    if (impossible.length > 0) {
```

```
        warnings.push({ type: "IMPOSSIBLE", constraints: in
    }
  }

  return { passed: warnings.length === 0, warnings };
}
```

16.2.3. Layer 3: Grounding Validation

Check that inputs are grounded in known facts:

```
function validateGrounding(input, knowledgeGraph) {
  const ungrounded = [];

  // Extract entities from input
  const entities = extractEntities(input.content);

  // Check each entity exists in knowledge graph
  for (const entity of entities) {
    if (!knowledgeGraph.has(entity)) {
      ungrounded.push(entity);
    }
  }

  return {
    passed: ungrounded.length === 0,
    ungrounded,
    coverage: (entities.length - ungrounded.length) / ent
  };
}
```

16.2.4. The Validation Pipeline

```
async function preLLMValidation(input, knowledgeGraph) {
  const results = {
    syntax: validateSyntax(input),
    semantics: validateSemantics(input),
    grounding: validateGrounding(input, knowledgeGraph)
  };

  const allPassed = results.syntax.passed &&
    results.semantics.passed &&
    results.grounding.passed;

  if (!allPassed) {
    // Don't call LLM - fix inputs first
    throw new ValidationError(results);
  }

  return results;
}
```

16.3. Drift Detection: Monitoring Context Changes Over Time

Context drift is the gradual misalignment we discussed in Chapter 1. Drift detection monitors for this over time.

16.3.1. What Causes Drift

- **Vocabulary drift:** Same terms mean different things

- **Assumption drift:** Unstated assumptions change
- **Relationship drift:** Entity relationships shift
- **Goal drift:** Original objectives evolve

16.3.2. Drift Detection Metrics

```
class DriftDetector {
  constructor(baseline) {
    this.baseline = baseline;
    this.history = [];
  }

  detectDrift(current) {
    const metrics = {
      vocabularyDrift: this.compareVocabulary(baseline, current),
      assumptionDrift: this.compareAssumptions(baseline, current),
      relationshipDrift: this.compareRelationships(baseline, current)
    };

    const totalDrift = (
      metrics.vocabularyDrift * 0.3 +
      metrics.assumptionDrift * 0.3 +
      metrics.relationshipDrift * 0.4
    );

    return {
      metrics,
      totalDrift,
      isCritical: totalDrift > 0.3,
      recommendation: totalDrift > 0.3 ? "REBASELINE" : "OK"
    };
  }
}
```

16.3. Drift Detection: Monitoring Context Changes Over Time

```
compareVocabulary(baseline, current) {
  const baselineTerms = new Set(baseline.vocabulary);
  const currentTerms = new Set(current.vocabulary);

  const added = [...currentTerms].filter(t => !baselineTerm
  const removed = [...baselineTerms].filter(t => !currentTe

  return (added.length + removed.length) / baselineTerms.si
}
}
```

16.3.3. Drift Alerts

```
// Alert thresholds
const DRIFT_WARNING = 0.15; // 15% drift
const DRIFT_CRITICAL = 0.30; // 30% drift

// Monitor in production
setInterval(() => {
  const drift = detector.detectDrift(currentContext);

  if (drift.isCritical) {
    notify(`CRITICAL: Context drift ${drift.totalDrift.toFixed(2)}%`);
    pauseNewWork();
    triggerRebaseline();
  } else if (drift.totalDrift > DRIFT_WARNING) {
    notify(`WARNING: Context drift ${drift.totalDrift.toFixed(2)}%`);
  }
}, 24 * 60 * 60 * 1000); // Daily check
```

16.4. Impact Analysis: What Changes Affect What

When context changes, you need to know **what else might be affected**.

16.4.1. Dependency Graph

```
const contextDependencies = {
  "AGENTS.md": ["SOUL.md", "skills/*", "memory/*"],
  "domain-model.yaml": ["prompts/*", "knowledge-graph/*"],
  "skill-catalog.yaml": ["skills/*", "compression/*"],
  "prompts/system-prompt.md": ["prompts/*", "validation/*"]
};
```

16.4.2. Impact Analysis Function

```
function analyzeImpact(changedFile) {
  const affected = new Set();
  const visited = new Set();

  function traverse(file) {
    if (visited.has(file)) return;
    visited.add(file);

    for (const dependent of contextDependencies[file] || []) {
      affected.add(dependent);
      traverse(dependent);
    }
  }
}
```

```

}

traverse(changedFile);

return {
  changed: changedFile,
  affected: [...affected],
  severity: affected.length > 10 ? "HIGH" :
            affected.length > 5 ? "MEDIUM" : "LOW"
};
}

```

16.5. Integration with CI/CD Pipelines

Validation should be **automated** in your CI/CD pipeline.

16.5.1. CI Pipeline Stages

```

# .github/workflows/ai-validation.yml
stages:
  - name: Pre-LLM Validation
    run: |
      npm run validate:input

  - name: LLM Generation
    run: |
      npm run generate

  - name: KLS Validation
    run: |

```

```
npm run validate:kls

- name: Post-Generation Validation
  run: |
    npm run validate:output

- name: Drift Detection
  run: |
    npm run detect:drift

- name: Impact Analysis
  run: |
    npm run analyze:impact

- name: Gate Decision
  run: |
    # Only proceed if all validations pass
    if [ "$GATE_STATUS" = "PASSED" ]; then
      echo "Proceeding to next stage..."
    else
      echo "GATE FAILED - review required"
      exit 1
    fi
```

16.5.2. Git Hooks

```
# .git/hooks/pre-commit
#!/bin/bash
npm run validate:kls
npm run detect:drift
```


16.6. LLM as a Judge for Context Validation

The validation layers above are deterministic – they check computable properties like syntax, schema conformance, and graph coverage. But some quality dimensions resist deterministic checking: Is the output *helpful*? Is it *complete*? Does it *correctly address* the user’s intent?

LLM-as-Judge evaluation uses a separate LLM (distinct from the generator) to score outputs against defined rubrics. This fills the gap between deterministic checks and full human review.

16.6.1. Four Evaluation Modes

Mode	Description	Best For
Pointwise	Score a single response in isolation	Quality assessment, moderation
Reference-based	Compare to a known good answer	Accuracy, regression testing
Pairwise	Pick the better of two responses	A/B testing, preference data
Listwise	Rank multiple responses	Leaderboards, best-of-N selection

Reference-based evaluation is the most reliable mode when ground truth exists. Pointwise evaluation (scoring without a reference) is the least reliable and should be avoided for quality gates.

16.6.2. Known Biases

LLM judges introduce systematic biases that must be mitigated:

Bias	Description	Mitigation
Position bias	Favours the first response shown (10-20% swing in pairwise)	Run comparisons twice with swapped positions
Verbosity bias	Longer responses score higher regardless of quality	Include length-penalised scoring criteria
Self-enhancement bias	Models prefer outputs matching their own style	Use a different model family for judging than for generation
Authority bias	Confident responses score higher even when hedging is appropriate	Require evidence citations in judge reasoning
Format bias	Well-formatted responses (markdown, bullets) score higher	Evaluate content separately from presentation

16.6.3. The Hybrid Verification Pipeline

The recommended approach combines deterministic and LLM-based validation:

```
async function hybridValidation(content, knowledgeGraph,
  // Layer 1: Deterministic checks (free, instant)
  const syntaxResult = validateSyntax(content);
  const groundingResult = validateGrounding(content, know

  if (!syntaxResult.passed || !groundingResult.passed) {
```

```

    return { passed: false, stage: "deterministic", results: {
  }

  // Layer 2: LLM-as-Judge for nuance (costs per evaluation)
  const judgeResult = await llmJudge(content, rubric, {
    mode: "reference_based",
    model: "different_from_generator",
    requireReasoning: true
  });

  if (judgeResult.score < rubric.minimumScore) {
    return { passed: false, stage: "llm_judge", results: judgeResult
  }

  // Layer 3: Sampling for human calibration (5-10% of evaluations)
  if (shouldSampleForHumanReview()) {
    queueForHumanReview(content, judgeResult);
  }

  return { passed: true, results: { syntaxResult, groundingResult,
}

```

16.6.4. Calibration Against Human Judgement

Before deploying LLM judges in production, calibrate against 100-200 human-annotated examples. Measure:

- **Correlation:** Does the LLM ranking match human ranking?
- **Cohen's Kappa:** Agreement beyond what chance would produce?
- **Systematic bias:** Does the judge consistently score higher or lower than humans?

If disagreement exceeds 20-30%, revisit the rubric or switch judge models. LLM judges drift as models update, so recalibration is not a one-time activity – it is ongoing maintenance.

16.6.5. When NOT to Use LLM Judges

Use deterministic validation instead when:

- **Ground truth exists:** Maths, code execution, structured output – use computable checks
- **Safety-critical decisions:** Content moderation, medical – use as one signal among many with mandatory human review
- **High volume, low stakes:** Millions of responses where simpler classifiers are more cost-effective

LLM-as-Judge is most valuable in the middle ground: outputs that are too nuanced for rule-based checks but too numerous for full human review.

16.7. Where Validation Criteria Come From: Context Engineering Meets Spec-Driven Development

The validation layers above – KLS, pre-LLM checks, drift detection, LLM-as-Judge – share a hidden dependency. Each one requires **criteria**: a schema to validate against, a knowledge graph to ground against, a rubric for the judge to score against. Where do those criteria come from?

If the criteria are wrong, perfect verification still produces wrong outputs. A flawless type checker enforcing an incorrect domain

contract protects nothing. A BDD scenario that encodes a misunderstood business rule will pass every time, confirming the wrong behaviour. Validation is only as trustworthy as the specifications it enforces.

This is the upstream problem that connects everything in this book. In Chapter 3, we established context engineering as the discipline of managing semantic alignment – ensuring that every participant (human and machine) shares the same understanding of the problem space. In Chapter 4, we introduced the reconciled domain model as the anchor: the shared vocabulary of entities, relationships, and invariants that defines what the system means. The validation layers in this chapter are the *downstream operationalisation* of that upstream work. But we have not yet made the connection explicit. A complementary discipline – **spec-driven development** – provides the missing link.

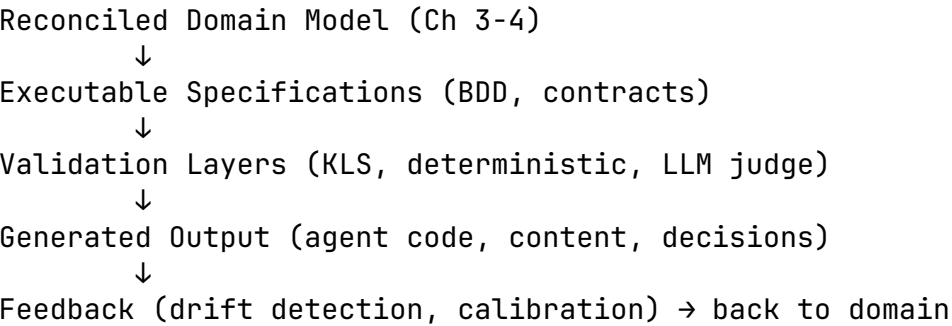
16.7.1. The Two-Layer Model

Context engineering and spec-driven development are complementary layers, not competing approaches. They operate at different altitudes and answer different questions:

	Context Engineering	Spec-Driven Development
Operates at	Problem space	Solution space
Primary artefact	Reconciled domain model	Executable specifications
Question answered	“Does everyone mean the same thing?”	“Is the generated output correct?”
Human role	Curate semantic alignment	Define acceptance criteria

	Context Engineering	Spec-Driven Development
Failure mode	Agents do the wrong thing coherently	Agents do the right thing incorrectly
Automation	KG search, hooks, CLAUDE.md	Test runners, type checkers, verification agents

The relationship is sequential. Context engineering produces the domain model – the shared understanding of entities, relationships, and business rules. Spec-driven development consumes that model and transforms it into executable specifications: BDD scenarios, domain contracts, acceptance criteria. Those specifications then feed the validation layers described earlier in this chapter.



This maps directly to a stage-gated lifecycle. In the Zestic AI Development Process (discussed further in Chapter 18), the Discovery and Define stages are context engineering: building the domain model, establishing vocabulary, aligning stakeholders. The Design and Develop stages are spec-driven development: translating that context into specifications and implementing against them. The

two layers are not separate activities bolted together – they are the upstream and downstream halves of a single quality pipeline.

The critical insight: **you need a reconciled domain model before specifications are meaningful.** A BDD scenario that says “Given a user with insufficient funds” only works if everyone agrees on what “user,” “insufficient,” and “funds” mean in this domain. Without upstream context engineering, specs encode assumptions that may be wrong – and the verification machinery will happily confirm those wrong assumptions at every gate.

16.7.2. The Swiss Cheese Model of Verification

Ankit Jain, writing in Latent Space, articulates why this two-layer model matters for AI-native development. His argument: human code review was not ubiquitous until 2012-2014, and it is now an unsustainable bottleneck. Teams with high AI adoption merge 98% more pull requests, but review time increases by 91%. The solution is not faster review – it is layered verification that moves human judgement upstream from “Did you write this correctly?” to “Are we solving the right problem?”

Jain proposes a five-layer Swiss Cheese model, where each layer catches failures that the others miss:

Layer 1: Compare multiple options. Ask three agents to implement independently. Rank by verification pass rate, smallest diff, or fewest new dependencies. This is a competition pattern – the agents do not collaborate, they compete, and the best output wins.

Layer 2: Deterministic guardrails. Tests, type checkers, contract verification – pass/fail artefacts, not opinions. This maps directly to

the pre-LLM validation layers earlier in this chapter: syntax checking, semantic validation, and knowledge graph grounding. These guardrails enforce what is computable without intelligence.

Layer 3: Humans define acceptance criteria. BDD specifications authored by humans before any code is generated. The specification is the primary artefact, not additional documentation. The agent implements; the BDD framework verifies; the human never reads the implementation unless verification fails. This is the spec-driven development layer – and it depends on the context engineering upstream to ensure the specs themselves are correct.

Layer 4: Permission systems as architecture. Narrow the filesystem scope per task. Trigger automatic human review for sensitive changes: authentication logic, database schema modifications, new dependencies. This is scope management, not intelligence – a structural constraint that limits blast radius regardless of agent confidence.

Layer 5: Adversarial verification. Separate the coding agent from the verification agent architecturally. The agent that writes the code never reviews its own output. An optional red-team agent attempts to break what the first agent built. This connects to the build-review codon loops discussed in the next chapter, where context-walled agents evaluate each other's work with fresh eyes.

No single layer is sufficient. The power comes from composition: if a wrong assumption passes through the domain model (Layer 3 missed it), the deterministic guardrails (Layer 2) might catch the resulting type violation. If the type system is too permissive, the adversarial agent (Layer 5) might find the flaw. Defence in depth means that failures must penetrate every layer simultaneously to reach production.

16.7.3. BDD Specifications as the Bridge Artefact

Behaviour-Driven Development specifications occupy a unique position in this architecture. They are the artefact that bridges upstream context and downstream verification – human-readable enough to validate against the domain model, machine-executable enough to serve as automated tests.

A BDD scenario has a precise structure:

Feature: Fund Transfers

Scenario: Insufficient funds

Given a user with 50 GBP in their account

When they attempt to transfer 100 GBP

Then the transfer is blocked

And an `InsufficientFunds` error is returned

And the account balance remains 50 GBP

Every noun in this scenario – “user,” “account,” “GBP,” “transfer,” “`InsufficientFunds`” – should be a term in the reconciled domain model. Every relationship – a user *has* an account, a transfer *debits* an account – should match the model’s entity relationships. Every constraint – “the account balance remains 50 GBP” – should derive from a documented business invariant. If any of these terms are undefined or ambiguous in the domain model, the specification is built on sand.

This is where context engineering directly feeds spec-driven development. The domain model provides the vocabulary; the BDD spec uses that vocabulary to express testable behaviour; the validation layers execute those tests against the generated implementation. The human’s job shifts from reviewing code to reviewing specifications – a fundamentally different activity that operates at a higher level of abstraction.

The workflow becomes:

1. **Context engineering** produces the domain model (entities, relationships, invariants)
2. **Humans write BDD specs** using domain model vocabulary
3. **Agents implement** against the specs (humans do not review the code)
4. **Automated verification** runs the BDD scenarios, type checks, and contract validation
5. **Failures feed back** to either the spec (if the requirement was unclear) or the agent (if the implementation was wrong)

Human time concentrates on steps 1 and 2 – understanding the domain and expressing requirements. Agent time concentrates on step 3. Verification is automated. This is the practical expression of the two-layer model: context engineering ensures the specs are right; spec-driven development ensures the code matches the specs.

Tools like Spec-Kit (an open-source toolkit with wide adoption across AI coding assistants) operationalise this workflow with a five-phase process: constitution (project-wide rules), specification (user scenarios), plan (technical design), tasks (executable work items), and implementation. The parallels to this book’s approach are direct – the constitution maps to CLAUDE.md (Chapter 13), the specification phase maps to the BDD layer described here, and the planning phase maps to the domain model work in Chapters 3-4. The key difference is that Spec-Kit assumes context is given (starting from a constitution), while context engineering treats context as the thing that must be engineered first.

16.7.4. Implications for the Validation Architecture

This synthesis reshapes how we think about the validation layers described earlier in this chapter:

Deterministic checks (syntax, semantics, grounding) validate against the **domain model**. When the grounding layer checks whether entities exist in the knowledge graph, it is confirming that the output uses the vocabulary established by context engineering. This is upstream validation – checking that the right concepts are in play.

LLM-as-Judge evaluation validates against the **BDD specifications**. When the judge scores an output against a rubric, that rubric should derive from the acceptance criteria in the specs. This is downstream validation – checking that the output satisfies the requirements expressed in those specs.

Human review moves upstream to **specification approval**. Instead of reviewing generated code line by line (which Jain’s data shows is an unsustainable bottleneck), humans review specifications before generation begins. The question changes from “Is this code correct?” to “Does this spec capture what we actually need?” This is a fundamentally more tractable review task – specifications are shorter, expressed in domain language, and focused on behaviour rather than implementation.

The hybrid verification pipeline from the previous section gains a new dimension. It is not just deterministic-then-LLM-then-human. It is *domain-model-grounded deterministic checks*, followed by *spec-derived LLM evaluation*, followed by *upstream human spec review*. Each layer has a clear source of truth, and that source traces back through specifications to the reconciled domain model.

This completes the argument of this chapter. Validation is not merely a set of technical checks applied after generation. It is the downstream operationalisation of upstream context engineering – the mechanism by which semantic alignment becomes verifiable quality.

16.8. Summary: Validation as Gate

Validation transforms AI generation from “trust me” to “prove it”:

1. **KLS protocol:** Custom structured validation across conceptual, structural, technical dimensions
2. **Pre-LLM validation:** Catch syntax, semantics, grounding issues before generation
3. **Drift detection:** Monitor context changes over time, trigger rebaselining
4. **LLM-as-Judge:** Automated evaluation for nuanced quality dimensions, with bias mitigation
5. **Impact analysis:** Know what changes affect what
6. **CI/CD integration:** Automate validation in pipelines
7. **Context-to-spec pipeline:** Validation criteria derive from specifications, which derive from the reconciled domain model – context engineering upstream, spec-driven verification downstream

These gates ensure that AI outputs meet standards before they reach users – and that the standards themselves are grounded in a shared understanding of the problem space.

16.9. Key Sources

- KLS Schema Reference (custom validation protocol)
- Claude Code hooks and automation
- Context engineering validation patterns
- LLM-as-Judge best practices literature (Zheng et al., MT-Bench; Huyen, AI Engineering, 2025)
- Drift detection literature
- Ankit Jain, “How to Kill the Code Review” (Latent Space, 2025)
 - Swiss Cheese verification model

- Spec-Kit (GitHub) – spec-driven development toolkit
- Dan North, “Introducing BDD” (2006) – Behaviour-Driven Development origins

17. Chapter 15: Multi-Agent Context Management

In the previous chapter, we saw how validation gates ensure quality. Now we address how to manage context when **multiple AI agents work together** — orchestrating sub-agents, sharing context, and maintaining coherence.

Modern AI systems rarely use a single agent. They spawn sub-agents for parallel work, delegate tasks, and coordinate responses. This creates new context challenges: what context should each agent have? How do they share information? How do you prevent context explosion?

17.1. Sub-Agent Orchestration Patterns in Claude Code

Claude Code supports **sub-agent spawning** — creating child agents that can work independently or in parallel.

17.1.1. Spawn Patterns

17. Chapter 15: Multi-Agent Context Management

```
// Pattern 1: Sequential sub-agents
// Agent A does part 1, passes to Agent B for part 2
async function sequentialWork(task) {
  const part1 = await spawn({
    task: "Analyze requirements",
    runtime: "subagent"
  });

  const part2 = await spawn({
    task: "Implement based on analysis",
    context: part1.results, // Pass context
    runtime: "subagent"
  });

  return part2.results;
}

// Pattern 2: Parallel sub-agents
// Agents A, B, C work simultaneously, then merge
async function parallelWork(task) {
  const [analysis, implementation, testing] = await Promise.all([
    spawn({ task: "Analyze requirements", runtime: "subagent" }),
    spawn({ task: "Implement features", runtime: "subagent" }),
    spawn({ task: "Write tests", runtime: "subagent" })
  ]);

  return mergeResults([analysis, implementation, testing]);
}

// Pattern 3: Hierarchical sub-agents
// Manager agent coordinates specialist agents
async function hierarchicalWork(task) {
  const manager = await spawn({
    task: "Coordinate feature implementation",
```



```

    runtime: "subagent"
  });

  // Manager spawns specialists internally
  return manager.results;
}

```

17.1.2. Context Passing

```

// What gets passed to sub-agents?
const subAgentConfig = {
  // Minimal: just what's needed
  context: {
    task: "Specific task description",
    relevantFiles: ["file1.ts", "file2.ts"],
    constraints: ["Use TypeScript", "No external APIs"],
    background: "Previous work summary"
  },

  // Or full context (careful with token limits)
  fullContext: false, // Don't pass everything
  maxTokens: 8000     // Limit sub-agent context
};

```

17.2. Session Persistence and Memory Across Turns

When an agent works across multiple turns, it needs **memory** — not just the immediate context, but what happened before.

17.2.1. Memory Types

Type	Contents	Retention
Working memory	Current conversation	Current session
Session memory	Recent turns (last 50)	Current session
Long-term memory	Past sessions, key learnings	Permanent
Shared memory	Context visible to all agents	Per orchestration

17.2.2. Memory Management

```

class AgentMemory {
  constructor() {
    this.working = [];      // Current turn
    this.session = [];      // Last N turns
    this.longTerm = {};    // Persistent key-value
  }

  // Add to current turn
  addWorking(content) {
    this.working.push(content);
  }

  // Promote to session
  promoteToSession() {
    this.session.push(...this.working);
    this.working = [];
  }
}

```

```

    // Trim if too long
    if (this.session.length > 50) {
      this.session = this.session.slice(-50);
    }
  }

  // Store long-term
  storeLongTerm(key, value) {
    this.longTerm[key] = value;
  }

  // Retrieve with decay (older = less relevant)
  retrieve(query, maxAge = 10) {
    const recent = this.session
      .filter(t => t.age < maxAge)
      .filter(t => t.relevance(query) > 0.5);

    return recent;
  }
}

```

17.2.3. Persistence Strategies

```

// Strategy 1: Summarize and store
async function persistMemory(memory) {
  const summary = await summarize(memory.session);
  await store("sessions/current", {
    summary,
    keyDecisions: memory.keyDecisions,
    learnedPatterns: memory.learnedPatterns
  });
}

```

```
// Strategy 2: Compress old turns
function compressMemory(memory, maxTokens = 2000) {
  // Keep recent turns verbatim
  // Compress older turns
  const recent = memory.session.slice(-10);
  const older = memory.session.slice(0, -10);
  const compressed = compress(older);

  return [...compressed, ...recent];
}
```

17.3. Context Isolation vs Shared Context Tradeoffs

When multiple agents work together, you face a fundamental trade-off: **isolation** (each agent has its own context) vs **sharing** (agents see each other's context).

17.3.1. Isolation Model

```
// Each agent gets isolated context
const isolated = {
  agents: [
    { id: "A", context: { files: ["file1.ts"] } },
    { id: "B", context: { files: ["file2.ts"] } },
    { id: "C", context: { files: ["file3.ts"] } }
  ],
  // Agents cannot see each other's work
}
```

```
// Safe but redundant  
};
```

Pros: - No cross-contamination - Each agent focused on its task -
Simpler to reason about

Cons: - Duplicated context (three agents, three copies) - No collaboration on shared concepts - Harder to maintain coherence

17.3.2. Shared Context Model

```
// All agents see shared context  
const shared = {  
  sharedContext: {  
    files: ["file1.ts", "file2.ts", "file3.ts"],  
    knowledgeGraph: kg,  
    memory: sharedMemory  
  },  
  agents: [  
    { id: "A", role: "analyzer", access: "full" },  
    { id: "B", role: "implementer", access: "full" },  
    { id: "C", role: "reviewer", access: "full" }  
  ]  
};
```

Pros: - Efficient (one copy of context) - Natural collaboration -
Coherent outputs

Cons: - Risk of context pollution - Harder to debug - Token limits
hit faster

17.3.3. Hybrid Approach

```
// Most context isolated, key info shared
const hybrid = {
  // Shared: essential information
  shared: {
    knowledgeGraph: kg,
    currentTask: task.description,
    constraints: task.constraints,
    decisions: [] // Append-only decision log
  },

  // Isolated: agent-specific
  agents: [
    {
      id: "A",
      isolated: {
        files: ["file1.ts"],
        analysis: []
      }
    },
    // ...
  ]
};
```

17.4. Terraphim Knowledge Graph Integration for Agents

The Terraphim knowledge graph (Chapters 5-9) provides a **shared semantic backbone** that multiple agents can use.

17.4.1. Agent-KG Integration

```
class AgentKGIntegration {
  constructor(knowledgeGraph) {
    this.kg = knowledgeGraph;
  }

  // Agent queries the shared knowledge graph
  async query(agent, query) {
    const results = await this.kg.search(query);

    // Log query for context
    await this.log(agent, query, results);

    return results;
  }

  // Agent validates against KG
  async validate(agent, output) {
    const entities = extractEntities(output);
    const validation = [];

    for (const entity of entities) {
      const exists = await this.kg.has(entity);
      const relationships = await this.kg.getRelationships(entity);

      validation.push({ entity, exists, relationships });
    }

    return validation;
  }

  // Log agent activity for audit
```

```
async log(agent, action, results) {
  const entry = {
    timestamp: Date.now(),
    agent: agent.id,
    action,
    resultsCount: results.length
  };

  await this.kg.addLogEntry(entry);
}
```

17.4.2. Shared Vocabulary

```
// All agents use the same vocabulary
const sharedVocabulary = {
  // Entities
  entities: ["Feature", "Bug", "Test", "CodeReview"],

  // Relationships
  relationships: [
    { from: "Feature", type: "implemented_by", to: "CodeReview" },
    { from: "Bug", type: "tests", to: "Test" },
    { from: "Test", type: "validates", to: "Feature" }
  ],

  // Agents can add to vocabulary
  addEntity: (entity) => vocabulary.entities.push(entity),
  addRelationship: (rel) => vocabulary.relationships.push(rel)
};
```


17.5. Build-Review Loops: Context-Walled Agent Coordination

A particularly effective multi-agent pattern is the **build-review codon loop**, articulated by Hrish at Anthropic (2025). The core insight: agents that evaluate their own work tend to praise it, even when the quality is poor. Separating the builder from the reviewer – with a context wall between them – produces dramatically better results.

17.5.1. The Context Wall Principle

Place agents in sealed steps (“codons”), each with its own context. One agent builds, another reviews. Only the output crosses the wall:

```
// Build-review codon loop
const buildReviewLoop = {
  iterations: 3,
  codons: [
    { id: "build", model: "sonnet", contextMode: "fresh" },
    { id: "review", model: "haiku", contextMode: "fresh" },
    { id: "update", model: "sonnet", contextMode: "fresh" }
  ]
};
```

Each codon starts with clean context, receiving only the output from the previous step. This prevents the self-praise problem: the reviewer has no memory of writing the code, so it evaluates with fresh eyes.

17.5.2. Model Selection Asymmetry

A counterintuitive finding from Anthropic’s work: **smaller models are better critics for plans, while larger models are better critics for code.**

For plan review, a smaller model surfaces problems that a larger model would paper over with intelligence and intuition. The point of critique is to find problems, not compensate for them. For code review, a larger model is better because it needs to spin up from fresh context, navigate the codebase, and provide precise critiques.

Loop Type	Builder	Reviewer	Rationale
Plan-Review	Opus	Haiku	Smaller critic surfaces problems
Code-Review	Sonnet	Opus	Larger critic navigates code
Visual-Review	Sonnet	Opus	Larger critic catches detail

17.5.3. Automated Review Surfaces

The most cost-effective variant replaces agentic review with **deterministic review** for checks that do not require intelligence:

- Run tests automatically between build and review codons
- Extract diffs programmatically rather than asking an agent to find them
- Collect metrics (coverage, linting, type checking) via scripts

This pattern – called “automated review surfaces” – reduced costs by 10x in Anthropic’s comic generation pipeline while improving output quality. The principle: “What is an agent doing that we don’t need an agent for?”

The combination of deterministic “rigs” (scripts between codons) and agentic review (only where judgement is needed) creates a continuous cost-to-quality curve. Each additional iteration provides diminishing but real marginal improvement, letting you trade cost for quality along a smooth gradient.

This pattern validates the hybrid approach described throughout this book: deterministic methods for what can be checked deterministically, AI judgement for what requires it.

17.6. The Meta-Cortex Concept: Agent-to-Agent Communication

The **meta-cortex** is a conceptual pattern where agents share a communication layer — a way for them to coordinate without direct intervention.

17.6.1. Meta-Cortex Architecture

```
// The meta-cortex is a shared communication layer
class MetaCortex {
  constructor() {
    this.messageBus = [];
    this.sharedState = {};
    this.agents = new Map();
  }

  // Register an agent
  register(agent) {
    this.agents.set(agent.id, {
      agent,
```

```

        subscriptions: [],
        lastSeen: Date.now()
    });
}

// Agent publishes to shared layer
async publish(agent, message) {
    const entry = {
        from: agent.id,
        type: message.type, // "finding", "request", "decision"
        content: message.content,
        timestamp: Date.now()
    };

    this.messageBus.push(entry);

    // Notify subscribers
    for (const subscriber of this.agents.values()) {
        if (subscriber.subscriptions.includes(message.type)) {
            await this.notify(subscriber.agent, entry);
        }
    }
}

// Agent subscribes to message types
subscribe(agent, messageTypes) {
    const record = this.agents.get(agent.id);
    record.subscriptions.push(...messageTypes);
}

// Query shared state
query(key) {
    return this.sharedState[key];
}

```

```
// Update shared state
update(key, value) {
  this.sharedState[key] = value;
}
}
```

17.6.2. Communication Patterns

```
// Pattern 1: Finding
agentA.publish({
  type: "finding",
  content: "Found memory leak in auth module"
});

// Pattern 2: Request
agentB.publish({
  type: "request",
  content: "Need database schema for user table"
});

// Pattern 3: Decision
agentC.publish({
  type: "decision",
  content: "Using JWT tokens, not sessions"
});

// Pattern 4: Confirmation
agentA.publish({
  type: "confirmation",
  content: "Memory leak fixed, tests passing"
});
```

17.6.3. Why Meta-Cortex Matters

1. **Coordination without orchestration:** Agents coordinate themselves
2. **Audit trail:** All communications logged
3. **Shared state:** Agents agree on common facts
4. **Decentralized:** No single point of failure

17.7. Summary: Multi-Agent Context

Multi-agent systems require explicit context management:

1. **Sub-agent orchestration:** Sequential, parallel, hierarchical patterns
2. **Session persistence:** Working, session, long-term, shared memory
3. **Isolation vs sharing:** Tradeoffs between focus and efficiency
4. **Knowledge graph integration:** Shared semantic backbone
5. **Build-review loops:** Context-walled codons with model selection asymmetry
6. **Meta-cortex:** Agent-to-agent communication layer

These patterns enable AI systems to scale beyond single agents while maintaining coherence.

17.8. Key Sources

- Claude Code sub-agent documentation
- Terraphim knowledge graph implementation
- Hrishi (Anthropic), “Agentic Programming Patterns: Build-Review Codon Loops” (2025)

- Prithvi (Anthropic), Generator-Evaluator pattern and automated review surfaces
- Multi-agent orchestration patterns
- Meta-cortex conceptual model from context engineering research

18. Chapter 16: Hooks and Automation for Context

In the previous chapter, we saw how multi-agent systems coordinate. Now we address **hooks** — the automation layer that intercepts and processes AI interactions in real-time.

Hooks are functions that run at specific points in the AI's execution: before a tool is used, after a response is generated, when a session starts. They enable validation, logging, correction, and continuous learning.

18.1. Session Hooks: Before/After LLM Calls

Session hooks run around every LLM interaction, enabling preprocessing and postprocessing.

18.1.1. Before LLM Hook

```
// Pre-process: validate and augment input
async function beforeLLMHook(context) {
  // 1. Check token budget
  const budget = getTokenBudget();
  if (budget.remaining < MIN_TOKENS) {
```

```
    await compressContext();
  }

  // 2. Add relevant context
  const relevant = await retrieveRelevantContext(context);
  context.prompt = augmentPrompt(context.prompt, relevant);

  // 3. Validate input
  const validation = validateInput(context.prompt);
  if (!validation.passed) {
    throw new ValidationError(validation.errors);
  }

  return context;
}
```

18.1.2. After LLM Hook

```
// Post-process: validate and store output
async function afterLLMHook(context, response) {
  // 1. Validate output
  const validation = validateOutput(response);
  if (!validation.passed) {
    // Retry or flag
    await handleValidationFailure(validation.errors);
  }

  // 2. Log interaction
  await logInteraction({
    query: context.query,
    response: response.content,
    tokens: response.usage,
  });
}
```

```

        duration: response.duration
    });

    // 3. Update session memory
    context.session.addTurn({
        query: context.query,
        response: response.content,
        timestamp: Date.now()
    });

    return response;
}

```

18.1.3. Hook Pipeline

```

class HookPipeline {
    constructor() {
        this.beforeHooks = [];
        this.afterHooks = [];
    }

    addBeforeHook(hook) {
        this.beforeHooks.push(hook);
    }

    addAfterHook(hook) {
        this.afterHooks.push(hook);
    }

    async runBefore(context) {
        for (const hook of this.beforeHooks) {
            context = await hook(context);
        }
    }
}

```

```
    }  
    return context;  
  }  
  
  async runAfter(context, response) {  
    for (const hook of this.afterHooks) {  
      response = await hook(context, response);  
    }  
    return response;  
  }  
}
```

18.2. Tool Use Hooks: Validation, Logging, Correction

Tool use hooks intercept tool calls — validating, logging, or even correcting them.

18.2.1. Pre-Tool-Use Hook

```
// Validate before executing a tool  
async function preToolUseHook(toolCall) {  
  // 1. Validate arguments  
  const validation = validateToolArgs(toolCall.name, toolCall.arguments);  
  if (!validation.valid) {  
    throw new ToolValidationError(validation.errors);  
  }  
  
  // 2. Check permissions
```

```

const allowed = checkPermissions(toolCall.name, toolCall.arguments);
if (!allowed) {
  throw new PermissionError(toolCall.name);
}

// 3. Log the call
await logToolCall(toolCall);

return toolCall;
}

```

18.2.2. Post-Tool-Use Hook

```

// Process tool results
async function postToolUseHook(toolCall, result) {
  // 1. Validate result
  const validation = validateToolResult(toolCall.name, result);
  if (!validation.valid) {
    // Correct if possible
    await attemptCorrection(toolCall, result);
  }

  // 2. Log result
  await logToolResult(toolCall, result);

  // 3. Check for sensitive data
  if (containsSensitiveData(result)) {
    await redact(result);
  }

  return result;
}

```

18.2.3. Example: File Edit Validation

```
// Pre-hook for file edits
async function preEditFileHook(toolCall) {
  const { path, oldString, newString } = toolCall.args;

  // Check file exists
  if (!await exists(path)) {
    throw new Error(`File not found: ${path}`);
  }

  // Check file is not protected
  const protected = [".env", "secrets.yaml", "credentials"]
  if (protected.some(p => path.endsWith(p))) {
    throw new Error(`Cannot edit protected file: ${path}`);
  }

  // Check oldString matches actual content
  const content = await readFile(path);
  if (!content.includes(oldString)) {
    throw new Error("oldString does not match file content");
  }

  return toolCall;
}
```

18.3. Claude Code Hooks: mcp-hook, pre-tool-use, post-tool-use

Claude Code provides built-in hook points:

18.3.1. MCP Hook (Model Context Protocol)

```
# .claude/hooks/mcp-hook
- name: knowledge-graph
  trigger: ".*"
  action: |
    # Query knowledge graph for relevant context
    RESULT=$(terraphim-agent query "$CLAUDEUDE" --role engine
    if [ -n "$RESULT" ]; then
      echo "--- Knowledge Graph Context ---"
      echo "$RESULT"
    fi
```

18.3.2. Pre-Tool-Use Hook

```
// In claude_settings.json
{
  "hooks": {
    "pre-tool-use": [
      {
        "match": "bash",
        "action": "inspect",
        "description": "Validate bash commands before executi
      ]
    ]
  }
}
```

18.3.3. Post-Tool-Use Hook

```
// Post-tool-use hook example
{
  "hooks": {
    "post-tool-use": [
      {
        "match": "write",
        "action": "lint",
        "description": "Lint written files for syntax errors"
      }
    ]
  }
}
```

18.3.4. Hook Configuration

```
# Complete hook configuration
hooks:
  pre_tool_use:
    - name: validate_bash
      match: bash
      action: validate
      config:
        allowed_commands: ["npm", "git", "docker"]
        timeout: 300

    - name: validate_file_edit
      match: edit
      action: validate
      config:
```



```
protected_files: [".env", "secrets/*"]

post_tool_use:
  - name: lint_code
    match: write
    action: lint

  - name: log_changes
    match: ".*"
    action: log
```

18.4. Context Mutation Warning: When Hooks Change What the LLM Sees

A critical design consideration: hooks that modify context before the LLM sees it are **context mutations**. They change what the model reasons about without the model knowing it was changed.

This is powerful but dangerous:

- **Pre-tool-use hooks** that rewrite file paths can cause the LLM to reference files it never actually read
- **Augmentation hooks** that inject retrieved context can overflow the token budget silently
- **Correction hooks** that modify outputs before returning them can mask systematic problems

The safest pattern is to make mutations **visible and auditable**:

```
// Anti-pattern: silent mutation
function preHook(context) {
  context.prompt = context.prompt.replace("npm", "bun"); //
```

```
    return context;
}

// Better: auditable mutation
function preHook(context) {
  const mutations = [];
  if (context.prompt.includes("npm")) {
    context.prompt = context.prompt.replace("npm", "bun");
    mutations.push({ type: "text_replace", from: "npm", to: "bun" });
  }
  context.metadata.mutations = mutations; // Audit trail
  return context;
}
```

Treat hooks as you would database triggers: essential for automation, but a source of subtle bugs when they mutate state invisibly.

18.5. Continuous Learning from Agent Interactions

Hooks can enable **continuous learning** — improving the AI system based on interactions.

18.5.1. Learning Hook

```
// Extract learnings from interactions
async function learningHook(context, response) {
  const learnings = [];
```

```

// 1. What worked?
if (response.quality === "high") {
  learnings.push({
    type: "success_pattern",
    pattern: extractPattern(context, response),
    confidence: 0.9
  });
}

// 2. What failed?
if (response.errors && response.errors.length > 0) {
  learnings.push({
    type: "failure_pattern",
    pattern: extractPattern(context, response),
    errors: response.errors,
    confidence: 0.8
  });
}

// 3. Store learnings
for (const learning of learnings) {
  await storeLearning(learning);
}

return response;
}

// Store in knowledge graph
async function storeLearning(learning) {
  const node = {
    type: learning.type,
    pattern: learning.pattern,
    confidence: learning.confidence,
    timestamp: Date.now()
  }

```

```
};  
  
await knowledgeGraph.add(node);  
await knowledgeGraph.connect(node, "derived_from", learn  
}
```

18.5.2. Pattern Extraction

```
// Extract reusable patterns  
function extractPattern(context, response) {  
  return {  
    // What type of task?  
    taskType: classifyTask(context.query),  
  
    // What approach worked?  
    approach: extractApproach(context.steps),  
  
    // What context was needed?  
    contextNeeded: identifyContextUsed(context),  
  
    // What tools were used?  
    toolsUsed: extractTools(response.toolCalls)  
  };  
}
```

18.6. Building the Hook Taxonomy

Organize hooks into a **taxonomy** for maintainability:

18.6.1. Hook Categories

Category	Purpose	Examples
Validation	Check inputs/outputs	input validation, output validation
Security	Protect system	permission check, redaction
Logging	Record activity	audit log, metrics
Transformation	Modify data	augmentation, correction
Learning	Improve system	pattern extraction, knowledge update

18.6.2. Hook Metadata

```
# Hook registry
hooks:
  - id: "validate-input"
    category: "validation"
    trigger: "before_llm"
    priority: 100 # Runs first
    enabled: true

  - id: "log-interaction"
    category: "logging"
    trigger: "after_llm"
    priority: 50
    enabled: true

  - id: "extract-learnings"
    category: "learning"
```

```
trigger: "after_llm"  
priority: 10 # Runs last  
enabled: true
```

18.6.3. Hook Testing

```
// Test hooks in isolation  
async function testHook(hook, input) {  
  const context = createContext(input);  
  
  try {  
    const result = await hook(context);  
    return { passed: true, result };  
  } catch (error) {  
    return { passed: false, error: error.message };  
  }  
}
```

18.7. Case Study: Cron-Driven Self-Improvement Digest

A concrete implementation of hook-driven continuous learning is the **AI Self-Improvement Digest** pattern, developed within the Terraphim ecosystem. Rather than relying on real-time learning hooks (which add latency to every interaction), this pattern uses scheduled automation to maintain context quality.

18.7.1. The Daily Digest Workflow

1. **Deduplication:** Read a posted-items log, skip already-processed content
2. **Scan sources:** Check tiered sources (daily, twice-weekly, weekly) for new content relevant to AI self-improvement
3. **Filter:** Include only items directly relevant to the agent's capabilities
4. **Format:** Produce 3-5 items with summary, relevance to current setup, and actionable takeaway
5. **Experiment:** Suggest one small experiment the agent can try
6. **Setup review:** Compare findings against the agent's existing configuration and suggest concrete updates
7. **Track:** Append to the posted-items log for deduplication

18.7.2. Why Scheduled Beats Real-Time

Real-time learning hooks (the pattern described in the previous section) are appropriate for capturing failures and corrections. But for broader self-improvement – discovering better patterns, new tools, updated practices – scheduled digests are more practical:

- **No latency impact:** The digest runs outside of interactive sessions
- **Curated quality:** A dedicated scan-filter-format pipeline produces higher quality learnings than extracting patterns from noisy interaction logs
- **Auditable:** Each digest is a dated artefact that can be reviewed

This pattern connects hooks (capturing what happened) with scheduled automation (reflecting on what to improve). The hooks record;

the digest analyses. Together they form a feedback loop that maintains context freshness without adding overhead to interactive work.

18.8. Summary: Automation Layer

Hooks provide the automation layer for context management:

1. **Session hooks:** Before/after LLM calls for preprocessing and postprocessing
2. **Tool use hooks:** Validate, log, and correct tool interactions
3. **Claude Code hooks:** Built-in hook points (mcp-hook, pre/post-tool-use)
4. **Context mutation discipline:** Make all hook-driven context changes visible and auditable
5. **Continuous learning:** Extract and store learnings from interactions
6. **Scheduled digests:** Cron-driven self-improvement complements real-time learning hooks
7. **Hook taxonomy:** Organize hooks by category with metadata

Hooks transform AI systems from passive responders to active, self-improving systems.

18.9. Key Sources

- Claude Code hooks documentation
- MCP (Model Context Protocol) specification
- Tool use validation patterns
- Terraphim AI Self-Improvement Digest skill pattern
- Continuous learning in AI systems

19. Chapter 17: The OpenClaw Context System

In previous chapters, we explored validation, multi-agent systems, and hooks. Now we examine a **real-world implementation** — the OpenClaw context system, which demonstrates how all these patterns work together.

OpenClaw is an AI assistant system that manages context across sessions, skills, and autonomous operations. It embodies the principles from this book.

19.1. Architecture Overview

OpenClaw’s context architecture consists of several layers:

```
workspace/
├── AGENTS.md           # Identity and operating principles
├── SOUL.md             # Core persona
├── USER.md            # User preferences and context
├── HEARTBEAT.md        # Autonomous loop instructions
├── memory/            # Session and long-term memory
├── skills/             # Modular skill definitions
├── kg/                # Knowledge graph
└── primitives/        # Task orchestration
```

19.2. Memory Architecture

OpenClaw uses a layered memory system:

```
const memory = {  
  // Constitutional - never compressed  
  constitutional: ["AGENTS.md", "SOUL.md", "USER.md"],  
  
  // Session - current conversation  
  session: [], // Last N turns  
  
  // Long-term - persistent across sessions  
  longTerm: {}, // key-value store  
  
  // Short-term - ephemeral, current task  
  shortTerm: {} // cleared each session  
};
```

19.2.1. Memory Loading Rules

```
// What loads when  
const loadRules = {  
  // Always loaded  
  always: ["AGENTS.md", "SOUL.md", "USER.md"],  
  
  // Loaded on startup  
  startup: ["HEARTBEAT.md", "skill-catalog"],  
  
  // Loaded on demand  
  onDemand: ["skills/*/SKILL.md", "memory/**"],  
};
```

```
// Never loaded in shared sessions
never: ["MEMORY.md"] // Security: personal context
};
```

19.3. Skill System

Skills in OpenClaw follow the architecture from Chapter 12:

```
skills:
  - name: github
    triggers: ["gh", "github", "pr", "issue"]
    tier: core

  - name: gog
    triggers: ["gmail", "calendar", "email"]
    tier: core

  - name: weather
    triggers: ["weather", "forecast", "temperature"]
    tier: optional
```

19.3.1. Skill Activation

```
async function activateSkills(query) {
  const triggered = skills.filter(s =>
    s.triggers.some(t => query.includes(t))
  );

  // Sort by tier priority (core first, then standard, then o
```

```
const tierOrder = { core: 0, standard: 1, optional: 2 }
triggered.sort((a, b) => (tierOrder[a.tier] ?? 2) - (tierOrder[b.tier] ?? 2))

// Max 5 active
return triggered.slice(0, 5);
}
```

19.4. HEARTBEAT.md: Autonomous Loops

HEARTBEAT.md defines autonomous behavior:

```
# Autonomous Agent Loop

On heartbeat (every 30 minutes):
1. Scan primitives/tasks for open items
2. Pick highest priority task
3. Execute if not blocked
4. Log progress
5. If blocked, ask human a crisp question
```

19.4.1. Heartbeat Implementation

```
async function heartbeat() {
  const tasks = await scanTasks();
  const actionable = tasks.filter(t => !t.blocked);

  if (actionable.length > 0) {
    const task = prioritize(actionable)[0];
    await execute(task);
  }
}
```

```

} else if (tasks.some(t => t.blocked)) {
  // Ask human the unblocking question
  await notifyHuman(getBlockerQuestion(tasks));
}
}

```

19.5. Context Security

OpenClaw separates **shared** vs **personal** context:

```

// Shared session - safe for group chats
const sharedContext = {
  workspace: true,
  skills: true,
  memory: false, // Never
  personal: false
};

// Personal session - full access
const personalContext = {
  workspace: true,
  skills: true,
  memory: true,
  personal: true
};

```

This prevents private context from leaking to shared sessions.

19.6. Summary

OpenClaw demonstrates: - Layered memory architecture - Skill-based context modularity - Autonomous heartbeat loops - Security separation for shared vs personal

Key Sources: OpenClaw documentation

20. Chapter 18: Enterprise Context Governance

This chapter addresses how to apply context engineering at **enterprise scale** — with multiple teams, projects, and stakeholders.

20.1. Gated Lifecycle

Enterprise AI projects require formal **gates** — checkpoints where context is validated before progression:

```
lifecycle:
  stages:
    - name: discovery
      gates:
        - domain_model_baseline
        - vocabulary_approved

    - name: design
      gates:
        - architecture_approved
        - knowledge_graph_baseline

    - name: development
      gates:
        - code_quality_passed
```

```
- validation_tests_passed

- name: deployment
  gates:
    - integration_passed
    - security_review_passed
```

20.1.1. Gate Validation

```
async function validateGate(stage, gate) {
  const requirements = getGateRequirements(stage, gate);

  const results = {};
  for (const req of requirements) {
    results[req.name] = await req.validate();
  }

  const passed = Object.values(results).every(r => r.passed);

  return { passed, results, recommendation: passed ? "Approved" : "Not Approved" };
}

// Gate requirements define what must be validated at each stage
function getGateRequirements(stage, gate) {
  const gateConfig = {
    discovery: {
      domain_model_baseline: [
        { name: "domainModel", validate: validateDomainModel },
        { name: "vocabulary", validate: validateVocabulary },
      ],
      vocabulary_approved: [
        { name: "vocabulary", validate: validateVocabulary },
      ],
    },
  };
  const requirements = gateConfig[gate.stage];
  return requirements.map(req => {
    name: req.name,
    validate: req.validate,
  });
}
```



```

    ],
  },
  development: {
    code_quality_passed: [
      { name: "tests", validate: validateTests },
      { name: "artifacts", validate: validateArtifacts }
    ]
  }
};

return gateConfig[stage]?.[gate] ?? [];
}

```

20.2. Change Management

Context changes must be **tracked** and **impact-analyzed**:

```

class ContextChangeManager {
  async requestChange(change) {
    // 1. Analyze impact
    const impact = await analyzeImpact(change);

    // 2. Get approvals
    const approvals = await getApprovals(impact);

    // 3. Apply if approved
    if (approvals.allPassed) {
      await applyChange(change);
      await logChange(change, approvals);
    }

    return { approved: approvals.allPassed, impact, approvals }
  }
}

```

```
}
}
```

20.3. Context Health Metrics

Track context health across projects:

```
const healthMetrics = {
  // Structural health
  vocabularyCoverage: "percentage of domain terms in KG",
  artifactReconciliation: "percentage of artifacts aligned",

  // Operational health
  contextDrift: "rate of context change per week",
  retrievalAccuracy: "percentage of KG queries returning",

  // Governance health
  gatePassRate: "percentage of gates passed first time",
  changeApprovalRate: "percentage of changes approved"
};

async function calculateHealth(project) {
  const metrics = {};

  for (const [metric, definition] of Object.entries(healthMetrics)) {
    metrics[metric] = await measure(project, definition);
  }

  return {
    overall: average(Object.values(metrics)),
    metrics,
    status: getStatus(metrics.overall)
  };
}
```

```
};  
}
```

20.4. Summary

Enterprise governance requires: - Gated lifecycle with validation at each stage - Formal change management with impact analysis - Health metrics to track context quality

Key Sources: Zestic AI framework, enterprise software governance

21. Chapter 19: Hybrid Pipeline Patterns

In Chapter 6, we established that deterministic search and vector embeddings serve different purposes. In Chapter 7, we showed how Terraphim’s graph embeddings achieve deterministic precision for vocabulary-matched terms. But real-world systems rarely face only one type of query. This chapter shows how to **combine** deterministic and probabilistic approaches into hybrid pipelines that get the best of both worlds.

The central argument is simple: no single retrieval method handles all queries well. Deterministic search excels at precision for known vocabulary. Semantic search handles novel queries and paraphrasing. Knowledge graph grounding provides factual verification. A hybrid pipeline routes each query to the right method – or combines multiple methods in sequence.

21.1. The Evolution from RAG to Context Graphs

Before diving into implementation patterns, it helps to understand where hybrid pipelines sit in the broader evolution of AI context retrieval. The progression from basic RAG to sophisticated context graphs provides the conceptual foundation.

21.1.1. The Eight-Step Progression

1. **LLMs answer from training data** – the baseline, with no external context
2. **RAG**: Stuff prompts with text chunks retrieved by semantic similarity over vector embeddings
3. **GraphRAG**: Flexible knowledge representations with rich relationships, moving beyond text chunks and semantic similarity alone
4. **OntologyRAG**: Structured ontologies for precision, improved recall, and granular retrieval
5. **Information retrieval analytics**: Specialised strategies for temporal data, anomaly detection, clustering, and domain-specific challenges
6. **Self-describing information stores**: Systems carrying meta-data about their own structure, enabling adaptive retrieval
7. **Dynamic information retrieval strategies**: LLMs deriving retrieval strategies for unseen information types
8. **Autonomous learning**: Systems reingesting their own outputs with metadata, adjusting retrieval for new and old data

Most production systems today operate at steps 2-4. Steps 5-8 represent the research frontier. The hybrid pipeline patterns in this chapter primarily address steps 3-4, with patterns that extend towards step 5.

The critical insight from this progression: each step does not replace the previous one. A mature system uses basic RAG for some queries, graph-grounded retrieval for others, and ontology-driven precision for a third category. The hybrid pipeline is the orchestration layer that routes queries to the right strategy.

21.1.2. Why Structured Formats Improve LLM Responses

A counterintuitive finding that underpins hybrid pipeline design: **structured formats like Cypher and RDF improve LLM responses despite higher token overhead**. The structure itself carries information – it encodes what is a node, what is a property, and what is a relationship. When you pass a Cypher query result to an LLM, the syntax tells the model about the data's semantics, not just its content.

This means hybrid pipelines should not always flatten graph results into plain text before passing them to the LLM. Preserving structure is often worth the extra tokens.

21.2. The Hybrid Architecture

A hybrid pipeline routes queries based on their characteristics:

```
async function hybridPipeline(query, context) {
  // Step 1: Classify the query
  const classification = classifyQuery(query, context);

  // Step 2: Route to appropriate strategy
  switch (classification.strategy) {
    case "deterministic":
      // Known vocabulary, exact match needed
      return await deterministicSearch(query);

    case "semantic":
      // Novel query, needs similarity matching
      return await semanticSearch(query);

    case "grounded":
```

```

    // Fact-checking, needs KG verification
    return await groundedSearch(query);

    case "hybrid":
    // Complex query, combine multiple methods
    return await multiMethodSearch(query, classification);
  }
}

```

21.2.1. Query Classification

The router is the most important component. A naive router uses pattern matching; a production router considers the query's relationship to the known vocabulary:

```

function classifyQuery(query, context) {
  // Check vocabulary coverage
  const vocabMatch = matchVocabulary(query, context.known);

  if (vocabMatch.coverage > 0.8) {
    // Most terms are known -- deterministic is sufficient
    return { strategy: "deterministic", confidence: 0.95 };
  }

  if (vocabMatch.coverage > 0.4) {
    // Partial match -- use deterministic for known terms
    // semantic for unknown
    return { strategy: "hybrid", confidence: 0.7 };
  }

  // Check if factual verification is needed
  if (requiresFactCheck(query)) {
    return { strategy: "grounded", confidence: 0.8 };
  }
}

```



```

}

// Novel query -- semantic search
return { strategy: "semantic", confidence: 0.5 };
}

```

The vocabulary coverage score is the key metric. When most query terms appear in the knowledge graph, deterministic search will be faster and more precise. When terms are novel, semantic search handles the flexibility. The hybrid strategy combines both.

21.3. Routing Patterns

21.3.1. Pattern 1: Cascade (Try Cheap First)

The cascade pattern tries the cheapest method first and escalates only if results are insufficient:

```

async function cascadeSearch(query) {
  // Try deterministic first (free, fast)
  const deterministicResults = await deterministicSearch(query);

  if (deterministicResults.confidence > 0.8) {
    return { results: deterministicResults, method: "deterministic" };
  }

  // Escalate to semantic (costs per query)
  const semanticResults = await semanticSearch(query);

  if (semanticResults.confidence > 0.7) {
    return { results: semanticResults, method: "semantic" };
  }
}

```

```
}

// Final escalation: ground in KG
const grounded = await kgGround(semanticResults);
return { results: grounded, method: "grounded" };
}
```

This pattern optimises for cost. Deterministic search is essentially free (precompiled automata, no API calls). Semantic search requires embedding computation. KG grounding adds verification overhead. The cascade ensures you only pay for what you need.

21.3.2. Pattern 2: Fan-Out (Search All, Merge Results)

The fan-out pattern runs multiple methods in parallel and merges results:

```
async function fanOutSearch(query) {
  // Run all methods in parallel
  const [deterministic, semantic, graphTraversal] = await Promise.all([
    deterministicSearch(query),
    semanticSearch(query),
    graphTraversal(query)
  ]);

  // Merge and deduplicate
  return mergeResults(
    { results: deterministic, weight: 0.4 },
    { results: semantic, weight: 0.35 },
    { results: graphTraversal, weight: 0.25 }
  );
}
```

```

function mergeResults(...sources) {
  const merged = new Map();

  for (const source of sources) {
    for (const result of source.results) {
      const existing = merged.get(result.id);
      if (existing) {
        existing.score += result.score * source.weight;
        existing.methods.push(source.method);
      } else {
        merged.set(result.id, {
          ...result,
          score: result.score * source.weight,
          methods: [source.method]
        });
      }
    }
  }

  return [...merged.values()].sort((a, b) => b.score - a.score);
}

```

Fan-out is more expensive but produces higher recall. Documents found by multiple methods get boosted scores, which is a strong relevance signal.

21.3.3. Pattern 3: Semantic Enrichment Pipeline

This pattern uses deterministic search to seed context, then enriches it with semantic analysis. It is particularly effective when integrating Terraphim's graph embeddings with tools that manage LLM context files.

```
def semantic_enrichment_pipeline(query, knowledge_graph):
    # Phase 1: Deterministic seed
    # Use Aho-Corasick to find all known terms in the query
    matched_terms = knowledge_graph.match_terms(query)

    # Phase 2: Graph expansion
    # Traverse the KG to find related concepts
    expanded_terms = set(matched_terms)
    for term in matched_terms:
        neighbours = knowledge_graph.get_neighbours(term)
        expanded_terms.update(neighbours)

    # Phase 3: Gap detection
    # Check if the expanded set covers the query's intent
    query_concepts = extract_concepts(query)
    missing = query_concepts - expanded_terms

    if missing:
        # Phase 4: Semantic fallback for unknown concepts
        semantic_results = semantic_search(missing)
        expanded_terms.update(semantic_results)

    # Phase 5: Retrieve and rank documents
    documents = retrieve_documents(expanded_terms)
    return rank_by_connectivity(documents, knowledge_graph)
```

This pipeline demonstrates a key principle: **use deterministic methods to establish a precise foundation, then use probabilistic methods to fill gaps.** The knowledge graph provides the structure; semantic search provides the flexibility.

21.3.4. Multi-Dimensional Context Analysis

When combining retrieval methods, it helps to categorise what each contributes:

Context Type	Source	Purpose	Cost
Direct Context	Deterministic search	Exact matches for known vocabulary	Free
Semantic Context	Vector embeddings	Novel terms, paraphrases	Per-query
Structural Context	Graph traversal	Relationships, paths, connectivity	Free
Gap Context	Semantic fallback	Missing concepts identified by graph analysis	Per-query

The hybrid pipeline selects from these context types based on query classification, then assembles the final context from multiple sources.

21.4. RDF, Property Graphs, and the Non-Argument

A practical question that arises in hybrid pipeline design: should the knowledge graph backend be a triplestore (RDF), a property graph (Neo4j/Cypher), or something else entirely?

The answer, perhaps surprisingly, is that **it does not matter much for the pipeline architecture**. The same information can be

stored as RDF triples, property graph nodes and edges, or even joined relational tables. One production system runs over a billion nodes and edges in Apache Cassandra. Another uses Neo4j. The retrieval agents built on top of these stores perform equivalently.

What matters is:

- **The ontology quality:** Well-defined classes, properties, and relationships
- **The synonym coverage:** How many ways can users refer to the same concept?
- **The relationship completeness:** Are the important connections captured?
- **The update mechanism:** How quickly can the graph reflect new knowledge?

The storage backend is an implementation detail. The hybrid pipeline operates at the abstraction layer above it.

21.5. Mature Graph Algorithms: The Untapped Toolbox

Decades of graph algorithm research remain largely untapped in AI retrieval systems. Current GraphRAG implementations make minimal use of these tools. Hybrid pipelines can leverage them for more sophisticated retrieval:

- **Graph traversal depth optimisation:** Determine the optimal traversal depth per query type, rather than using a fixed depth
- **Clustering analysis:** Group related documents by graph structure, not just content similarity
- **Density calculations:** Identify densely connected subgraphs as high-confidence knowledge domains

- **Outlier detection:** Flag entities with unusual connectivity patterns as potential errors or novel discoveries
- **Community detection:** Find tightly connected groups of concepts that form coherent topics

```
function leverageGraphAlgorithms(query, knowledgeGraph) {
  // Use PageRank to find important nodes near the query
  const seedNodes = knowledgeGraph.match(query);
  const important = pageRank(seedNodes, knowledgeGraph, { dep

  // Use community detection to find topic clusters
  const communities = detectCommunities(important);

  // Return documents from the most relevant community
  const relevantCommunity = communities
    .sort((a, b) => b.relevanceToQuery - a.relevanceToQuery)[0];

  return relevantCommunity.documents;
}
```

These algorithms are well-understood, efficient, and deterministic. They represent a significant opportunity for hybrid pipelines to improve retrieval quality without introducing probabilistic uncertainty.

21.6. Temporal Context in Hybrid Pipelines

One of the most challenging aspects of hybrid pipeline design is handling **temporal context**: information that changes over time, where freshness and accuracy are different dimensions.

A common mistake is treating recency as a proxy for accuracy. In practice:

21. Chapter 19: Hybrid Pipeline Patterns

- Old, obscure data that remains corroborated across multiple sources may be more valid than newer unconfirmed data
- “Freshness” and “recency” are not the same as “accuracy” and “precision”
- Temporal relationships establish ground truth through consistency of observation over time

Hybrid pipelines should weight temporal signals differently depending on the query type:

```
function temporalWeight(document, queryType) {
  const age = Date.now() - document.lastUpdated;
  const corroboration = document.sourceCount;

  if (queryType === "factual") {
    // For facts: corroboration matters more than recency
    return corroboration * 0.7 + recencyScore(age) * 0.3;
  }

  if (queryType === "current_state") {
    // For current state: recency dominates
    return corroboration * 0.3 + recencyScore(age) * 0.7;
  }

  // Default: balanced
  return corroboration * 0.5 + recencyScore(age) * 0.5;
}
```

21.7. Cost Optimisation

Hybrid pipelines optimise **cost** by using expensive methods only when needed:

Method	Per-Query Cost	Cold Start	Precision	Recall
Deterministic (Aho-Corasick)	~0	0	High (known terms)	Low (novel terms)
Graph Traversal	~0	0	High (connected)	Medium
Semantic Search	~\$0.01	2s	Medium	High
KG Grounding	~\$0.001	0	High	Medium
LLM Re-ranking	~\$0.05	0	High	High

The cascade pattern exploits this cost structure: start with the free methods, escalate only when confidence is low. In practice, 60-80% of queries in a well-built domain system can be handled by deterministic search alone, keeping per-query costs near zero.

```
function estimateCost(queryBatch, router) {
  let totalCost = 0;
  const breakdown = { deterministic: 0, semantic: 0, grounded: 0 };

  for (const query of queryBatch) {
    const route = router.classify(query);
    breakdown[route.strategy]++;

    if (route.strategy === "semantic") totalCost += 0.01;
    if (route.strategy === "grounded") totalCost += 0.011;
  }
}
```

```
    return { totalCost, breakdown, avgCost: totalCost / que  
  }
```

21.8. Case Study: Terraphim + ProCoder Semantic Enrichment

A concrete example of a hybrid pipeline in production: integrating Terraphim's graph embeddings with ProCoder AI Pack for automated file selection.

21.8.1. The Problem

ProCoder uses manually defined glob patterns to select which files to include in LLM context:

```
# Manual file selection -- fragile and incomplete  
context_globs = ["src/auth/*.rs", "src/models/user.rs"]  
working_globs = ["src/api/login.rs"]
```

This works for simple tasks but breaks down when: - The developer does not know which files are relevant - Synonyms and related concepts span multiple directories - New files are added but glob patterns are not updated

21.8.2. The Hybrid Solution

Replace manual globs with a Terraphim-driven semantic retrieval pipeline:

```

fn semantic_file_selection(prompt: &str, knowledge_graph: &KG
    // Step 1: Extract key terms from the prompt
    let key_terms = extract_terms(prompt);

    // Step 2: Expand via Terraphim (deterministic, <1ms)
    let expanded = terraphim_expand(&key_terms, knowledge_graph);
    // "authentication" → ["auth", "login", "credential", "s

    // Step 3: Search the indexed codebase (deterministic, <1ms)
    let matched_files = terraphim_search(&expanded, knowledge_graph);

    // Step 4: Check for missed concepts (graph traversal)
    let missed = check_concept_gaps(&matched_files, &key_terms);

    // Step 5: Retrieve additional files for gaps (semantic search)
    let gap_files = if !missed.is_empty() {
        semantic_search_files(&missed)
    } else {
        vec![]
    };

    // Combine and return
    [matched_files, gap_files].concat()
}

```

21.8.3. Comparison: Manual vs Hybrid

Aspect	Manual Globs	Hybrid Pipeline
File Selection	User-defined patterns	Semantic via graph embeddings
Synonym Handling	None (literal matches)	Automatic via KG synonym edges

Aspect	Manual Globs	Hybrid Pipeline
Related Terms	Manual addition	Graph traversal for expansion
Missed Concepts	No detection	Graph gap analysis flags missing files
Speed	Instant config	<2ms total (deterministic + graph)
Maintenance	Manual updates per task	Self-adapting via KG

The hybrid approach is not always better. For a simple, well-understood task, manual globs are faster to set up. But for complex queries that span multiple concerns, the hybrid pipeline consistently retrieves more complete context.

21.9. Summary

Hybrid pipelines combine retrieval methods based on query characteristics:

- **Deterministic** for known vocabulary: fast, precise, free, explainable
- **Graph traversal** for relationships: follows connections, no API cost
- **Semantic** for novel queries: flexible, handles paraphrasing
- **Grounding** for accuracy-critical queries: verifies against knowledge graph

Three routing patterns serve different needs: - **Cascade**: try cheap first, escalate on low confidence (optimises cost) - **Fan-out**: run

all methods in parallel, merge results (optimises recall) - **Semantic enrichment**: deterministic seed, then expand and fill gaps (optimises completeness)

The field is evolving rapidly from basic RAG through GraphRAG and OntologyRAG towards self-describing stores and autonomous learning. The hybrid pipeline is the orchestration layer that makes this evolution practical – routing each query to the retrieval strategy that serves it best.

Key Sources: Terraphim hybrid architecture, TrustGraph context graph research, ProCoder AI Pack integration patterns, RAG optimisation literature

22. Chapter 20: Building a Context Engineering Practice

This chapter addresses how to **build a practice** — the people, processes, and culture to make context engineering work at scale.

22.1. Roles and Responsibilities

A context engineering practice needs defined roles:

roles:

- name: Context Engineer
responsibility: Building and maintaining knowledge graphs
skills: [ontology, graph databases, semantic modeling]
- name: Prompt Engineer
responsibility: Designing and optimizing prompts
skills: [LLM mechanics, testing, iteration]
- name: Validation Engineer
responsibility: Building validation gates
skills: [testing, monitoring, drift detection]
- name: Governance Lead
responsibility: Ensuring context quality
skills: [process design, auditing, compliance]

22.1.1. Role Interactions

```
const roleInteractions = {
  "Context Engineer": {
    collaboratesWith: ["Prompt Engineer", "Validation Engineer"],
    reportsTo: "Governance Lead",
    owns: ["knowledge-graph", "vocabulary", "thesaurus"]
  },
  "Prompt Engineer": {
    collaboratesWith: ["Context Engineer", "Validation Engineer"],
    reportsTo: "Engineering Lead",
    owns: ["prompts", "skill-definitions"]
  },
  // ...
};
```

22.2. Skill Development

Train teams on context engineering:

```
const trainingCurriculum = {
  foundation: [
    "Context Engineering fundamentals",
    "Knowledge graph basics",
    "Token budget management"
  ],
  intermediate: [
    "Building knowledge graphs",
  ]
};
```



```

    "Validation gate design",
    "Drift detection"
  ],

  advanced: [
    "Multi-agent orchestration",
    "Enterprise governance",
    "Custom tool development"
  ]
};

```

22.3. Measuring Success

Track practice effectiveness:

```

const practiceMetrics = {
  // Efficiency
  contextRetrievalTime: "ms to retrieve relevant context",
  tokenUsage: "average tokens per query",

  // Quality
  validationPassRate: "percentage passing gates first time",
  hallucinationRate: "grounded vs ungrounded outputs",

  // Velocity
  featureDeliveryTime: "time from spec to deployed",
  iterationCount: "prompt/skill improvement cycles"
};

async function reportPracticeHealth() {
  const metrics = await measureMetrics(practiceMetrics);
  const score = calculateScore(metrics);
}

```

```
return {  
  score,  
  metrics,  
  recommendations: getRecommendations(score, metrics)  
};  
}
```

22.4. Summary

Building a practice requires: - Defined roles with clear ownership
- Training curriculum for skill development - Metrics to measure success

Key Sources: Zestic AI framework, engineering practice building

23. Chapter 21: The Knowledge Graph Market Landscape

The previous chapters presented knowledge graphs as engineering infrastructure. This chapter steps back to survey the **market landscape** – what solutions exist, where the industry is heading, and how to evaluate options for your own context engineering practice.

The knowledge graph market in 2025-2026 is at an inflection point. After years of niche enterprise adoption, the convergence of LLMs with structured knowledge is driving mainstream interest. Understanding this landscape helps you make informed technology choices.

23.1. Categories of Knowledge Graph Solutions

The market divides into four broad categories, each serving different needs:

23.1.1. 1. Graph Databases (General Purpose)

These are the established platforms for storing and querying graph-structured data.

Platform	Model	Query Language	Deployment
Neo4j	Property graph	Cypher	Self-hosted, AuraDB (cloud)
Amazon Neptune	Property graph + RDF	Gremlin, SPARQL, openCypher	AWS managed
Azure Cosmos DB (Gremlin API)	Property graph	Gremlin	Azure managed
TigerGraph	Property graph	GSQL	Self-hosted, cloud
Stardog	RDF/OWL	SPARQL	Enterprise

Neo4j remains the most widely adopted graph database, with a mature ecosystem and strong community. Its Cypher query language has become the basis for the ISO GQL standard. For teams already on AWS, Neptune offers a managed alternative with both property graph and RDF support.

These platforms provide the storage layer but do not, by themselves, solve the AI context problem. They store and query graphs efficiently – the context engineering pipeline is built on top.

23.1.2. 2. LLM-Powered Graph Construction (GraphRAG)

A newer category uses LLMs to automatically extract entities and relationships from unstructured text, building knowledge graphs without manual curation.

Microsoft GraphRAG is the most prominent example. Its workflow:

1. **Indexing:** LLMs extract entities and relationships from a document corpus
2. **Community detection:** Graph algorithms identify clusters of related entities
3. **Summarisation:** LLMs generate summaries at multiple levels of the hierarchy
4. **Query:** Questions are answered using graph-structured context rather than raw document chunks

```
# Simplified GraphRAG extraction pipeline
def extract_graph(documents):
    entities = []
    relationships = []
    for doc in documents:
        # LLM extracts structured knowledge from text
        extraction = llm_extract(doc, schema=entity_relations)
        entities.extend(extraction.entities)
        relationships.extend(extraction.relationships)

    # Build graph and detect communities
    graph = build_graph(entities, relationships)
    communities = detect_communities(graph)
    return graph, communities
```

Strengths: Handles large unstructured corpora; automated extraction reduces manual curation effort; community detection provides multi-scale summarisation.

Limitations: Extraction quality depends on the LLM – errors propagate into the graph; no deterministic guarantees; extraction costs scale with corpus size; the graph reflects the LLM’s interpretation, not ground truth.

GraphRAG is best suited for exploratory analysis of large document collections where manual curation is impractical. It is less suitable

for safety-critical applications where extraction errors carry real consequences.

23.1.3. 3. Context Graph Platforms

A emerging category optimises graph-structured data specifically for AI consumption, rather than general-purpose graph queries.

TrustGraph represents this approach. Built on Apache Cassandra as its default triplestore, TrustGraph focuses on the pipeline from raw data to AI-ready context:

- Stores triples (subject-predicate-object) optimised for AI retrieval
- Supports RDF natively, with translation to property graph formats (Neo4j) available
- One production deployment runs over a billion nodes and edges in Cassandra (TrustGraph, 2025)
- Retrieval agents built on top of the store perform equivalently regardless of the storage backend

The context graph approach differs from general-purpose graph databases in its design priorities: rather than optimising for ad-hoc queries, it optimises for feeding structured context into LLM prompts. This includes preserving structured formats (Cypher, RDF) in the context, because the structure itself carries semantic information that improves LLM responses.

23.1.4. 4. Deterministic Knowledge Retrieval

This category uses knowledge graphs not for storage or extraction but for **retrieval and validation** – matching queries against a curated vocabulary with deterministic precision.

Terraphim operates in this space. As described in Chapters 7 and 8, its approach:

- Compiles curated vocabularies into Aho-Corasick automata
- Matches query terms against the vocabulary in <1ms
- Scores results by graph connectivity
- Provides complete explainability for every result

The tradeoff is explicit: Terraphim requires upfront vocabulary curation but delivers deterministic, explainable, zero-cost-per-query retrieval. This makes it complementary to, rather than competitive with, the other categories.

23.2. Market Trends

23.2.1. Trend 1: RAG Fatigue

By 2025, enterprises that adopted basic RAG (Retrieval-Augmented Generation) are encountering its limitations at scale:

- **Precision problems:** “Semantically similar” does not mean “relevant.” Vector similarity retrieves documents that mention related topics but may not answer the actual question.
- **No explainability:** When a RAG system returns poor results, there is no audit trail explaining *why* those documents were retrieved.
- **No validation:** The LLM can ignore retrieved documents entirely, or hallucinate despite having correct context.
- **Cost at scale:** Embedding computation and vector database queries add per-query costs that compound at enterprise scale.

This “RAG fatigue” is driving interest in structured approaches – knowledge graphs, ontologies, and hybrid pipelines – that offer the precision and explainability that pure vector RAG lacks. The progression from RAG to GraphRAG to OntologyRAG described in Chapter 19 reflects this market movement.

23.2.2. Trend 2: Regulation Driving Structured Knowledge

Regulatory frameworks are creating demand for explainable AI systems:

- **EU AI Act:** Requires transparency and explainability for high-risk AI applications. Knowledge graphs provide the audit trail that black-box retrieval cannot.
- **FDA guidance on AI/ML:** Medical AI must demonstrate the reasoning behind recommendations. Graph-grounded systems can trace every output to its source knowledge.
- **Financial regulations:** Trading and compliance AI must explain its outputs to regulators. Deterministic retrieval with full audit trails meets this requirement.

Regulation is shifting the market from “what gives the best results” to “what gives explainable results.” Knowledge graphs are well-positioned for this shift because explainability is inherent in their structure – every answer can be traced through the graph.

23.2.3. Trend 3: Hybrid Architectures

The market is converging on hybrid architectures that combine multiple retrieval methods:

- **Deterministic search** for known vocabulary: fast, precise, free

- **Graph traversal** for relationship queries: follows connections, no API cost
- **Semantic search** for novel queries: flexible, handles paraphrasing
- **LLM-based extraction** for building graphs from unstructured data

No single approach handles all use cases. The winners will be platforms and architectures that integrate multiple methods, routing each query to the most appropriate strategy. Chapter 19 describes these hybrid pipeline patterns in detail.

23.2.4. Trend 4: The Semantic Web, Revisited

Perhaps the most unexpected trend is the revival of semantic web concepts. RDF, OWL, SKOS, and other standards from the 2000s-era semantic web movement are finding new relevance in the AI era. LLMs can read and understand machine-readable formats like RDF and Cypher, potentially realising the interoperability vision that the semantic web promised but could not deliver with pre-LLM technology.

MCP (Model Context Protocol) and A2A (Agent-to-Agent) protocols pursue the same interoperability goal as the semantic web – enabling systems to share structured knowledge across boundaries. Whether they succeed where the semantic web struggled remains an open question.

23.3. Evaluation Framework

When selecting knowledge graph technology for context engineering, consider these dimensions:

Dimension	Questions to Ask
Curation model	Manual vocabulary? Automated extraction? Hybrid?
Query determinism	Same query always returns same results?
Explainability	Can you trace why each result was returned?
Update latency	How quickly does the graph reflect new knowledge?
Scale	Thousands of entities? Millions? Billions?
Integration	Does it plug into your existing LLM pipeline?
Cost model	Per-query costs? Fixed infrastructure?
Regulatory fit	Does it satisfy your industry's explainability requirements?

The “right” choice depends on your specific constraints. A startup building a chatbot has different needs from a hospital deploying a diagnostic assistant. The evaluation framework above helps you ask the right questions for your context.

23.4. Summary

The knowledge graph market is at an inflection point driven by three forces: RAG fatigue pushing enterprises toward structured approaches, regulation requiring explainability, and hybrid architectures combining the strengths of multiple retrieval methods.

Four categories of solutions serve different needs:

1. **Graph databases** (Neo4j, Neptune): General-purpose graph storage and querying
2. **LLM-powered construction** (GraphRAG): Automated graph building from unstructured text
3. **Context graph platforms** (TrustGraph): Graph-structured data optimised for AI consumption
4. **Deterministic retrieval** (Terraphim): Curated vocabulary with precision, speed, and explainability

The trend is toward hybrid architectures that combine these approaches. The rest of this book provides the engineering patterns to build those architectures.

Key Sources: Microsoft GraphRAG documentation, Neo4j ecosystem documentation, TrustGraph context graph research (Keith Strier, 2025), EU AI Act requirements, Terraphim AI technical documentation

24. Chapter 22: Open Problems and Research Directions

This final chapter identifies **open problems** – the unsolved challenges that researchers and practitioners should address. Context engineering is a young discipline. The previous 21 chapters provide a foundation; what follows are the questions we cannot yet answer.

24.1. The Verification Bottleneck

The most fundamental open problem in AI software development is not generation – it is **verification**. As AI generation becomes exponentially cheaper, the bottleneck shifts from producing code and content to confirming that it is correct.

24.1.1. The Missing Junior Loop

Historically, junior engineers served as a training pipeline: they did measurable work (write code, run tests, fix bugs), seniors reviewed their output, and through that review cycle, juniors developed the judgement to become seniors. AI agents are now filling the junior role – generating code at scale, running tests, fixing linting errors.

But the review cycle that trained seniors has no equivalent. When a junior engineer writes code, a senior can review it and provide

feedback. When an AI generates 1,000 lines of code in an hour, no human can review it at that pace. The “junior loop” that created expertise is breaking.

This creates a structural problem: who trains the next generation of senior engineers if the junior work is automated? And who verifies the AI’s output if the senior pipeline thins?

24.1.2. The Codifier’s Curse

When you codify expertise into AI systems, you lose the tacit knowledge that was never explicitly documented. The system appears to work until it encounters edge cases where that tacit knowledge mattered. This is the “Codifier’s Curse” – the very act of automating expertise can destroy the conditions that created it.

For context engineering specifically: the knowledge graphs, vocabularies, and validation rules described in this book all represent codified expertise. If the engineers who built them leave, and the junior pipeline that would train replacements is broken, the system becomes a black box – working, but unmaintainable.

24.1.3. The Economics of Verification

The core economic insight: verification costs grow faster than generation costs. Consider the progression:

1. AI generates 1,000 lines of code in an hour
2. Human review requires days of careful analysis
3. Pressure to ship leads to superficial verification
4. Subtle bugs accumulate as “hidden technical debt”

This is the “AI sandwich” firm structure: AI does the measurable generation, humans do the unmeasurable verification, AI does the final polish. But the middle layer does not scale.

At some point, it becomes economically rational to deploy unverified AI outputs. The result is accumulating “hidden systemic risk” – failures that are statistically invisible until they are catastrophic.

24.1.4. Measurability Bias

The shift from “skill-biased” to “measurability-biased” technological progress is a crucial reframing. Tasks that are easy to verify get automated first – not because they are simple, but because we can confirm the AI did them correctly. This creates a dangerous selection bias: we automate precisely the things we think we understand, while the unmeasurable (but critical) work remains neglected.

For context engineering, this means: the parts of context management that are easy to measure (token counts, retrieval accuracy, response latency) get optimised, while the parts that are hard to measure (conceptual coherence, domain alignment, long-term maintainability) get ignored.

24.1.5. Goal-Driven Execution as Partial Solution

One approach to scaling verification: build it into the generation loop. Rather than generating first and verifying later, define verifiable success criteria before generation begins:

- “Add validation” becomes “Write tests for invalid inputs, then make them pass”
- “Fix the bug” becomes “Write a test that reproduces it, then make it pass”

- “Refactor X” becomes “Ensure tests pass before and after”

This transforms open-ended tasks into closed-loop verification cycles. It does not solve the fundamental problem – someone must still define the success criteria, and that requires expertise – but it reduces the review surface area.

24.2. LLM as a Judge: Automated Verification at Scale

One promising direction for addressing the verification bottleneck is using LLMs themselves as judges – automated evaluation systems that assess AI-generated outputs against quality criteria.

24.2.1. The Approach

LLM-as-Judge evaluation uses a separate LLM (distinct from the generator) to score outputs against defined rubrics. Four evaluation modes exist:

Mode	Description	Best For
Pointwise	Score a single response in isolation	Quality assessment, moderation
Reference-based	Compare to a known good answer	Accuracy, regression testing
Pairwise	Pick the better of two responses	A/B testing, preference data
Listwise	Rank multiple responses	Leaderboards, best-of-N selection

24.2.2. Known Limitations

LLM judges introduce their own biases:

- **Position bias:** LLMs tend to favour the first response shown (10-20% swing in pairwise results)
- **Verbosity bias:** Longer responses score higher regardless of quality
- **Self-enhancement bias:** Models prefer outputs matching their own style (GPT rates GPT higher, Claude rates Claude higher)
- **Authority bias:** Confident, assertive responses score higher, even when hedging is appropriate
- **Format bias:** Well-formatted responses (markdown, bullets) score higher than identical content in plain text

24.2.3. Metacognitive Self-Correction

Research from FINAL-Bench (HuggingFace, 2025) reveals a counter-intuitive finding about AI self-evaluation: **94.8% of performance gain comes from error recovery (acting on detected errors), not metacognitive accuracy (correctly identifying uncertainty levels).**

This means it matters more that agents *act* on detected errors than that they *correctly identify* their uncertainty level. For verification systems, this suggests biasing towards correction attempts even when confidence in the detection is moderate. A false-positive correction attempt is cheaper than a missed error.

24.2.4. The Hybrid Verification Pipeline

The recommended approach combines multiple verification methods:

1. **Deterministic checks first:** Validate format, structure, computable criteria (syntax, type checking, test execution)
2. **LLM-as-Judge for nuance:** Assess subjective qualities like helpfulness, completeness, and clarity
3. **Ensemble across models:** Use multiple judge LLMs to reduce single-model bias
4. **Human review for calibration:** Sample 5-10% for expert evaluation, calibrate the automated judges against human judgement
5. **Escalation for edge cases:** Flag low-confidence or borderline scores for human review

This is context engineering applied to verification itself – structuring the evaluation context to maximise the value of each verification method.

24.3. Cross-Context Coherence

Challenge: Maintaining semantic alignment across sessions, agents, and time.

When multiple AI agents work on the same project, each operates within its own context window. Agent A may establish a convention in session 1 that Agent B contradicts in session 2. Over time, the accumulated context across sessions becomes incoherent.

Current approaches (shared CLAUDE.md files, handover documents, memory systems) are ad-hoc. No formal theory exists for

how to compose contexts from multiple sources while maintaining coherence. Key questions:

- How do you detect when two agent sessions have established contradictory conventions?
- What is the “merge conflict” equivalent for context, and how do you resolve it?
- Can context coherence be measured, or is it inherently subjective?

24.4. Real-Time Context Updates

Challenge: Hot-reloading knowledge graphs without breaking coherence.

When a knowledge graph is updated while agents are actively using it, several problems arise:

- Existing context may reference entities or relationships that have been modified
- Cached query results become stale
- In-flight operations may see an inconsistent graph state

This is analogous to database consistency under concurrent writes, but with an additional complication: the “clients” (LLMs) have long-lived context windows that cannot be atomically updated.

24.5. Privacy-Preserving Context

Challenge: Using personal or sensitive context without exposing it.

AI systems that leverage personal context (user preferences, organisational knowledge, proprietary data) face a tension: more context improves response quality, but exposing that context creates privacy and security risks.

Open questions include:

- Can context be injected into prompts in a way that prevents the LLM from reproducing it verbatim?
- How do you audit what context an LLM “learned” from a conversation vs what it already knew?
- Can federated or differential privacy techniques be applied to context engineering?

24.6. Automated Vocabulary Building

Manually building domain vocabularies does not scale. The vocabularies described in Chapter 8 require domain experts to define terms, synonyms, and relationships. For large or rapidly evolving domains, this bottleneck limits adoption.

Research directions:

- **LLM-assisted extraction:** Use LLMs to extract candidate vocabulary from domain documents, with human validation of the results
- **Incremental learning:** Start with a seed vocabulary and expand it as the system encounters new terms in use
- **Cross-domain transfer:** Adapt vocabularies from related domains rather than building from scratch

The challenge is quality: an automatically built vocabulary with 90% accuracy may be worse than no vocabulary at all, because the 10% of errors create false confidence.

24.7. Context Composability

Challenge: Combining contexts from multiple sources into a coherent whole.

A practical AI system needs to compose: - User-specific context (preferences, history) - Project-specific context (domain model, conventions) - Organisation-specific context (policies, standards) - Tool-specific context (API documentation, configuration)

Currently, this composition is ad-hoc: contexts are concatenated, hoping the LLM will sort out conflicts. A formal composition algebra would define:

- **Precedence rules:** When contexts conflict, which wins?
- **Scope boundaries:** Which context applies to which queries?
- **Inheritance:** Can child contexts override parent contexts?

24.8. Context Benchmarking

We need **standard benchmarks** for context quality. Current evaluation is subjective (“the AI seems to give better answers with this context”). Rigorous benchmarks would measure:

Metric	Definition	Current State
Retrieval accuracy	Percentage of relevant documents retrieved	Well-measured for RAG
Grounding rate	Percentage of claims verified against KG	Emerging

Metric	Definition	Current State
Drift detection speed	Time to detect meaningful context drift	No standard
Context efficiency	Quality improvement per token of context	No standard
Composability	Quality after merging multiple contexts	No standard

Without benchmarks, we cannot compare approaches rigorously. This holds back the entire field.

24.9. The Frontier: Towards Autonomous Learning

The ultimate goal of context engineering is systems that improve their own context over time – autonomous learning systems that reingest their outputs with metadata, adjusting retrieval strategies for new and old data.

This frontier involves:

- **Self-describing information stores:** Systems carrying meta-data about their own structure, enabling adaptive retrieval without human configuration
- **Dynamic retrieval strategies:** LLMs deriving retrieval strategies for unseen information types, rather than relying on pre-configured pipelines

- **Temporal reasoning:** Weighing repeated observations over decades against recent unconfirmed data, distinguishing freshness from accuracy

We are genuinely scratching the surface of what graph tooling can do. Decades of graph algorithms – traversal optimisation, clustering, density calculation, outlier detection, community detection – remain largely unused in AI retrieval. The hybrid pipeline patterns in Chapter 19 begin to leverage this toolbox, but the majority of its potential is unexplored.

24.10. Summary

The field is young. The foundational problems are:

1. **Verification bottleneck:** Generation scales, verification does not. Who checks the AI's work?
2. **Cross-context coherence:** Multiple agents, sessions, and time – how to keep context consistent?
3. **Real-time updates:** Changing knowledge graphs without breaking in-flight operations
4. **Privacy in context:** Using personal data without exposing it
5. **Automated vocabulary:** Building domain vocabularies without requiring full-time experts
6. **Context composability:** Merging multiple context sources with formal rules
7. **Benchmarking:** Standard metrics for comparing context engineering approaches
8. **Autonomous learning:** Systems that improve their own retrieval over time

This book provides a foundation. The rest is for you to build.

24. *Chapter 22: Open Problems and Research Directions*

Key Sources: “Some Simple Economics of AGI” (verification bottleneck), Karpathy LLM coding guidelines, FINAL-Bench metacognitive research, TrustGraph context graph progression, LLM-as-Judge best practices literature

25. Appendix A: Context Optimizer Skill (SKILL.md)

This appendix provides a reference implementation for a **context optimizer skill** – a self-contained module that manages token budgets, skill activation, and compression within an AI agent system.

The skill follows the architecture described in Chapter 12 and the token budget principles from Chapter 10.

25.1. SKILL.md Format

Every skill in a context-engineered system is defined by a `SKILL.md` file. This is both human-readable documentation and machine-parseable configuration.

```
# Context Optimizer

## Identity
- Name: context-optimizer
- Tier: core
- Version: 1.0.0

## Triggers
- context
- tokens
```

- budget
- compress
- optimize

Description

Manages token budget allocation, skill activation/deactivation and context compression. Runs automatically when context exceeds warning thresholds.

Dependencies

- None (this skill is self-contained)

Configuration

- max_active_skills: 5
- token_warning_threshold: 12000
- token_critical_threshold: 15000
- auto_deactivate_after: 10 # turns

Tools Required

- File read (for loading skill content)
- Token counter (for budget tracking)

25.2. Reference Implementation

25.2.1. Skill Loader

The skill loader reads SKILL.md files and parses them into a structured format:

```
function parseSkillMd(content) {  
  const sections = {};  
  let currentSection = null;
```

```

for (const line of content.split("\n")) {
  if (line.startsWith("## ")) {
    currentSection = line.slice(3).trim().toLowerCase();
    sections[currentSection] = [];
  } else if (currentSection && line.trim()) {
    sections[currentSection].push(line.trim());
  }
}

return {
  name: extractField(sections, "identity", "Name"),
  tier: extractField(sections, "identity", "Tier"),
  version: extractField(sections, "identity", "Version"),
  triggers: extractList(sections, "triggers"),
  description: sections["description"]?.join(" ") ?? "",
  dependencies: extractList(sections, "dependencies"),
  config: extractKeyValues(sections, "configuration")
};
}

function extractField(sections, section, field) {
  const line = sections[section]?.find(l => l.startsWith(`${field}-`));
  return line?.split(":").slice(1).join(":").trim() ?? null;
}

function extractList(sections, section) {
  return (sections[section] ?? [])
    .filter(l => l.startsWith("- "))
    .map(l => l.slice(2).trim());
}

function extractKeyValues(sections, section) {
  const config = {};
  for (const line of sections[section] ?? []) {

```

```

    if (line.startsWith("- ") && line.includes(":")) {
      const [key, ...rest] = line.slice(2).split(":");
      const value = rest.join(":").trim();
      config[key.trim()] = isNaN(value) ? value : Number(
    }
  }
  return config;
}

```

25.2.2. Context Optimizer Core

The optimizer manages the full lifecycle: activation, budget tracking, compression, and deactivation.

```

class ContextOptimizer {
  constructor(skillCatalog, config = {}) {
    this.catalog = skillCatalog;
    this.activeSkills = new Map();
    this.turnCounter = 0;

    this.maxActive = config.max_active_skills ?? 5;
    this.warnAt = config.token_warning_threshold ?? 12000;
    this.criticalAt = config.token_critical_threshold ?? 12000;
    this.deactivateAfter = config.auto_deactivate_after ?? 12000;
  }

  // Main entry point: called each turn
  async optimizeContext(userQuery, currentContext) {
    this.turnCounter++;

    // 1. Determine which skills should be active
    const candidates = this.findCandidates(userQuery);
  }
}

```

```

// 2. Activate new skills (up to limit)
for (const candidate of candidates) {
  if (!this.activeSkills.has(candidate.name)) {
    await this.activate(candidate);
  } else {
    this.activeSkills.get(candidate.name).lastUsedTurn = ...
  }
}

// 3. Deactivate stale skills
this.deactivateStale();

// 4. Enforce limit
this.enforceLimits();

// 5. Check budget
const budget = this.calculateBudget(currentContext);

if (budget.total > this.criticalAt) {
  this.compress(currentContext);
}

return {
  activeSkills: [...this.activeSkills.values()],
  budget,
  compressed: budget.total > this.warnAt
};
}

findCandidates(query) {
  const q = query.toLowerCase();
  return this.catalog
    .filter(skill => skill.triggers.some(t => q.includes(t)))
    .sort((a, b) => {

```

```

        const tierOrder = { core: 0, standard: 1, optional: 2 };
        return (tierOrder[a.tier] ?? 2) - (tierOrder[b.tier] ?? 2);
    });
}

async activate(skill) {
    const content = await loadSkillContent(skill.name);
    this.activeSkills.set(skill.name, {
        ...skill,
        content,
        tokens: estimateTokens(content),
        activatedTurn: this.turnCounter,
        lastUsedTurn: this.turnCounter
    });
}

deactivateStale() {
    for (const [name, skill] of this.activeSkills) {
        if (this.turnCounter - skill.lastUsedTurn > this.deactivationLimit) {
            this.activeSkills.delete(name);
        }
    }
}

enforceLimits() {
    if (this.activeSkills.size >= this.maxActive) return;

    const sorted = [...this.activeSkills.entries()]
        .sort(([a], [b]) => {
            const tierOrder = { core: 0, standard: 1, optional: 2 };
            const tierDiff = (tierOrder[a.tier] ?? 2) - (tierOrder[b.tier] ?? 2);
            if (tierDiff !== 0) return tierDiff;
            return b.lastUsedTurn - a.lastUsedTurn;
        });
}

```

```

    // Keep top N, remove the rest
    const toRemove = sorted.slice(this.maxActive);
    for (const [name] of toRemove) {
        this.activeSkills.delete(name);
    }
}

calculateBudget(context) {
    const constitutional = context.constitutionalTokens ?? 20;
    const catalog = estimateTokens(JSON.stringify(this.catalog));
    const skills = [...this.activeSkills.values()]
        .reduce((sum, s) => sum + s.tokens, 0);
    const memory = context.memoryTokens ?? 0;
    const working = context.workingTokens ?? 0;

    return {
        constitutional,
        catalog,
        skills,
        memory,
        working,
        total: constitutional + catalog + skills + memory + working;
    };
}

compress(context) {
    // Priority: drop optional skills first, then summarise mandatory
    const optional = [...this.activeSkills.entries()]
        .filter(([_, s]) => s.tier === "optional");

    for (const [name] of optional) {
        this.activeSkills.delete(name);
    }
}

```

```
}
```

25.2.3. Usage Example

```
// Load catalog at startup (~200-300 tokens)
const catalog = await loadSkillCatalog("skills/catalog.yaml");

// Create optimizer
const optimizer = new ContextOptimizer(catalog);

// Each turn
const result = await optimizer.optimizeContext(
  userQuery,
  { constitutionalTokens: 2000, memoryTokens: 1500, workingMemoryTokens: 1000 }
);

console.log(`Active: ${result.activeSkills.map(s => s.name).join(", ")}
console.log(`Budget: ${result.budget.total} tokens`);
```

25.3. File Structure

A complete context optimizer skill lives in a single directory:

```
skills/
  context-optimizer/
    SKILL.md           # Skill definition (shown above)
    optimizer.js       # Core implementation
    tests/
      optimizer.test.js # Unit tests
```


25.4. Key Sources

- Chapter 10: Token Budget Architecture (budget allocation)
- Chapter 11: Compression Strategies (compression priority order)
- Chapter 12: Skill Architecture for AI Agents (catalog, activation, deactivation)
- OpenClaw context management (production patterns)

26. Appendix B: Token Tracker Implementation

This appendix provides a reference implementation for a **token tracking system** that monitors context composition, enforces budgets, and triggers compression when thresholds are exceeded.

The design follows the token budget architecture described in Chapter 10.

26.1. Token Estimation

Before tracking tokens, you need to estimate them. Exact tokenisation requires model-specific tokenisers, but a practical approximation works for budget management:

```
// Approximate token estimation
// English text averages ~4 characters per token
// Code averages ~3.5 characters per token
// YAML/JSON averages ~3 characters per token
function estimateTokens(text, type = "text") {
  const charsPerToken = {
    text: 4,
    code: 3.5,
    yaml: 3,
    json: 3
  }
```

```
};

const ratio = charsPerToken[type] ?? 4;
return Math.ceil(text.length / ratio);
}

// For precise counting, use a tokeniser library
// import { encode } from "tiktoken";
// function countTokens(text) { return encode(text).length;
```

26.2. Token Tracker

The tracker maintains a running count of tokens by category and raises alerts when thresholds are crossed.

```
class TokenTracker {
  constructor(config = {}) {
    this.budget = {
      constitutional: 0,
      skillCatalog: 0,
      activeSkills: 0,
      memory: 0,
      working: 0,
      buffer: 0
    };

    this.limits = {
      warn: config.warnAt ?? 12000,
      critical: config.criticalAt ?? 15000,
      hard: config.hardLimit ?? 20000
    };
  }
}
```

```

    this.history = [];
    this.alerts = [];
}

// Update a category
set(category, tokens) {
    if (!(category in this.budget)) {
        throw new Error(`Unknown budget category: ${category}`);
    }
    this.budget[category] = tokens;
    this.check();
}

// Increment a category
add(category, tokens) {
    this.set(category, this.budget[category] + tokens);
}

// Get total usage
get total() {
    return Object.values(this.budget).reduce((a, b) => a + b,
}

// Get remaining budget
get remaining() {
    return this.limits.warn - this.total;
}

// Get usage as percentage of warning threshold
get usagePercent() {
    return Math.round((this.total / this.limits.warn) * 100);
}

// Check thresholds and raise alerts

```

```

check() {
  const total = this.total;

  if (total > this.limits.hard) {
    this.alert("HARD_LIMIT", `Token count ${total} exceeds
    return "hard_limit";
  }

  if (total > this.limits.critical) {
    this.alert("CRITICAL", `Token count ${total} exceeds
    return "critical";
  }

  if (total > this.limits.warn) {
    this.alert("WARNING", `Token count ${total} exceeds
    return "warning";
  }

  return "ok";
}

alert(level, message) {
  const entry = {
    level,
    message,
    total: this.total,
    breakdown: { ...this.budget },
    timestamp: Date.now()
  };

  this.alerts.push(entry);
}

// Record a snapshot for history

```

```

snapshot() {
  this.history.push({
    total: this.total,
    breakdown: { ...this.budget },
    timestamp: Date.now()
  });
}

// Get a formatted breakdown
report() {
  const lines = [];
  lines.push(`Token Budget Report (${this.usagePercent}% of
  lines.push("-".repeat(50));

  for (const [category, tokens] of Object.entries(this.budget)) {
    const bar = "█".repeat(Math.round(tokens / 200));
    const pct = Math.round((tokens / this.total) * 100);
    lines.push(`  ${category.padEnd(16)} ${String(tokens).padEnd(10)} ${pct}%`);
  }

  lines.push("-".repeat(50));
  lines.push(`  ${"TOTAL".padEnd(16)} ${String(this.total).padEnd(10)} 100%`);
  lines.push(`  ${"Remaining".padEnd(16)} ${String(this.remaining).padEnd(10)} 0%`);

  return lines.join("\n");
}
}

```

26.3. Budget Allocation Presets

Pre-defined allocations for common scenarios:

```
const BUDGET_PRESETS = {  
  // Standard: balanced allocation for general use  
  standard: {  
    constitutional: 2000,  
    skillCatalog: 100,  
    activeSkills: 5000,  
    memory: 2000,  
    working: 2000,  
    buffer: 900  
  },  
  
  // Skill-heavy: more room for active skills  
  skillHeavy: {  
    constitutional: 2000,  
    skillCatalog: 200,  
    activeSkills: 7000,  
    memory: 1500,  
    working: 1500,  
    buffer: 800  
  },  
  
  // Memory-heavy: more room for conversation history  
  memoryHeavy: {  
    constitutional: 2000,  
    skillCatalog: 100,  
    activeSkills: 3000,  
    memory: 4000,  
    working: 2000,  
    buffer: 900  
  },  
  
  // Minimal: tight budget for smaller models  
  minimal: {  
    constitutional: 1500,
```



```

    skillCatalog: 100,
    activeSkills: 3000,
    memory: 1000,
    working: 1000,
    buffer: 400
  }
};

function createTrackerWithPreset(preset) {
  const alloc = BUDGET_PRESETS[preset];
  if (!alloc) throw new Error(`Unknown preset: ${preset}`);

  const tracker = new TokenTracker({
    warnAt: Object.values(alloc).reduce((a, b) => a + b, 0),
    criticalAt: Math.round(Object.values(alloc).reduce((a, b) => a + b, 0) / 2),
    hardLimit: Math.round(Object.values(alloc).reduce((a, b) => a + b, 0) / 4)
  });

  return tracker;
}

```

26.4. Compression Trigger

When the tracker raises an alert, the compression pipeline activates:

```

function onBudgetAlert(tracker, context) {
  const status = tracker.check();

  if (status === "warning") {
    // Soft compression: summarise old memory, drop optional
    context.memory = summariseOldTurns(context.memory, keepLa

```

```

    tracker.set("memory", estimateTokens(context.memory))
  }

  if (status === "critical") {
    // Hard compression: drop all non-core skills, aggregate
    context.activeSkills = context.activeSkills.filter(s => s.isCore);
    context.memory = summariseOldTurns(context.memory, key);

    tracker.set("activeSkills", context.activeSkills.reduce(
      (sum, s) => sum + estimateTokens(s.content), 0
    ));
    tracker.set("memory", estimateTokens(context.memory))
  }

  if (status === "hard_limit") {
    // Emergency: constitutional only
    context.activeSkills = [];
    context.memory = extractKeyFacts(context.memory);
    context.working = truncateToEssentials(context.working);

    tracker.set("activeSkills", 0);
    tracker.set("memory", estimateTokens(context.memory));
    tracker.set("working", estimateTokens(context.working));
  }
}

```

26.5. Usage Example

```

const tracker = createTrackerWithPreset("standard");

// Set constitutional (never changes)

```

```

tracker.set("constitutional", 2000);
tracker.set("skillCatalog", 250);

// As skills activate
tracker.set("activeSkills", 3500);

// As conversation grows
tracker.set("memory", 1800);
tracker.set("working", 1200);

// Check status
console.log(tracker.report());
// Token Budget Report (73% of warning threshold)
// -----
//   constitutional    2000 (23%) ██████████
//   skillCatalog      250 ( 3%) █
//   activeSkills      3500 (40%) ████████████████████
//   memory            1800 (21%) ██████████
//   working           1200 (14%) ████████
//   buffer              0 ( 0%)
// -----
//   TOTAL              8750
//   Remaining          3250

// Take a snapshot for history
tracker.snapshot();

```

26.6. Key Sources

- Chapter 10: Token Budget Architecture (12,000 token rule, allocation breakdown)

26. *Appendix B: Token Tracker Implementation*

- Chapter 11: Compression Strategies (priority order: dedupe, summarise, truncate, drop)
- Chapter 12: Skill Architecture (skill activation/deactivation lifecycle)

27. Appendix C: Skill Catalog Schema

This appendix defines the complete schema for skill catalogs used in context-engineered AI agent systems. The schema covers the lightweight catalog format (loaded at startup) and the full SKILL.md format (loaded on activation).

27.1. Catalog Entry Schema (YAML)

The catalog is the minimal representation loaded at startup, consuming approximately 200-300 tokens for 20 skills.

```
# skill-catalog.schema.yaml
type: object
required: [skills]
properties:
  skills:
    type: array
    items:
      type: object
      required: [name, triggers, tier]
      properties:
        name:
          type: string
          description: "Unique skill identifier (kebab-case)"
```

```

    pattern: "^[a-z][a-z0-9-]*$"
    examples: ["web-search", "terrapih-memory", "t

triggers:
  type: array
  items:
    type: string
  minItems: 2
  maxItems: 10
  description: "Keywords that activate this skill

tier:
  type: string
  enum: [core, standard, optional]
  description: >
    Priority tier for activation and eviction.
    core: always preferred, last to evict.
    standard: normal priority.
    optional: first to evict under pressure.

estimated_tokens:
  type: integer
  minimum: 100
  maximum: 5000
  description: "Expected token cost when fully lo

dependencies:
  type: array
  items:
    type: string
  description: "Other skills required (loaded to

auto_deactivate:
  type: integer

```

```

    minimum: 1
    default: 10
    description: "Turns of non-use before automatic evi

compression_priority:
  type: integer
  minimum: 1
  description: "Drop order when budget is exceeded (1

```

27.2. Example Catalog

A production catalog for an AI coding assistant:

```

skills:
  # Core skills (always preferred)
  - name: file-editor
    triggers: ["edit", "modify", "change", "update", "write",
    tier: core
    estimated_tokens: 600

  - name: code-runner
    triggers: ["run", "execute", "command", "bash", "shell",
    tier: core
    estimated_tokens: 400

  - name: terraphim-memory
    triggers: ["memory", "recall", "remember", "kg", "knowled
    tier: core
    estimated_tokens: 1200

  # Standard skills
  - name: web-search

```

27. Appendix C: Skill Catalog Schema

```
triggers: ["search", "web", "find", "lookup", "google"]
tier: standard
estimated_tokens: 800

- name: git-operations
  triggers: ["git", "commit", "branch", "merge", "push"]
  tier: standard
  estimated_tokens: 700

- name: database-query
  triggers: ["database", "sql", "query", "table", "schema"]
  tier: standard
  estimated_tokens: 900

- name: test-runner
  triggers: ["test", "spec", "assert", "coverage", "jest"]
  tier: standard
  estimated_tokens: 500

- name: documentation
  triggers: ["docs", "readme", "document", "explain", "help"]
  tier: standard
  estimated_tokens: 600

# Optional skills (first to evict)
- name: image-processor
  triggers: ["image", "picture", "screenshot", "diagram"]
  tier: optional
  estimated_tokens: 1000
  compression_priority: 1

- name: calendar
  triggers: ["calendar", "meeting", "schedule", "event"]
  tier: optional
```



```
estimated_tokens: 500
compression_priority: 2
```

27.3. SKILL.md Schema

The full skill definition loaded when a skill is activated:

```
# [Skill Name]

## Identity
- Name: [kebab-case identifier]
- Tier: [core | standard | optional]
- Version: [semver]

## Triggers
- [trigger word 1]
- [trigger word 2]
- [trigger word N]

## Description
[What this skill does, in 1-3 sentences]

## Dependencies
- [other-skill-name]
- None

## Configuration
- [key]: [value]

## Tools Required
- [Tool 1 description]
- [Tool 2 description]
```

```
## Prompt Template
[The actual prompt or instructions used when this skill is invoked]

## Examples
[Input/output examples showing expected behaviour]
```

27.3.1. Constraints

Field	Required	Constraint
Name	Yes	Must match catalog entry name
Tier	Yes	Must match catalog entry tier
Version	Yes	Semantic versioning (MAJOR.MINOR.PATCH)
Triggers	Yes	Must be superset of catalog triggers
Description	Yes	Max 500 characters
Dependencies	No	Must reference valid skill names
Configuration	No	Key-value pairs only
Tools Required	No	Informational (not enforced)
Prompt Template	Yes	The core content loaded into context
Examples	No	Recommended for complex skills

27.4. Tier Behaviour

Property	Core	Standard	Optional
Activation priority	First	Second	Third
Eviction order	Last	Middle	First
Max simultaneous	3	2	1
Budget allocation	60%	30%	10%
Auto-deactivate	20 turns	10 turns	5 turns

27.5. Trigger Design Guidelines

1. **Minimum 2 triggers** per skill (avoids single-point activation failures)
2. **Maximum 10 triggers** per skill (more causes false positives)
3. **No single-character triggers** (too generic)
4. **Minimise overlap** between skills (if unavoidable, tier priority resolves conflicts)
5. **Include synonyms** (e.g., “search”, “find”, “lookup” for the same skill)
6. **Include domain terms** (e.g., “kubernetes”, “k8s”, “kubectl” for a K8s skill)

27.6. Key Sources

- Chapter 12: Skill Architecture for AI Agents (catalog structure, activation patterns)
- Chapter 10: Token Budget Architecture (budget allocation per type)
- Terraphim knowledge graph configuration (role-based skill definitions)
- OpenClaw skill loading patterns (production implementation)

28. Appendix D: Knowledge Graph Schema

This appendix documents the schema formats used for knowledge graphs throughout this book, with concrete examples from the Terraphim knowledge graph configuration.

28.1. Thesaurus Format

Terraphim uses a JSON-based thesaurus format for defining role-specific vocabularies. Each role gets its own thesaurus file, containing term-to-concept mappings with synonym group identifiers.

```
{
  "name": "Terraphim AI Developer",
  "data": {
    "learning": { "id": 1, "nterm": "knowledge" },
    "knowledge": { "id": 1, "nterm": "knowledge" },
    "lesson": { "id": 1, "nterm": "knowledge" },
    "insight": { "id": 1, "nterm": "knowledge" },
    "discovery": { "id": 1, "nterm": "knowledge" },
    "setup": { "id": 2, "nterm": "configuration" },
    "configuration": { "id": 2, "nterm": "configuration" },
    "install": { "id": 2, "nterm": "configuration" },
    "deployment": { "id": 2, "nterm": "configuration" },
    "initialise": { "id": 2, "nterm": "configuration" }
  }
}
```

```
}  
}
```

28.1.1. Field Definitions

Field	Type	Description
name	String	Role name – determines which synonym set is active
data	Object	Term-to-concept mapping
data[term]	Object	A searchable term (surface form)
data[term].id	Integer	Synonym group identifier – terms with the same id are treated as synonyms
data[term].nterm	String	Normalised term (canonical form) – the concept this term maps to

28.1.2. How Synonym Groups Work

All terms sharing the same id are compiled into the same Aho-Corasick automaton pattern group. When any term in the group is matched, the system treats it as a match for the canonical nterm.

Example: searching for “lesson” matches the knowLedge concept (id: 1), which also matches documents containing “learning”, “in-sight”, or “discovery”.

28.2. Role Definitions

Each role specifies a focus area, embedding strategy, and search depth:

Role	Focus	Embeddings	Depth
Terraphim AI Developer	Rust-based AI assistant development	local	deep
OpenClaw Developer	TypeScript gate-way/agent platform	local	standard
Last30Days Skill Developer	Web research skill development	none	basic
System Operator	System administration and operations	none	minimal
Learning Capture	Personal knowledge capture	local	deep
Terraphim Engineer (Default)	General engineering	local	deep

28.2.1. Role Configuration

```
roles:
  - name: "Terraphim AI Developer"
```

```
focus: "Rust-based AI assistant"
embeddings: local
depth: deep
thesaurus: "terraphim-ai-developer.json"
haystacks:
  - path: "~/terraphim-ai"
    type: "code"
  - path: "~/.config/terraphim/docs"
    type: "documentation"

- name: "System Operator"
  focus: "System administration"
  embeddings: none
  depth: minimal
  thesaurus: "system-operator.json"
  haystacks:
    - path: "/etc"
      type: "config"
```

28.3. Synonym Group Reference

A production knowledge graph typically defines 15-25 synonym groups covering the domain vocabulary. The following groups illustrate the pattern:

ID	Canonical Term	Synonyms	Domain
1	Knowledge	learning, lesson, insight, discovery	General

ID	Canonical Term	Synonyms	Domain
2	Configuration	setup, install, deployment, initialise	DevOps
3	Git	gitea, github, repository, version control	SCM
4	AI Assistant	terrapih, agent, claude, kimi	AI
5	Tool	skill, capability, function, feature	Platform
6	Storage	memory, persistence, database	Infrastructure
7	Context Management	arscontexta, onecontext, context, progress	CE
8	Retrieval	search, query, find, lookup	Search
9	Failure	error, mistake, bug, issue, problem	Debugging
10	Documentation	docs, guide, manual, readme	Docs
11-13	Languages	Rust/Cargo, JavaScript/Node, Python	Languages
14	Testing	test, spec, assertion, coverage	QA

ID	Canonical Term	Synonyms	Domain
15	Security	auth, encryption, vulnerability, CVE	Security
16	Performance	latency, throughput, optimisation, benchmark	Performance
17	Build	compile, bundle, package, release	Build
18	Execution	run, execute, invoke, spawn	Runtime
19	Debugging	debug, trace, log, inspect, breakpoint	Debugging
20	Repair	fix, patch, hotfix, workaround	Maintenance

28.4. Graph Schema (Property Graph Format)

For systems using property graphs (Neo4j, Memgraph), the following schema defines the node and relationship types:

```
// Node types
(:Concept {id: INT, label: STRING, definition: STRING})
(:Term {surface_form: STRING, language: STRING})
(:Document {path: STRING, type: STRING, last_modified: DA
```

```
(:Role {name: STRING, focus: STRING, depth: STRING})

// Relationship types
(:Term)-[:IS_SYNONYM_OF {group_id: INT}]→(:Concept)
(:Concept)-[:RELATES_TO {type: STRING, weight: FLOAT}]→(:Concept)
(:Document)-[:MENTIONS {position: INT, count: INT}]→(:Concept)
(:Role)-[:HAS_VOCABULARY]→(:Concept)
(:Role)-[:SEARCHES]→(:Document)
```

28.5. Graph Schema (RDF/Turtle Format)

For systems using triplestores, the equivalent schema in Turtle notation:

```
@prefix ce: <http://context-engineering.org/schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .

ce:Concept a rdfs:Class ;
    rdfs:label "A domain concept in the knowledge graph" .

ce:Term a rdfs:Class ;
    rdfs:label "A surface form (searchable text) for a concept" .

ce:isSynonymOf a rdf:Property ;
    rdfs:domain ce:Term ;
    rdfs:range ce:Concept ;
    rdfs:label "Maps a surface form to its canonical concept" .

ce:relatesTo a rdf:Property ;
    rdfs:domain ce:Concept ;
    rdfs:range ce:Concept ;
    rdfs:label "Semantic relationship between concepts" .
```

```
ce:hasRole a rdf:Property ;  
    rdfs:domain ce:Concept ;  
    rdfs:range ce:Role ;  
    rdfs:label "Concept is part of a role-specific vocabulary"
```

As discussed in Chapter 19, the choice between property graphs and triplestores is an implementation detail. Both representations encode the same semantic information. The important decisions are about the ontology quality, synonym coverage, and relationship completeness – not the storage format.

28.6. Key Sources

- Terraphim knowledge graph configuration (production implementation)
- SKOS (Simple Knowledge Organisation System) specification
- Neo4j graph data modelling documentation
- RDF 1.1 Turtle specification

29. Appendix E: Context Health Checklist

This appendix provides a practical checklist for assessing the health of a context engineering system. Use it during periodic reviews (weekly or per-sprint) and at quality gates before major releases.

29.1. Structural Health

These checks assess whether the foundational context artefacts are complete and consistent.

29.1.1. Domain Model

- ☐ Domain model exists and is versioned
- ☐ All key domain terms are defined with unambiguous definitions
- ☐ Synonym groups are documented (term X = term Y in this context)
- ☐ Relationships between concepts are explicitly typed (not just “related to”)
- ☐ Domain model has been reviewed by a domain expert (not just the engineer who wrote it)
- ☐ No orphan terms (terms defined but not used in any artefact)

29. Appendix E: Context Health Checklist

- ☐ No undefined terms (terms used in artefacts but not in the domain model)

29.1.2. Knowledge Graph

- ☐ Knowledge graph is populated and queryable
- ☐ Term coverage: >80% of domain model terms appear in the KG
- ☐ Synonym coverage: key terms have at least 2-3 synonyms defined
- ☐ Relationship coverage: key entity pairs have typed relationships
- ☐ Role-based graphs are configured for each active context role
- ☐ KG build time is acceptable (<1s for graphs under 100K terms)
- ☐ Search results are deterministic (same query returns same results)

29.1.3. Vocabulary

- ☐ Vocabulary file exists and is maintained
- ☐ Industry-standard terms are included (not just internal jargon)
- ☐ Acronyms are expanded in the vocabulary
- ☐ Vocabulary is validated against actual usage in codebase/documents

29.2. Operational Health

These checks assess whether the context system is functioning correctly in production.

29.2.1. Retrieval Quality

- ☐ Retrieval accuracy is measured (% of queries returning relevant results)
- ☐ Precision at top-5 results is tracked
- ☐ Zero-result queries are logged and investigated
- ☐ Retrieval latency is within acceptable bounds (<100ms for deterministic, <2s for semantic)

29.2.2. Context Freshness

- ☐ Knowledge graph was updated within the last sprint/iteration
- ☐ No stale entries (documents referenced but no longer existing)
- ☐ New code/documents added since last KG update are identified
- ☐ Temporal metadata (last_updated, source_date) is present on KG entries

29.2.3. Drift Detection

- ☐ Baseline context snapshot exists
- ☐ Drift is measured at regular intervals
- ☐ Vocabulary drift: new terms appearing in usage that are not in the KG
- ☐ Assumption drift: conventions established in recent sessions that contradict earlier ones
- ☐ Relationship drift: entity connections that have changed meaning
- ☐ Drift threshold triggers are defined (e.g., >15% = warning, >30% = critical)

29.3. Governance Health

These checks assess whether the context management process is sound.

29.3.1. Change Management

- ☐ Context changes are tracked (who changed what, when, why)
- ☐ Impact analysis is performed before significant changes
- ☐ Changes to the domain model require review/approval
- ☐ Vocabulary additions follow a defined process (not ad-hoc)

29.3.2. Quality Gates

- ☐ Pre-LLM validation is automated (syntax, semantics, grounding)
- ☐ Post-LLM validation checks outputs against KG
- ☐ Gate pass rates are tracked over time
- ☐ Failed gates produce actionable feedback (not just “FAILED”)

29.3.3. Documentation

- ☐ Context architecture is documented (which systems, how they connect)
- ☐ KG schema is documented (node types, relationship types, properties)
- ☐ Role definitions are documented (who uses which vocabulary)
- ☐ Runbook exists for common context operations (rebuild KG, add terms, fix drift)

29.4. Agent-Specific Health

These checks apply when AI agents are consuming the context.

29.4.1. Agent Context Configuration

- ☐ Each agent role has a defined context scope (what it can see)
- ☐ System prompts reference the knowledge graph for grounding
- ☐ CLAUDE.md (or equivalent) is maintained and current
- ☐ Skill definitions include context requirements
- ☐ Token budget is defined and monitored per agent

29.4.2. Multi-Agent Coherence

- ☐ Shared vocabulary is consistent across agents
- ☐ Handover documents are generated between agent sessions
- ☐ No contradictory conventions established by different agents
- ☐ Agent outputs are validated against the shared domain model

29.4.3. Feedback Loop

- ☐ Failed agent actions are captured for learning
- ☐ Agent queries with zero results are logged
- ☐ Context gaps discovered by agents are fed back to the KG
- ☐ Agent performance metrics are tracked over time

29.5. Scoring Guide

For each section, count the number of checked items and calculate a percentage:

Score	Status	Action
90-100%	Healthy	Continue monitoring
70-89%	Adequate	Address gaps in next sprint
50-69%	At Risk	Prioritise context improvements
Below 50%	Critical	Stop feature work, fix context foundations

29.6. Recommended Review Cadence

Check Category	Frequency	Typical Duration
Retrieval quality spot-check	Daily (automated)	0 min (CI/CD)
Drift detection	Weekly (automated)	0 min (scheduled)
Full structural health	Per sprint	30-60 min
Full governance review	Monthly	1-2 hours
Agent coherence audit	Per major release	2-4 hours

29.7. Key Sources

- Chapter 14: Validation and Quality Gates (KLS protocol, drift detection)
- Chapter 18: Enterprise Context Governance (gated lifecycle, health metrics)
- Chapter 7: Terraphim Graph Embeddings (deterministic search properties)

- Chapter 15: Multi-Agent Context Management (coherence patterns)

30. Appendix F: Resources and Tools

This appendix compiles the tools, frameworks, standards, and references cited throughout this book.

30.1. Knowledge Graph Tools

Tool	Type	Description	Chapters
Ter-raphim	Determin-istic search engine	Aho-Corasick automaton-based search with role-based knowledge graphs. Sub-millisecond matching across 1.4M+ patterns.	6, 7, 9, 19
Neo4j	Graph database	Property graph database with Cypher query language. Market leader for enterprise knowledge graphs.	5, 19, 21

Tool	Type	Description	Chapters
Mem-graph	Graph database	In-memory property graph compatible with Cypher. Optimised for real-time queries.	19, 21
Apache Jena	RDF frame-work	Java framework for building Semantic Web and Linked Data applications. Supports SPARQL.	5, 19
Trust-Graph	Graph store	Apache Cassandra-backed graph store designed for billion-node knowledge graphs with trust scoring.	19, 21

30.2. Embedding and RAG Tools

Tool	Type	Description	Chapters
Pinecone	Vector database	Managed vector database for similarity search. Cloud-hosted.	6
Weaviate	Vector database	Open-source vector database with built-in vectorisation.	6

Tool	Type	Description	Chapters
Chroma	Vector database	Lightweight embedding database for AI applications.	6
LangChain	RAG framework	Framework for building LLM applications with retrieval pipelines.	19
Llamaindex	RAG framework	Data framework for connecting LLMs to external data sources.	19

30.3. AI Models and APIs

Model/Service	Provider	Relevance	Chapters
Claude	Anthropic	Primary LLM for context-engineered agents. 200K token context window.	10, 13, 15
GPT-4	OpenAI	128K context window. Token budget comparison baseline.	10

Model/Service	Provider	Relevance	Chapters
BERT	Google	Base embedding model (768 dimensions). Reference for embedding discussion.	6
text-embedding-3	OpenAI	Embedding model (up to 3072 dimensions).	6
MedGemma	Google	Medical domain LLM. Used in KG grounding case study.	9

30.4. Standards and Specifications

Standard	Organisation	Description	Chapters
RDF 1.1	W3C	Resource Description Framework for expressing data as triples.	5, 19, App D
OWL	W3C	Web Ontology Language for formal ontology definitions.	5, 19

Standard	Organisation	Description	Chapters
SKOS	W3C	Simple Knowledge Organisation System for vocabularies and thesauri.	5, 8, App D
SPARQL	W3C	Query language for RDF data.	5, 19
Cypher	Neo4j/GQL	Graph query language for property graphs.	5, 19
JSON-LD	W3C	JSON-based Linked Data serialisation format.	19
Turtle	W3C	Terse RDF Triple Language. Compact RDF serialisation.	19, App D

30.5. Algorithms

Algorithm	Description	Chapters
Aho-Corasick	Multi-pattern string matching in linear time. Foundation for Terraphim's deterministic search.	6, 7

Algorithm	Description	Chapters
PageRank	Graph centrality algorithm for ranking nodes by connectivity. Applicable to KG result ranking.	7, 21
Cosine Similarity	Vector similarity measure used in embedding-based search.	6
TF-IDF	Term frequency-inverse document frequency for classical text ranking.	6

30.6. Development Tools

Tool	Purpose	Chapters
Claude Code	AI coding assistant with CLAUDE.md support. Reference implementation for context engineering patterns.	13, 16
OpenClaw	AI assistant platform with layered memory, skill system, and autonomous heartbeat loops.	17

Tool	Purpose	Chapters
Quarto	Scientific publishing system. Used to produce this book.	–

30.7. Context Engineering Frameworks

Framework	Description	Chapters
Progressive Disclosure	UI design pattern applied to context: show only what is needed, reveal more on demand.	10, 12
Constitutional Context	Files that define AI identity and must never be compressed or dropped.	10, 17
KLS (Knowledge Layer Schema)	Custom validation protocol for AI-generated content across conceptual, structural, and semantic dimensions.	14
Token Budget Architecture	Systematic approach to allocating context window tokens across categories.	10, App B

Framework	Description	Chapters
Hybrid Pipeline	Retrieval architecture combining deterministic, semantic, and graph methods.	19

30.8. Research Papers and Articles

Reference	Relevance	Chapters
Karpathy, A. (2025). “Vibe Coding” and LLM coding guidelines.	Operational discipline for AI-assisted development.	3, 13
Hrishi & Prithvi (Anthropic). Build-review codon loops.	Multi-agent coordination with context walls.	15
FLAURA Trial. Osimertinib 80mg for EGFR-mutated NSCLC.	Reference for KG grounding safety case study.	9
“Some Simple Economics of AGI” analysis. Verification bottleneck.	Economics of AI verification at scale.	1, 22
LLM-as-a-Judge best practices.	Evaluation methodology: ensemble judging, bias cataloguing, calibration.	14, 22

Reference	Relevance	Chapters
Context Graphs: From RAG to Autonomous Learning.	Eight-step progression from basic RAG to self-describing context stores.	5, 19

30.9. Clinical References

Reference	Relevance	Chapters
SNOMED CT	Systemised Nomenclature of Medicine. 1.4M+ clinical terms used in Terraphim healthcare demo.	6, 7, 9
FLAURA Trial	Phase III clinical trial establishing Osimertinib 80mg daily for first-line EGFR-mutated NSCLC treatment.	9

30.10. Key Sources

- Individual chapter “Key Sources” sections
- Terraphim AI technical documentation
- W3C Semantic Web standards

30. *Appendix F: Resources and Tools*

- Neo4j graph data science documentation

31. Appendix G: Glossary

Key terms used throughout this book, organised alphabetically. Terms marked with (KG) appear in Terraphim knowledge graph synonym groups. Terms marked with (Ch N) reference the chapter where they are primarily defined.

31.1. A

Aho-Corasick Automaton (Ch 7) A finite-state pattern matching algorithm that finds all occurrences of a set of patterns in a single scan through the text. Used by Terraphim for deterministic term matching across millions of patterns in sub-millisecond time. Linear time complexity $O(n)$ regardless of pattern count.

AI Sandwich (Ch 22) A firm structure where AI handles generation (measurable work), humans handle verification (unmeasurable work), and AI handles final polish. The middle layer does not scale, creating a structural bottleneck.

Artefact (KG) A versioned, reviewable output of a development process – code, documents, configurations, domain models. In context engineering, artefacts include knowledge graphs, vocabularies, and skill definitions.

31.2. C

Cascade Pattern (Ch 19) A hybrid pipeline routing pattern that tries the cheapest retrieval method first and escalates to more expensive methods only when confidence is low. Optimises for cost.

CLAUDE.md (Ch 13) A project-level configuration file that defines rules, conventions, and context for AI coding assistants. Acts as persistent context that survives across sessions.

Codifier's Curse (Ch 22) The phenomenon where codifying expertise into AI systems destroys the tacit knowledge that was never explicitly documented. The system works until it encounters edge cases where that tacit knowledge mattered.

Compression (Ch 11) Techniques for reducing context size while preserving meaning. Includes summarisation, extraction, and structured representation.

Connectivity-Based Ranking (Ch 7) A scoring method where documents are ranked by the strength and number of their connections to query terms in the knowledge graph. More connections through typed relationships yield higher scores.

Context Drift (Ch 14) The gradual misalignment between a system's knowledge representation and the actual state of the domain. Types include vocabulary drift, assumption drift, relationship drift, and goal drift.

Context Engineering (Ch 2) The discipline of managing the information that AI systems use to generate responses. Encompasses knowledge representation, retrieval, validation, and governance.

Context Graph (Ch 19) A triples-representation of data optimised for usage with AI. Represents the next evolution beyond basic RAG, pulling from graph theory, semantic web standards, and ontology work.

31.3. D

Deterministic Precision (Ch 6, Ch 7) Synonym for **100% accuracy for known terms**. The property of a search system that guarantees every vocabulary-matched term is found with zero false negatives and zero false positives. Achieved by Terraphim through Aho-Corasick automata operating on a pre-defined vocabulary. The scope is limited to terms that exist in the knowledge graph; for unknown terms, hybrid pipelines (Ch 19) provide coverage.

Deterministic Search (Ch 6) Search where the same query always produces the same results. Contrasted with probabilistic/semantic search where results may vary.

Domain Model (Ch 4) A formal representation of the concepts, relationships, and rules within a specific problem domain. The foundation for knowledge graphs and context engineering.

Drift Detection (Ch 14) Monitoring for changes in context alignment over time. Includes vocabulary drift (new terms appearing), assumption drift (conventions changing), and relationship drift (connections shifting).

31.4. E

Embedding (Ch 6) A numerical vector representation of text, used for semantic similarity search. Terraphim's "graph embeddings" are deterministic alternatives that preserve full graph structure rather than compressing into vectors.

Ensemble Judging (Ch 22) Using multiple LLM judges and aggregating their scores to reduce single-model bias in automated evaluation.

Explainability (Ch 7) The ability to trace exactly why a search result was returned – which terms matched, at what positions,

through which synonym expansions, and along which relationship paths. A key advantage of deterministic search over vector similarity.

31.5. F

Fan-Out Pattern (Ch 19) A hybrid pipeline routing pattern that runs multiple retrieval methods in parallel and merges results. Documents found by multiple methods receive boosted scores. Optimises for recall.

31.6. G

Grounding (Ch 9) Verifying that AI-generated claims are supported by facts in the knowledge graph. A safety gate that prevents hallucination.

GraphRAG (Ch 19) An evolution of basic RAG that uses knowledge graph representations with rich relationships, moving beyond text chunks and semantic similarity.

31.7. H

Haystack (KG) In Terraphim, a collection of documents indexed for search. Each role can search across different haystacks (local folders, repositories, documentation).

Hook (Ch 16) A script or function that executes automatically in response to events (pre-commit, post-tool-use, etc.). Used to automate context validation, knowledge capture, and safety checks.

Hybrid Pipeline (Ch 19) A retrieval architecture that combines deterministic, semantic, and graph-based methods, routing queries to the most appropriate strategy based on their characteristics.

31.8. K

KLS (Knowledge Layer Schema) (Ch 14) A structured validation protocol for AI-generated content. Validates across conceptual, structural, technical, semantic, and style dimensions.

Knowledge Graph (Ch 5) A data model for organising knowledge as entities (nodes) connected by typed relationships (edges). Distinct from the database that stores it.

31.9. L

LLM-as-Judge (Ch 22) Using a large language model to evaluate the quality of AI-generated outputs against defined rubrics. Offers ~80% of human evaluation accuracy at ~1% of cost.

31.10. M

Measurability Bias (Ch 22) The tendency for AI automation to target tasks that are easy to verify, not necessarily tasks that are simple. Creates selection bias where measurable work is automated while unmeasurable but critical work is neglected.

Missing Junior Loop (Ch 22) The structural problem caused by AI automating junior engineering roles, which historically served as the training pipeline for developing senior expertise.

31.11. O

Ontology (Ch 5) A machine-readable specification of concepts, their hierarchies, and relationships. More formal than a vocabulary or taxonomy. Key standards include OWL and SKOS.

OntologyRAG (Ch 19) An evolution of GraphRAG that uses structured ontologies for precision, improved recall, and more granular retrieval.

31.12. P

Property Graph (Ch 5, Ch 19) A graph model where nodes and edges can have properties (key-value pairs). Used by Neo4j, Memgraph. Queried with Cypher. Contrasted with RDF triple-stores.

31.13. R

RAG (Retrieval-Augmented Generation) (Ch 19) A technique where relevant documents are retrieved and included in an LLM prompt to ground responses in factual context.

RDF (Resource Description Framework) (Ch 19) A W3C standard for expressing data as subject-predicate-object triples. Uses URIs for global identification. Serialisation formats include Turtle, JSON-LD, and N-Triples.

Role-Based Graph (Ch 7) A knowledge graph where the same entities can have different synonym edges and relationships depending on the active context role. Enables the same term to mean different things in different contexts.

Rolegraph (KG) Terraphim's implementation of role-based graphs. Each rolegraph handles ingestion and ranking via Aho-Corasick automata, with search inference in 5-10 nanoseconds.

31.14. S

Semantic Enrichment Pipeline (Ch 19) A hybrid pattern that uses deterministic search to establish a precise foundation, then uses semantic methods to detect and fill conceptual gaps.

Skill (Ch 12) A self-contained capability that can be attached to an AI agent. Includes a prompt template, tool definitions, and context requirements.

Synonym Group (Ch 7, Appendix D) A set of terms that are treated as equivalent for search purposes. All terms in a group map to a single canonical concept via a shared group identifier.

31.15. T

Temporal Context (Ch 19, Ch 22) Information about when facts were established, how long they have been corroborated, and how their validity changes over time. Freshness and recency are not the same as accuracy and precision.

Thesaurus (Appendix D) In Terraphim, a JSON file mapping surface forms (searchable terms) to canonical concepts via synonym group identifiers. Each role has its own thesaurus.

Token Budget (Ch 10) The allocation strategy for distributing available context window tokens across system prompt, user context, retrieved documents, and generation space.

Triple (Ch 5, Ch 19) The fundamental unit of knowledge graph data: Subject-Predicate-Object. Predicates encode richer information than simple verb-based connections.

31.16. V

Validation Gate (Ch 14) A checkpoint where AI-generated content must pass quality criteria before progressing. Gates can be automated (CI/CD) or manual (human review).

Vector Embedding (Ch 6) A dense numerical representation of text in high-dimensional space. Used for approximate nearest-neighbour search. Contrasted with Terraphim's deterministic graph embeddings.

Verification Bottleneck (Ch 22) The fundamental economic constraint where verification costs grow faster than generation costs, creating pressure to deploy unverified AI outputs.

Vocabulary (Ch 8) A human-readable collection of defined terms for a domain. Less formal than an ontology, more structured than ad-hoc terminology.

31.17. Key Sources

- Chapters 1-22 of this book
- Appendix D: Knowledge Graph Schema (thesaurus format, synonym groups)
- Terraphim AI technical documentation
- SKOS (Simple Knowledge Organisation System) specification
- "Some Simple Economics of AGI" (verification bottleneck terminology)

A. Appendix A: Context Optimizer Skill (SKILL.md)

This appendix provides a reference implementation for a **context optimizer skill** – a self-contained module that manages token budgets, skill activation, and compression within an AI agent system.

The skill follows the architecture described in Chapter 12 and the token budget principles from Chapter 10.

A.1. SKILL.md Format

Every skill in a context-engineered system is defined by a `SKILL.md` file. This is both human-readable documentation and machine-parseable configuration.

```
# Context Optimizer

## Identity
- Name: context-optimizer
- Tier: core
- Version: 1.0.0

## Triggers
- context
- tokens
```

- budget
- compress
- optimize

Description

Manages token budget allocation, skill activation/deactivation and context compression. Runs automatically when context exceeds warning thresholds.

Dependencies

- None (this skill is self-contained)

Configuration

- max_active_skills: 5
- token_warning_threshold: 12000
- token_critical_threshold: 15000
- auto_deactivate_after: 10 # turns

Tools Required

- File read (for loading skill content)
- Token counter (for budget tracking)

A.2. Reference Implementation

A.2.1. Skill Loader

The skill loader reads SKILL.md files and parses them into a structured format:

```
function parseSkillMd(content) {  
  const sections = {};  
  let currentSection = null;
```



```

for (const line of content.split("\n")) {
  if (line.startsWith("## ")) {
    currentSection = line.slice(3).trim().toLowerCase();
    sections[currentSection] = [];
  } else if (currentSection && line.trim()) {
    sections[currentSection].push(line.trim());
  }
}

return {
  name: extractField(sections, "identity", "Name"),
  tier: extractField(sections, "identity", "Tier"),
  version: extractField(sections, "identity", "Version"),
  triggers: extractList(sections, "triggers"),
  description: sections["description"]?.join(" ") ?? "",
  dependencies: extractList(sections, "dependencies"),
  config: extractKeyValues(sections, "configuration")
};
}

function extractField(sections, section, field) {
  const line = sections[section]?.find(l => l.startsWith(`${field}-`));
  return line?.split(":").slice(1).join(":").trim() ?? null;
}

function extractList(sections, section) {
  return (sections[section] ?? [])
    .filter(l => l.startsWith("- "))
    .map(l => l.slice(2).trim());
}

function extractKeyValues(sections, section) {
  const config = {};
  for (const line of sections[section] ?? []) {

```

```
    if (line.startsWith("- ") && line.includes(":")) {
      const [key, ...rest] = line.slice(2).split(":");
      const value = rest.join(":").trim();
      config[key.trim()] = isNaN(value) ? value : Number(
    }
  }
  return config;
}
```

A.2.2. Context Optimizer Core

The optimizer manages the full lifecycle: activation, budget tracking, compression, and deactivation.

```
class ContextOptimizer {
  constructor(skillCatalog, config = {}) {
    this.catalog = skillCatalog;
    this.activeSkills = new Map();
    this.turnCounter = 0;

    this.maxActive = config.max_active_skills ?? 5;
    this.warnAt = config.token_warning_threshold ?? 12000;
    this.criticalAt = config.token_critical_threshold ?? 12000;
    this.deactivateAfter = config.auto_deactivate_after ?? 12000;
  }

  // Main entry point: called each turn
  async optimizeContext(userQuery, currentContext) {
    this.turnCounter++;

    // 1. Determine which skills should be active
    const candidates = this.findCandidates(userQuery);
```

```

// 2. Activate new skills (up to limit)
for (const candidate of candidates) {
  if (!this.activeSkills.has(candidate.name)) {
    await this.activate(candidate);
  } else {
    this.activeSkills.get(candidate.name).lastUsedTurn = ...
  }
}

// 3. Deactivate stale skills
this.deactivateStale();

// 4. Enforce limit
this.enforceLimits();

// 5. Check budget
const budget = this.calculateBudget(currentContext);

if (budget.total > this.criticalAt) {
  this.compress(currentContext);
}

return {
  activeSkills: [...this.activeSkills.values()],
  budget,
  compressed: budget.total > this.warnAt
};
}

findCandidates(query) {
  const q = query.toLowerCase();
  return this.catalog
    .filter(skill => skill.triggers.some(t => q.includes(t)))
    .sort((a, b) => {

```

```

        const tierOrder = { core: 0, standard: 1, optional: 2 };
        return (tierOrder[a.tier] ?? 2) - (tierOrder[b.tier] ?? 2);
    });
}

async activate(skill) {
    const content = await loadSkillContent(skill.name);
    this.activeSkills.set(skill.name, {
        ...skill,
        content,
        tokens: estimateTokens(content),
        activatedTurn: this.turnCounter,
        lastUsedTurn: this.turnCounter
    });
}

deactivateStale() {
    for (const [name, skill] of this.activeSkills) {
        if (this.turnCounter - skill.lastUsedTurn > this.deactivationLimit) {
            this.activeSkills.delete(name);
        }
    }
}

enforceLimits() {
    if (this.activeSkills.size >= this.maxActive) return;

    const sorted = [...this.activeSkills.entries()]
        .sort(([a], [b]) => {
            const tierOrder = { core: 0, standard: 1, optional: 2 };
            const tierDiff = (tierOrder[a.tier] ?? 2) - (tierOrder[b.tier] ?? 2);
            if (tierDiff !== 0) return tierDiff;
            return b.lastUsedTurn - a.lastUsedTurn;
        });
}

```

```

    // Keep top N, remove the rest
    const toRemove = sorted.slice(this.maxActive);
    for (const [name] of toRemove) {
        this.activeSkills.delete(name);
    }
}

calculateBudget(context) {
    const constitutional = context.constitutionalTokens ?? 20;
    const catalog = estimateTokens(JSON.stringify(this.catalog));
    const skills = [...this.activeSkills.values()]
        .reduce((sum, s) => sum + s.tokens, 0);
    const memory = context.memoryTokens ?? 0;
    const working = context.workingTokens ?? 0;

    return {
        constitutional,
        catalog,
        skills,
        memory,
        working,
        total: constitutional + catalog + skills + memory + working;
    };
}

compress(context) {
    // Priority: drop optional skills first, then summarise mandatory
    const optional = [...this.activeSkills.entries()]
        .filter(([_, s]) => s.tier === "optional");

    for (const [name] of optional) {
        this.activeSkills.delete(name);
    }
}

```

```
}
```

A.2.3. Usage Example

```
// Load catalog at startup (~200-300 tokens)
const catalog = await loadSkillCatalog("skills/catalog.yaml")

// Create optimizer
const optimizer = new ContextOptimizer(catalog);

// Each turn
const result = await optimizer.optimizeContext(
  userQuery,
  { constitutionalTokens: 2000, memoryTokens: 1500, workingMemoryTokens: 1000 }
);

console.log(`Active: ${result.activeSkills.map(s => s.name).join(", ")}
console.log(`Budget: ${result.budget.total} tokens`);
```

A.3. File Structure

A complete context optimizer skill lives in a single directory:

```
skills/
  context-optimizer/
    SKILL.md           # Skill definition (shown above)
    optimizer.js       # Core implementation
    tests/
      optimizer.test.js # Unit tests
```

A.4. Key Sources

- Chapter 10: Token Budget Architecture (budget allocation)
- Chapter 11: Compression Strategies (compression priority order)
- Chapter 12: Skill Architecture for AI Agents (catalog, activation, deactivation)
- OpenClaw context management (production patterns)

